

從 LightNobel 到 FPGA 記憶體優化教學手冊

蛋白質結構預測模型的 memory-bound 問題，以及在 AMD Alveo U55C 上的常見解法

專題 reference : LightNobel (ISCA 2025)

程式碼 : github.com/antony3911/ForClaude (分支 `claude/lightnobel-memory-optimization-yhokyc`)

產生日期 : 2026-09-28

目錄

導讀：如何使用本手冊

手冊的結構

建議的閱讀路線

閱讀慣例

最短上手路徑 (10 分鐘)

速覽：十分鐘看懂這本書

1. 一張圖看懂來龍去脈

2. 這本書的故事線

3. LightNobel 一分鐘版

4. 本專題一分鐘版

5. 概念地圖：每個概念在哪一章

第 1 章 蛋白質結構預測與 pair representation

1.1 蛋白質結構預測在做什麼？

1.2 全景：一次預測從頭到尾在算什麼

1.3 兩種 representation、activation 與 token

1.4 Triangle Multiplication 與 Triangle Attention

1.5 Triangle Multiplication 裡的每一個運算

本章小結

自我檢測

第 2 章 記憶體牆：容量、頻寬與效能分析

2.1 問題一：記憶體放不下

2.2 問題二：搬資料比計算慢 (memory-bound)

2.3 記憶體階層：為什麼「搬運」有成本

2.4 一條公式看懂所有優化

2.5 算術強度與 Roofline 模型

2.6 Amdahl's Law 與瓶頸轉移

2.7 解法的地圖：資料表示、資料流、硬體結構

本章小結

自我檢測

第 3 章 資料流優化：少搬、搬得快、不存中間值

3.1 Tiling／資料重用：減少搬運總量 ($v0 \rightarrow v1$)

3.2 寬位元與 burst：提高每次搬運的效率 ($v1 \rightarrow v2$)

3.3 Double buffering：把等待時間藏起來 ($v2 \rightarrow v3$)

3.4 多通道 HBM：多條路同時搬

3.5 運算融合：讓中間值不寫回記憶體

3.6 重算：用計算換記憶體

3.7 分塊、token-wise attention 與其他手法

每一批算完就丟，晶片內永遠只有 m 、 ℓ （和輸出累加 o ）。對 Triangle Attention 來說，被省掉的就是 $L \times L \times L$ 的 score matrix（公式見 6.6 節）。

3.8 總整理：每一招解決了什麼

本章小結

自我檢測

第 4 章 量化：讓每筆資料變小

4.1 量化的原理：壓縮，不是標準化

4.2 Per-token、per-tensor、per-channel

4.3 尺的成本與格式

4.4 尺在什麼時候乘？

4.5 位元數怎麼選

4.6 代價、算術強度與容量

本章小結

自我檢測

第 5 章 案例研究（上）：LightNobel 的觀察、演算法與成果

5.1 論文速讀卡

5.2 他們的觀察

5.3 AAQ：三組、混合精度與 outlier

5.4 記憶體是靠什麼省下來的？

5.5 領域地圖：大家用什麼方法處理這個問題

5.6 讀原文時的指引

本章小結

自我檢測

第 6 章 案例研究（下）：LightNobel 的硬體架構

6.1 從需求推導架構：設計者面對的四個問題

6.2 整體架構與資料流

6.3 Token Aligner：記憶體的格式和運算的格式不一樣

6.4 RMPU：用 4-bit 小乘法器拼出任何精度

6.5 VVPU：向量運算、排序與執行時量化

6.6 Token-wise multi-head attention：不存 L^3 的 score matrix

6.7 互連與記憶體：為什麼 crossbar 佔了 70% 的面積

6.8 決定要放幾個：硬體設計空間探索

6.9 硬體怎麼評估：每個工具在做什麼

6.10 如果要在 FPGA 上實作這些設計（我們的分析）

本章小結

自我檢測

第 7 章 FPGA 與工具原理：為何 C++ 可以變成電路？

7.1 FPGA 是什麼？

7.2 為什麼 FPGA 適合做記憶體優化的實驗？

7.3 HLS：C++ 是怎麼變成電路的？

7.4 為什麼 C++ 可以直接模擬？

7.5 Vivado：從 RTL 到 bitstream

7.6 Vitis 與 v++：把 kernel 接到板子上

7.7 XRT：host 程式怎麼控制 FPGA

7.8 Python 端：PyTorch、HuggingFace、ESMFold

7.9 誰提供了什麼能力？

7.10 Claude 在這個對話中呼叫的工具

本章小結

自我檢測

第 8 章 實作：v0~v4 從程式碼到報告

8.1 全景圖

8.2 步驟一：界定範圍

8.3 步驟二：資料排列（memory layout）與算式

8.4 步驟三：寫 5 個 kernel

8.5 步驟四：正確性驗證（C 模擬）

8.6 步驟五：host 程式

8.7 步驟六：HBM 連接設定

8.8 步驟七：Makefile

8.9 步驟八：真實資料管線（加分項）

8.10 步驟九：在實際 server 上除錯

8.11 步驟十：讀懂 HLS 報告（以 v2 為例）

8.12 所有檔案總表

本章小結

自我檢測

第 9 章 評估方法：architectural simulation、分析模型與設計空間探索

9.1 Architectural simulation 是什麼？

9.2 分析模型：用公式預測硬體表現

9.3 設計空間探索（DSE）與 Pareto 前緣

9.4 容量模型：最多能處理多長的序列

本章小結

自我檢測

第 10 章 專題策略：定位、設計、海報與評審問答

10.1 定位：題目、選題理由、和 LightNobel 的關係

10.2 策略：從「比較手法」到「拆解與設計」

- 10.3 融合設計：兩個 kernel 與效益
- 10.4 硬體細節：LayerNorm、量化在源頭完成、該用幾個 bit
- 10.5 海報、報告講法與評審問答
- 10.6 參考資料
- 本章小結
- 自我檢測

第 11 章 研究方法：之後可以重複使用的步驟

- 11.1 整體流程
- 11.2 步驟一：讀 paper 的方法
- 11.3 步驟二：量化問題
- 11.4 步驟三：縮小範圍
- 11.5 步驟四：設計實驗
- 11.6 步驟五：實作與驗證
- 11.7 步驟六：分析
- 11.8 步驟七：呈現（報告要畫的圖）
- 11.9 除錯的思路（以 2025.2 路徑 bug 為例）
- 11.10 本專題套用這個流程的完整範例
- 11.11 案例：讀原文後發現的三個誤判
- 本章小結
- 自我檢測

附錄 B 指令手冊：做過的、可以做的、格式與用途

- A.1 先看懂指令的格式
- A.2 Linux 基本操作
- A.3 環境調查與設定
- A.4 Git：取得與更新程式碼
- A.5 tmux：斷線也不會中斷的終端機
- A.6 本專案的 make 指令（最重要）
- A.7 Vitis 工具的原始指令（Makefile 背後在做什麼）
- A.8 看結果
- A.9 host 程式與 Python 腳本的參數
- A.10 目前為止實際執行過的指令（時間軸）
- A.11 常見錯誤與排查
- 本章小結
- 自我檢測

附錄 C 術語表

- B.1 蛋白質與模型
- B.2 效能與記憶體概念
- B.3 量化
- B.4 FPGA 硬體

B.5 HLS 與硬體設計
B.6 AMD 工具鏈
B.7 軟體與開發工具
B.8 架構評估與設計方法
本附錄小結

附錄 C 自我檢測參考答案

第 1 章 蛋白質結構預測與 pair representation
第 2 章 記憶體牆：容量、頻寬與效能分析
第 3 章 資料流優化
第 4 章 量化
第 5 章 LightNobel 的觀察、演算法與成果
第 6 章 LightNobel 的硬體架構
第 7 章 FPGA 與工具原理
第 8 章 實作：v0 ~ v4 從程式碼到報告
第 9 章 評估方法
第 10 章 專題策略
第 11 章 研究方法
附錄 A 指令手冊

導讀：如何使用本手冊

這份手冊寫給完全沒有 FPGA 經驗、但要以 LightNobel (ISCA 2025) 為起點做研究或專題的人。讀完後你應該能：說明問題的來源、解釋每個優化手法為什麼有效、看懂程式碼與 HLS 報告、自己下指令跑實驗，並用同樣的方法讀下一篇 paper。

手冊的結構

這本手冊依「問題 → 原理 → 案例 → 實作 → 專題」編排，像一本教科書：前面的章節是後面的基礎。（各部回答什麼問題，見「速覽」第 2 節。）

部分	章	內容	讀完能做到
速覽	—	來龍去脈、全書地圖、LightNobel 與本專題各一分鐘版	10 分鐘抓到全貌
第一部 問題	1 蛋白質結構預測與 pair representation	模型架構、activation、token、Triangle Multiplication / Attention	說明「模型在算什麼」
	2 記憶體牆	容量與頻寬問題、記憶體階層、時間公式、roofline、Amdahl、三層框架	說明「問題是什麼、怎麼分析」
第二部 原理	3 資料流優化	tiling、burst、double buffering、多通道、融合、重算、分塊	說明「不改格式，怎麼少搬」
	4 量化	原理、per-token、尺的成本與格式、位元數選擇	說明「怎麼讓資料變小」
第三部 案例	5 LightNobel：觀察、演算法與成果	速讀卡、AAQ、記憶體省在哪、領域地圖	讀懂一篇頂會論文的演算法
	6 LightNobel：硬體架構	RMPU、VVPVU、Token Aligner、crossbar、硬體 DSE	用硬體設計者的角度讀懂加速器
第四部 實作	7 FPGA 與工具原理	FPGA、HLS、Vivado、Vitis、XRT	知道每個工具在做什麼
	8 v0 ~ v4 從程式碼到報告	十個步驟、程式碼逐段解說、除錯、讀報告	看懂本專題的程式與報告
	9 評估方法	architectural simulation、分析模型、DSE、容量模型	評估還沒上板的設計
第五部 專題	10 專題策略	定位、策略、融合設計、v5、海報與評審問答	說明「我們的貢獻」
	11 研究方法	讀 paper、設計實驗、分析、誤判的案例	把方法用在下一篇 paper

部分	章	內容	讀完能做到
附錄	A 指令手冊	指令格式、Linux、Git、tmux、make、排錯	自己跑實驗
	B 術語表	所有專有名詞的白話解釋	查詢
	C 參考答案	各章「自我檢測」的答案	確認自己懂了

建議的閱讀路線

你的情況	路線
第一次接觸，想完整學會	速覽 → 1 → 2 → ... → 11，每章做完自我檢測
只有 10 分鐘	速覽
學硬體設計、想弄懂加速器怎麼設計	速覽 → 1 → 2 → 3 → 4 → 5 → 6 → 7
要馬上開始跑實驗	附錄 A.1（指令格式）→ A.6（make 指令）→ 8.11（讀報告），之後再回頭讀 1~4
要準備海報或報告	速覽 → 1 → 2 → 10（尤其 10.5 評審問答）
之後要讀新的 paper	11 → 5（當作「讀一篇 paper」的範例）
看到不懂的詞	附錄 B

先備知識

- 需要：基本的程式概念（變數、迴圈、陣列）、高中程度的數學（次方、平方根、比例）。
- 不需要：FPGA、硬體設計、生物學、深度學習的背景；手冊會從頭解釋。
- 看得懂一點 C/C++ 會比較輕鬆，但不是必要的（程式碼都有逐段說明）。

閱讀慣例

每個概念的說明順序

1. 白話直覺：用比喻（例如「倉庫 vs 料理台」）先建立印象。
2. 正式定義：公式或精確的說法。
3. 本專題的例子：附上實際數字。

每章的固定結構

- 開頭的「本章重點」：先知道這章要講什麼。
- 結尾的「本章小結」：複習。
- 最後的「自我檢測」：答案在附錄 C。

指令的狀態圖示 (附錄 A)

圖示	意思
	已經做過
	下一步會用到
	進行中
	之後可以做
	小心使用 (可能刪除資料)

其他

- 程式碼區塊中的 # 後面是說明文字，不需要輸入。
- 「L」一律代表蛋白質的長度（胺基酸個數），「C」代表 channel 數（本專題為 128）。
- 數字若標示「估計」、「假設」，代表還沒有經過實際量測驗證。
- 手冊中的狀態（例如「進行中」）以 2026-09-26 為準；最新進度請看 docs/PROJECT_LOG.md。

最短上手路徑 (10 分鐘)

如果你手上有 server 帳號，照著做就能跑出第一個結果：

```
git clone -b claudelightnobel-memory-optimization-yhokyc
https://github.com/antony3911/ForClaude.git
cd ForClaude/fpga/trimul
make csim # 幾秒：確認 5 個版本都算對 (看到 ALL PASS)
make hls KERNEL=trimul_v2 # 幾分鐘：產生 v2 的硬體報告
head -n 60 hls_prj/trimul_v2/sol/syn/report/trimul_v2_csynth.rpt
```

每一行在做什麼，見附錄 A；報告怎麼看，見 8.11 節。

速覽：十分鐘看懂這本書

這一章給誰看：剛接手專題、想在 10 分鐘內抓到全貌的人。

讀完你會知道：① 這個領域在解決什麼問題 ② LightNobel 做了什麼 ③ 本專題做了什麼 ④ 這本書的每一部在講什麼、要去哪裡找細節。

1. 一張圖看懂來龍去脈

蛋白質結構預測 (AlphaFold2、ESMFold ; 2024 諾貝爾化學獎)

| 模型內部有 $L \times L \times 128$ 的 pair representation

瓶頸：序列一長，activation 以 L^2 成長 → 記憶體不夠、搬資料太慢

- ├ GPU 系統優化 (FastFold、ScaleFold)：更快，但記憶體問題仍在
- ├ 分塊計算 (Chunking、AutoChunk)：省記憶體，但變慢
- ├ 量化 (MEFold、各種 PTQ)：壓縮資料，但 outlier 會傷精度
- ├ 融合 kernel (OpenFold3 的 GPU 實作)：中間值不落地

LightNobel (ISCA 2025)：token-wise 自適應量化 + 專用加速器 (ASIC)

| 軟硬體協同設計，峰值記憶體最多降約 120 倍

本專題：在現成的 FPGA (U55C) 上實作其中的 Triangle Multiplication，
逐步拆解每一招 (含 token-wise 量化) 的貢獻，並用分析模型做設計空間探索

2. 這本書的故事線

全書依「問題 → 原理 → 案例 → 實作 → 專題」的順序安排，每一部都建立在前一部上：

部	章	回答的問題
第一部 問題	1 蛋白質結構預測與 pair representation	模型在算什麼？資料長什麼樣子？
	2 記憶體牆	為什麼放不下、跑不快？用什麼工具分析？
第二部 原理	3 資料流優化	不改資料格式，怎麼少搬、搬得有效率、不存中間值？
	4 量化	怎麼讓每筆資料變小，又不失去精度？
第三部 案例： LightNobel	5 觀察、演算法與成果	頂尖的研究怎麼把原理用到極致？
	6 硬體架構	這些演算法需要什麼樣的硬體？每個單元怎麼設計？

部	章	回答的問題
第四部 實作	7 FPGA 與工具原理	C++ 怎麼變成電路？
	8 v0~v4 從程式碼到報告	我們實際寫了什麼、量到什麼？
	9 評估方法	還沒上板，怎麼評估一個設計？
第五部 專題	10 專題策略	我們的定位、設計、要驗證什麼、怎麼報告？
	11 研究方法	這套方法怎麼用在下一篇 paper？
附錄	A 指令手冊、B 術語表、C 參考答案	查詢用

3. LightNobel 一分鐘版

問題：蛋白質結構預測模型的 activation 隨 L^2 (attention 隨 L^3) 成長，長序列時 GPU 放不下、也跑不快。

三個創新：

1. token-wise 自適應量化 (AAQ)：每個 token 一把尺；依 activation 在模型中的位置分成 A、B、C 三組，分別用 INT8 + outlier、INT4 + outlier、INT4。(第 5 章)
2. outlier 另外存：每個 token 在執行時挑出最大的幾個值，用 INT16 另外存，格式對硬體友善。(第 5 章)
3. 專用硬體：用 4-bit 小乘法器拼出任何精度的 RMPU、做向量運算與執行時量化的 VVPU、以及不存 L^3 score matrix 的 token-wise attention。(第 6 章)

結果：峰值記憶體最多降約 120 倍（和不分塊的 GPU 比），80 GB 內最長可處理 9,945 個胺基酸；比 A100/H100 最多快約 8.4 倍。這些是模擬器的結果，沒有實體晶片。

一句話：找到「token」這個合適的壓縮單位，並設計能有效率地處理這種格式的硬體，讓長序列放得下、也跑得快。

4. 本專題一分鐘版

蛋白質結構預測模型裡有一份 $L \times L \times 128$ 的資料，序列越長越放不下。LightNobel 用專用晶片加上 token-wise 量化和管線化資料流解決這個問題，但只有模擬結果。我們在現成的 FPGA 上實作其中的 Triangle Multiplication，逐步拆解每一種記憶體優化手法各自的貢獻，並用分析模型找出最佳設計。

- 繼承：問題定義、token-wise 量化、管線化（中間值不落地）的原則。
- 簡化：只做一個運算、單一精度（INT8，下一步 INT4）。
- 新增：FPGA 上的實作、逐步 ablation (v0~v5)、真實資料驗證、分析模型與設計空間探索。
- 誠實的限制：長序列時最花時間的是 Triangle Attention，列為未來工作；融合與執行時量化的原則來自 LightNobel。

完整的定位與策略見第 10 章。

5. 概念地圖：每個概念在哪一章

概念	一句話	詳見
全景：從序列到 3D 座標	一次預測的完整流程、狀態與中間值、東西放在哪	1.2 節
Pair representation、activation、token	$L \times L \times 128$ 的中間資料；一格 (i, j) 是一個 token	1.3 節
LayerNorm、linear、sigmoid、gating、residual	Triangle Multiplication 裡的每個運算	1.5 節
Triangle Multiplication／Attention	本專題的運算／長序列時最花時間的運算	1.4 節
記憶體容量、memory-bound	放不下、搬資料比計算慢	2.1、2.2 節
算術強度、Roofline、Amdahl	判斷瓶頸、估計優化上限	2.5、2.6 節
資料表示／資料流／硬體結構	所有解法的三層地圖	2.7 節
Tiling、burst、double buffering	減少搬運、提高搬運效率、藏起等待	3.1～3.3 節
融合、重算、分塊	中間值不寫回、用計算換記憶體	3.5～3.7 節
量化、per-token、尺	壓縮資料；尺的成本與乘的時機	第 4 章
AAQ、A/B/C 三組	LightNobel 的量化方法	5.3 節
RMPU、VVPD、crossbar	LightNobel 的硬體單元	第 6 章
FPGA、HLS	可重新接線的晶片；把 C++ 變成電路的工具	第 7 章
Architectural simulation、DSE	用模型與合成報告評估還沒做出來的硬體	第 9 章
TM-score	預測結構和真實結構有多接近（0～1）	附錄 B

接下來從第 1 章開始，完整了解問題的細節。

第 1 章 蛋白質結構預測與 pair representation

本章重點

- 蛋白質結構預測在做什麼，以及貫穿全書的變數 L （蛋白質長度）。
- 全景（top-down）：一次預測從輸入序列到 3D 座標的完整流程；哪些是狀態、哪些是中間值、各放在哪裡。
- pair representation ($L \times L \times 128$)、activation、token 是什麼。
- 本專題實作的 Triangle Multiplication 在算什麼；為什麼長序列時最花時間的是 Triangle Attention。
- Triangle Multiplication 裡每一個運算：LayerNorm、linear、sigmoid、gating、mask、einsum、residual。

1.1 蛋白質結構預測在做什麼？

白話直覺

蛋白質是一條由胺基酸串成的「珠鍊」。胺基酸共有 20 種，用英文字母表示，所以一條蛋白質可以寫成一串字：

```
MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDGTQDNLSGAEK...
```

這條珠鍊在細胞裡會自動摺疊成特定的 3D 形狀，而形狀決定功能：酵素能不能抓住受質、抗體能不能卡住病毒，都取決於形狀。

蛋白質結構預測 = 輸入那串字母，輸出每個原子的 3D 座標。

為什麼重要

- 用實驗測定結構（X 光繞射、冷凍電顯）要花數月到數年，而且很貴。
- AlphaFold2（DeepMind，2021）讓預測準確度接近實驗水準，相關研究獲得 2024 年諾貝爾化學獎。LightNobel 的名字就是在呼應這件事。
- 應用：藥物設計、理解疾病突變、設計新的酵素。

本專題的關鍵變數： L

L = 蛋白質的長度（胺基酸個數）。整份手冊都圍繞這個數字：

蛋白質	UniProt ID	L
血紅素 α 鏈	P69905	142
p53（抑癌蛋白）	P04637	393

蛋白質	UniProt ID	L
新冠病毒棘蛋白（單體）	P0DTC2	1,273
BRCA1（乳癌相關）	P38398	1,863
Dystrophin（肌肉蛋白）	P11532	3,685
Titin（人體最大的蛋白之一）	Q8WZ42	約 34,000

大部分蛋白質在 1,000 以內，但很多重要的蛋白質比這長。另外，預測蛋白質複合體時常把多條鏈接起來一起算，L 會變得更大。

1.2 全景：一次預測從頭到尾在算什麼

這一節先由上往下（top-down）看完整個流程，後面各節再放大細節。

第 0 層：輸入與輸出

ESMFold 不是在預測「下一個胺基酸」（那是 ChatGPT 這類生成模型的做法）。

- 輸入：一條完整已知的胺基酸序列（L 個字母全部都知道）。
- 輸出：這條蛋白質摺疊後的 3D 結構，也就是每個原子的 (x, y, z) 座標，外加每個位置的可信度。
- 什麼時候算：使用者丟一條序列進來，就從頭算一次；每條序列都重新算，沒有「歷史」可以沿用。

比喻：給你一條珠鍊的珠子順序，問它會捲成什麼形狀。

第 1 層：三個階段

輸入：胺基酸序列（L 個字母，幾 KB）

↓

① ESM-2 語言模型：讀整條序列，給每個胺基酸一組特徵

↓ → s : $L \times 1024$

↓ → z : $L \times L \times 128$

（每個胺基酸的「自我描述」）

（每一對胺基酸的「關係表」；一開始只有「在序列上相距多遠」）

↓

② Folding trunk：48 個 block 依序執行，每個 block 都把 s 和 z 修正得更準

↓ s、z 進去 → s、z 出來（形狀不變，內容更新）

↓ 整個 trunk 會把結果送回開頭再跑幾次（recycling，預設最多 4 次）

↓

③ Structure module：把最後的 s、z 轉成 3D 座標

↓

輸出：每個原子的座標（約 $L \times 37 \times 3$ 個數，很小）

真正花時間、花記憶體的是 ②：把 z 這張 $L \times L$ 的關係表反覆修正，讓它越來越接近真實的距離關係；③ 再根據這張表排出 3D 形狀。

（AlphaFold2 的對應部分叫 Evoformer，也有 48 個 block，同樣有 pair representation。差別在於 AlphaFold2 用多序列比對（MSA）當輸入，ESMFold 改用語言模型。）

第 2 層：一個 block 裡做了什麼

每個 block 都用同一個模式修正資料： $z \leftarrow z + \text{修正量}$ （這叫 residual，1.5 節）。依序是：

步驟	做什麼（白話）
Sequence attention、MLP	先修正每個胺基酸的特徵 s
Sequence $\rightarrow$ pair	把 s 的資訊加進 z
Triangle Multiplication (outgoing)	用「 $i-k$ 的關係、 $j-k$ 的關係」推論「 $i-j$ 的關係」（本專題的 kernel 在這裡）
Triangle Multiplication (incoming)	同上，方向反過來
Triangle Attention (starting/ending)	同樣的推論，但用 attention 決定哪些 k 比較重要（長序列時最花時間，1.4 節）
Pair transition (MLP)	每一格自己再加工一次

第 3 層：一個 Triangle Multiplication 步驟

z （進來的關係表， $L \times L \times 128$ ）		← 狀態：從上一步傳進來
└ LayerNorm	→ z_ln	← 中間值
└ 5 個 linear + sigmoid	→ 各種投影與閘門	← 中間值
└ gating（相乘）	→ a 、 b	← 中間值
└ einsum（跨 k 加總）	→ x	← 中間值（本專題的 kernel）
└ LayerNorm、linear、 $\times g$	→ 修正量	← 中間值
└ $z \leftarrow z + \text{修正量}$		← 狀態：更新後傳給下一步

- 狀態（完成值）：更新後的 z ，要一直傳給下一步、下一個 block，所以必須保存。
- 中間值： z_ln 、 a 、 b 、 g 、 x 、修正量。只在這一步有用，加回 z 之後就沒用了。但每一個都是 $L \times L \times 128$ ，和 z 一樣大；一般寫法會整份寫回記憶體，所以同時存在約 7 份（2.1 節）。
- 「中間值不存、直接處理完」：不要整份算完 z_ln 再寫回去，而是一次拿一格 (i, j) 的 128 個數，在晶片裡一路做完，只把之後還需要的東西寫出去（這叫融合，3.5 節）。

每個運算具體在算什麼，見 1.5 節。

第 4 層：本專題的 kernel

只做第 3 層裡的 einsum：從 HBM 讀 a 、 b ，算 $x[i][j][c] = \sum_k a[i][k][c] \times b[j][k][c]$ ，把 x 寫回 HBM。v0~v4 算的是同一件事，差別只在怎麼搬資料（第 3、4 章）。

東西放在哪裡？

東西	大小（ $L = 1000$ ，FP32）	存活多久	放在哪裡
輸入序列	幾 KB	整次預測	CPU 記憶體
權重（所有 linear 的矩陣）	約 7.9 GB（FP16）	整次預測，固定不變	GPU 記憶體或 HBM（晶片外）

東西	大小 (L = 1000 , FP32)	存活多久	放在哪裡
s、z (狀態)	s 約 4 MB、z 約 512 MB	整個 trunk，每一步被更新	GPU 記憶體或 HBM
中間值 (z _{ln} 、a、b、g...)	每份約 512 MB，同時約 7 份	只在一個步驟內	一般：HBM；融合後：只在晶片內一小塊一小塊存在
Triangle Attention 的 score matrix	L ³ 級，數十 GB	只在一個步驟內	一般：HBM (所以放不下)；LightNobel：邊算邊丟
正在計算的一小塊 (tile)	幾十 KB	幾微秒	晶片內 (FPGA 的 BRAM、GPU 的 shared memory)
輸出座標	幾百 KB	最後	傳回 CPU

1.3 兩種 representation、activation 與 token

名稱	形狀	意義	大小隨 L
Sequence representation	L × 1024	「第 i 個胺基酸」的特徵	線性 (L)
Pair representation	L × L × 128	「第 i 個和第 j 個胺基酸之間的關係」	平方 (L ²)

Pair representation 可以想成一張 L × L 的表格，每一格放 128 個數字。第 (i, j) 格記錄模型對「i 和 j 距離多遠、有沒有交互作用」的理解。

z 到底是什麼？

z 就是 pair representation：z[i][j] 是 128 個特徵值，描述「胺基酸 i 和 j 之間的關係」。描述「各自」的是 s。

z 在一次預測中的一生：

1. 一開始幾乎是空的：全部是 0，只加上「i 和 j 在序列上相距幾個位置」的資訊。
2. 每個 block 從 s 補資訊：把 s_i 和 s_j 的特徵相乘、相減，組合後加進 z[i][j] (sequence → pair)。
3. 每個 block 用三角形推理修正：Triangle Multiplication 和 Triangle Attention 讓 z[i][j] 參考所有第三者 k。
4. 最後被拿去預測距離：ESMFold 用一個 linear 把每一格 z[i][j] 轉成 64 個「距離區間」的機率 (distogram)，structure module 也用 z 排出 3D 形狀。

所以 z 最終的意義是「每一對胺基酸之間距離與交互作用的描述」。a、b、g、x 這些都是從 z 算出來、用完就丟的中間值，形狀和 z 一樣，但不是 z 本身。

兩個常被搞混的詞：activation 與 token

Activation = 模型推論過程中算出來的所有中間資料。它是一個總稱，不是某種特定的運算。

	是什麼	做菜的比喻	大小會不會隨輸入改變
權重 (weight)	訓練時學到的參數，例如 128×128 的 linear 矩陣	食譜	不會，固定 (ESMFold 約 7.9 GB)
Activation	用這次的輸入一步步算出來的資料	做菜過程中的每一盤半成品	會，蛋白質越長越大

Pair representation 就是最主要的 activation，但不只它一份：Triangle Multiplication 一個 block 裡，LayerNorm 的結果、5 個 linear 的結果、gating 後的 a、b、einsum 的結果…… 每一步算出來的 $L \times L \times 128$ 張量都是 activation（流程見 1.2 節第 3 層，每個運算的意義見 1.5 節）。

Token = 資料的「一格」。在不同的 representation 裡，一格代表的東西不同：

	Sequence representation	Pair representation
一個 token 代表	一個胺基酸 i	一對胺基酸 (i, j)
每個 token 有幾個數	1024	128
token 的數量	L 個	$L \times L$ 個 (L = 1000 時是 100 萬個)

為什麼一個胺基酸有 1024 個數，一對胺基酸卻只有 128 個？這兩個數字不是由胺基酸決定的，而是模型設計者選的特徵維度（ESMFold 的設定：sequence 1024、pair 128）。每個數字是一個「學出來的特徵」，就像描述一個人可以用 1024 項屬性、描述兩個人之間的關係用 128 項屬性。設計者刻意讓 pair 的維度小，是因為 pair 的 token 數量是 L^2 ：

L = 1000 時
sequence : 1,000 個 token $\times$ 1024 = 約 100 萬個數
pair : 1,000,000 個 token $\times$ 128 = 約 1.28 億個數 ← 仍然大了 128 倍
如果 pair 也用 1024 : 1,000,000 $\times$ 1024 = 約 10 億個數 ← 再大 8 倍，一份就 4 GB (FP32)

所以每個 token 小，不代表總量小；pair representation 的問題在於 token 的數量隨 L^2 成長。（ESM-2 語言模型本身的特徵是 2560 維，進入 folding trunk 前先投影到 1024 維。）

本手冊（以及 LightNobel 的量化）說的 token，都是指 pair representation 的一格 (i, j)。模型裡的 LayerNorm、linear 都是「一個 token 的 128 個數自己算自己的」，所以 token 是很自然的處理單位。

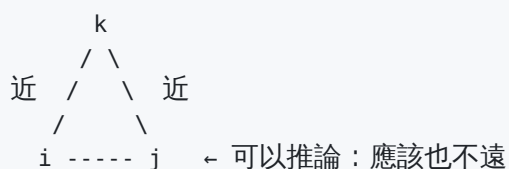
1.4 Triangle Multiplication 與 Triangle Attention

「三角形」的直覺

如果知道：

- 胺基酸 i 離 k 很近
- 胺基酸 j 離 k 也很近

那 i 和 j 大概也不會太遠，這就是三角不等式。



Triangle Multiplication 就是把這個推理寫成數學：更新 (i, j) 格時，把所有第三者 k 都看過一遍。

算式 (outgoing 版本)

$$z[i][j][c] = \sum_k a[i][k][c] \times b[j][k][c] \quad (k \text{ 從 } 0 \text{ 到 } L-1, c \text{ 從 } 0 \text{ 到 } 127)$$

- **a**、**b**：由 pair representation 經過 LayerNorm、線性轉換、gating 得到，形狀都是 $L \times L \times 128$ 。
- **z**：輸出，形狀也是 $L \times L \times 128$ 。
- 對每個 channel c 來說，這其實是一個 $L \times L$ 的矩陣乘法，總共做 128 次。

具體例子 ($L = 3$ ，只看一個 channel)

$$\begin{aligned}
 a &= \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} & b &= \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix} \\
 z[0][1] &= a_{00} \cdot b_{10} + a_{01} \cdot b_{11} + a_{02} \cdot b_{12} \\
 &= \sum_k a[0][k] \cdot b[1][k] && \leftarrow \text{用到 } a \text{ 的第 } 0 \text{ 列和 } b \text{ 的第 } 1 \text{ 列}
 \end{aligned}$$

也就是 $z = a \times b^T$ (a 乘以 b 的轉置)。

我們已經用 HuggingFace 的 ESMFold 驗證：這個算式和 ESMFold 模組內部的計算完全一致 (差值 0) (詳見第 8 章 8.9 節)。

Triangle Attention (我們沒有實作，但 LightNobel 有處理)

同一個 block 裡還有 Triangle Attention：對每一列 i，在 (j, k) 之間做 attention。它的中間值 (attention logits) 形狀是 $L \times L \times L \times \text{heads}$ ，是立方成長，所以 GPU 上一定要分塊 (chunking) 計算，否則 $L = 1000$ 時光這個中間值就需要數十 GB。LightNobel 則改用 token-wise 的 attention (類似 FlashAttention)，根本不把整個 score matrix 存下來。

它才是長序列時最花時間的運算。LightNobel 在 H100 上量測 ESMFold (不分塊) 的執行時間 (論文 Fig. 3)：

運算	77 個胺基酸 (R0271)	1,410 個胺基酸 (T1269)
Triangle Multiplication	36.1%	14.5%
Triangle Attention	29.0%	75.9%
整個 pair representation 資料流	69.4%	91.9%

序列短時，Triangle Multiplication 佔比最高；序列一長，立方成長的 Triangle Attention 就壓過其他所有運算。所以本專題選 Triangle Multiplication 不是因為它最花時間，理由見 10.1 節。

上面的 `a`、`b` 是怎麼來的、`einsum` 之後又做了什麼，見下一節。

1.5 Triangle Multiplication 裡的每一個運算

這一節把 1.2 節第 3 層的每個運算拆開：做什麼、公式、數字例子、在硬體上要什麼。這些零件（LayerNorm、linear、sigmoid、gating、residual）幾乎出現在所有深度學習模型裡（包括 ChatGPT 這類 Transformer），值得徹底弄懂。

完整的算式（ESMFold 的實作）

```
z_ln = LayerNorm_in(z)           ①
a    = mask × sigmoid(linear_a_g(z_ln))  ⑤ ③ ②
      × linear_a_p(z_ln)           ④ ②
b    = mask × sigmoid(linear_b_g(z_ln))
      × linear_b_p(z_ln)
x    = einsum(a, b)               ⑥
x    = linear_z(LayerNorm_out(x))  ② ①
g    = sigmoid(linear_g(z_ln))    ③ ②
修正量 = x × g                   ④ 輸出閘門
z    = z + 修正量                 ⑦（在 block 裡做）

① LayerNorm ② linear ③ sigmoid ④ gating ⑤ mask ⑥ einsum ⑦ residual
```

（為了好讀，以下例子只用 4 個 channel；實際是 128 個。）

① LayerNorm：把每個 token 的數字拉回標準範圍

做什麼：對一個 token 的 128 個數，先減掉平均、再除以標準差，讓它們變成「平均 0、大小約 1」；最後再乘上一組學出來的係數 γ 、加上 β （每個 channel 各一個）。

```
mean =  $\sum x_c / 128$ 
var  =  $\sum (x_c - \text{mean})^2 / 128$ 
y_c  =  $(x_c - \text{mean}) / \sqrt{\text{var} + \epsilon} \times \gamma_c + \beta_c$ 
```

例子： $x = [2, 4, 6, 8] \rightarrow \text{mean} = 5$ 、標準差 $\approx 2.236 \rightarrow [-1.34, -0.45, 0.45, 1.34]$ （ $\gamma = 1$ 、 $\beta = 0$ 時）。

為什麼需要：z 每經過一步就加上一筆修正量，數值會越滾越大（這就是 LightNobel A 組數值很大的原因，5.3 節），而且不同 token 的大小差很多。先標準化，後面的 linear 和 sigmoid 才能在穩定的範圍內工作。

硬體的比喻：自動增益控制（AGC）。訊號振幅忽大忽小，進入下一級（放大器、ADC）之前先把振幅調回固定範圍，下一級才不會飽和。LayerNorm 對每個 token 做同樣的事：減平均（去掉 DC offset）、除以標準差（調振幅），再乘 γ 、加 β （每個 channel 一組學出來的增益和偏移）。

硬體：兩次加總（一次算平均、一次算變異數）、一次開根號與倒數（每個 token 一次），再做 128 次減法與乘法。只需要這個 token 自己的 128 個數，可以整個放在晶片內（10.4 節）。

② Linear：把 128 個特徵重新混合

做什麼：一個 128×128 的權重矩陣 W 乘上這個 token 的 128 個數，再加上偏移 b ：

$$y = Wx + b \quad \text{每個輸出 } y_o = \sum_c W[o][c] \times x_c + b_o$$

每個輸出都是所有 128 個輸入的加權總和，權重是訓練時學到的。模型「學到的知識」大多就存在這些矩陣裡。

例子（2 個 channel）： $W = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}$ 、 $x = [3, 1] \rightarrow y = [1 \cdot 3 + 2 \cdot 1, 0 \cdot 3 - 1 \cdot 1] = [5, -1]$ 。

意義：一組「特徵偵測器」。 W 的每一列是一個偵測器：和輸入做內積，算出「這個 token 有多符合我要找的模式」。硬體的比喻是相關器（correlator）或 FIR 濾波器： W 的一列是一組係數，輸入越像這組係數，輸出越大。（下面假裝特徵有名字，方便理解；實際的特徵是模型自己學的，人看不懂。）

token 的特徵 x ：[距離近，有氫鍵，都疏水，電荷互斥]
偵測器 w ：[1.0 , 0.5 , 1.0 , -2.0] ← 「會形成穩定接觸」的偵測器

token A：[0.9, 0.1, 0.8, 0.0] $\rightarrow 0.9 + 0.05 + 0.8 + 0 = 1.75$ ← 很符合
token B：[0.1, 0.0, 0.2, 0.9] $\rightarrow 0.1 + 0 + 0.2 - 1.8 = -1.5$ ← 不符合

128×128 的 linear 就是 128 個偵測器同時看同一個 token，輸出 128 個新特徵。

Triangle Multiplication 有 6 個 linear（ a_p 、 a_g 、 b_p 、 b_g 、 g 、 z ），各有不同用途，但計算方式完全一樣。

硬體：矩陣乘向量，每個 token 要 $128 \times 128 = 16,384$ 次乘加。權重只有 64 KB（FP32），可以一直放在晶片內（weight-stationary，6.2 節）。

③ Sigmoid：把任何數壓到 0~1 之間

做什麼：

$$\text{sigmoid}(x) = 1 \div (1 + e^{(-x)})$$

x	-4	-2	0	2	4
sigmoid(x)	0.018	0.119	0.5	0.881	0.982

很負 $\rightarrow$ 接近 0；很正 $\rightarrow$ 接近 1；0 $\rightarrow$ 剛好 0.5。它是一條平滑的 S 形曲線。

用途：把 linear 算出來的任意數，變成「要讓多少通過」的比例（0 = 全擋、1 = 全開）。在這裡它當作閘門（gate），和下一個運算 gating 一起用。

另一個角色：非線性。如果模型只有 linear，疊再多層也等於一個 linear（矩陣乘矩陣還是矩陣）。sigmoid 這類非線性函數讓模型能表達複雜的關係，這是深度學習能「深」的原因之一。

硬體：需要指數和除法，完整實作很貴；硬體通常用查表或分段直線近似（LightNobel 的 VVPU 就有指數查表，6.5 節）。

有了 sigmoid，為什麼還需要 LayerNorm？

兩個做的是完全不同的事：

⑤ Mask：忽略補上去的位置

做什麼：乘上 0 或 1。為了對齊，序列常會補上假的位置（padding，例如我們的程式補到 32 的倍數）；mask 把這些位置的值設成 0，讓它們不影響結果。硬體的比喻：AND gate 或 enable 訊號。無效的位置若不歸零，會被 einsum 加進總和、汙染結果。

⑥ Einsum：唯一「跨 token」的運算

做什麼： $x[i][j][c] = \sum_k a[i][k][c] \times b[j][k][c]$ ，也就是對每個 channel 做 $a \times b^T$ 的矩陣乘法（1.4 節）。「einsum」是 Einstein summation 的縮寫：寫成 "ikc,jkc->ijc"，輸入有、輸出沒有的索引（k）就加總起來。

為什麼特別：①~⑤ 都只用這個 token 自己的 128 個數；只有 einsum 要讀整列 i 和整列 j 的所有 token。所以 einsum 是融合的「邊界」：前面的運算可以一個 token 一個 token 在晶片內做完，但 a、b 必須先寫出去，einsum 才能跨 token 讀取（3.5 節）。

硬體的比喻：就是矩陣乘法，由大量的乘加器（MAC）完成；意義是「收集所有第三者 k 的證據」。

硬體：最重的一步。每個輸出 token 要 $2 \times L \times 128$ 次運算， $L = 1000$ 時約 25.6 萬次；相比之下，6 個 linear 加起來每個 token 約 19.7 萬次。L 越長，einsum 越是主角，這也是本專題選它做 kernel 的原因之一。

LayerNorm_out + linear_z：einsum 之後的整理

- 為什麼要再做 LayerNorm：x 是 L 個乘積的總和，L 越長總和越大，不同長度的蛋白質差很多，所以要再做一次 AGC。
- 為什麼要 linear_z：einsum 的每個 channel 是「一個問題的證據總和」；linear_z 把 128 個證據重新組合，轉回 z 的特徵空間，才能加回 z。比喻：調查員各自回報的結果，要整理成報告的格式才能歸檔。

× g：輸出閘門

$g = \text{sigmoid}(\text{linear}_g(z_{\text{ln}}))$ 、修正量 = $x \times g$ 。和 ④ 一樣是 gating，但作用在最後的修正量上：由原本的 z 決定「這一步的修正要採納多少」。某一格本來就很確定時，模型可以把 g 調小、少改一點。

a、b 是什麼？為什麼只有它們要寫出去？

a、b 是兩種「訊息」。a[i][k] 是「i-k 這條邊」要提供的訊息，b[j][k] 是「j-k 這條邊」的訊息。兩者都來自 z_{ln} ，只是用不同的 linear（不同的偵測器）做出來，扮演不同的角色。einsum 把它們配對：channel c 可以想成一個「問題」（例如「靠得近嗎？」），i-k 和 j-k 在 c 上都大，乘起來就大，代表第三者 k 提供了「i 和 j 有關係」的證據；把所有 k 的證據加總，就是 (i, j) 的結論。

為什麼要寫出去？a[i][k] 會被 $x[i][0] \sim x[i][L-1]$ 共 L 個輸出用到，而且這些輸出分散在整個計算的不同時間點；a[i][k] 產生時，大部分用得到它的輸出都還沒輪到，只能先存起來（reuse distance 很長，3.5 節）。相比之下， z_{ln} 、a_p、a_g 只在算同一個 token 時用一次，可以留在晶片內。

融合後，每個值要不要存：

值	誰用它	用幾次、多久之後	要存到記憶體嗎？
z（輸入）	LayerNorm、最後的 residual、重算 g	一開始用，最後又要用	要（狀態）

值	誰用它	用幾次、多久之後	要存到記憶體嗎？
z_in、a_p、a_g、 b_p、b_g	linear、gating	同一個 token 馬上用完	不用
a、b	einsum	被 L 個輸出用，分散在整個計算中	要（融合的邊界）
g	最後的 $x \times g$	要等 einsum 做完才用	可以存，也可以最後從 z 重算（本專題選重算，3.6 節）
x（einsum 的結果）	LayerNorm_out、linear_z、 $x \times g$	同一個輸出 token 馬上用完	不用
新的 z	下一個步驟	要傳下去	要（狀態）

所以融合後記憶體裡只剩 z（進、出各一份）+ a、b（INT8，約半份） $\approx$ 2.5 份，一般寫法約 7 份（10.3 節）。

⑦ Residual：修正量加回原本的值

做什麼： $z = z + \text{修正量}$ 。每一步都不是重算 z，而是在原本的 z 上加一點修正。

為什麼需要：讓資訊可以一路傳下去、每一層只要學「還差多少」，模型才能疊到 48 層還訓練得起來。硬體的比喻：累加器，或回授迴路裡的「加上一個 delta」。副作用：48 個 block 不斷累加，z 的數值越來越大（LightNobel 的 A 組，5.3 節），所以每一步開頭都要先做 LayerNorm。

block 裡其他步驟的簡介

步驟	做什麼	主要零件
Sequence attention	讓每個胺基酸參考其他胺基酸，修正 s（也用 z 當參考）	linear、attention、residual
Sequence $\rightarrow$ pair	把 s_i 、 s_j 相乘、相減，加進 $z[i][j]$	LayerNorm、linear、residual
Triangle Multiplication	用第三者 k 推論 i-j 的關係	本節介紹的全部
Triangle Attention	同上，但用 attention 決定哪些 k 比較重要	linear、softmax、einsum、gating、residual
Pair transition (MLP)	每一格自己再加工一次	LayerNorm、linear、非線性函數、linear、residual

Attention 可以看成「先算每個 k 有多重要（權重，用 softmax 算出、總和為 1），再依權重加總」。Triangle Multiplication 對所有 k 一視同仁地加總；attention 則先算權重，這些權重形成 $L \times L \times L$ 的 score matrix，所以才那麼吃記憶體（1.4 節）。

總整理

運算	輸入範圍	每個 token 的運算量	在硬體上要什麼
① LayerNorm	一個 token	約 5×128	加總、開根號、乘法
② Linear ($\times 6$)	一個 token + 權重	$6 \times 2 \times 128^2 \approx 19.7$ 萬	大量乘加器、晶片內存權重
③ Sigmoid	一個值	128 (每個值一次)	指數查表
④ Gating、⑤ Mask	一個 token	128	逐元素乘法
⑥ Einsum	跨 token (整列 i、整列 j)	$2 \times L \times 128$	大量乘加器、大量搬運 (第 3、4 章)
⑦ Residual	一個 token	128	加法

本章小結

概念	一句話
L	蛋白質長度，全書的主角；pair representation 的大小隨 L^2 成長
Activation	推論時算出來的所有中間資料；pair representation 是最主要的一份
Token	pair representation 的一格 (i, j)，128 個數
Triangle Multiplication	對每個 channel 做 $z = a \times b^T$ ，本專題的 kernel
Triangle Attention	score matrix 是 L^3 ，長序列時佔 75.9% 的執行時間
狀態 vs 中間值	z 要一直傳下去（狀態）； z_{In} 、a、b、g、x 只在一步內有用（中間值），卻和 z 一樣大
基本零件	LayerNorm（標準化）、linear（混合特徵）、sigmoid（壓到 0~1）、gating（開門）、einsum（唯一跨 token）、residual（加回去）

自我檢測

讀完這一章，試著回答下面的問題（參考答案在附錄 C）：

1. Triangle Multiplication (outgoing) 對單一 channel 來說，等同於什麼矩陣運算？
2. 序列很長時，ESMFold 最花時間的是哪個運算？既然如此，為什麼本專題還是先做 Triangle Multiplication？
3. 在 pair representation 裡，一個 token 代表什麼？和 sequence representation 的 token 有什麼不同？
4. ESMFold 是在預測「下一個胺基酸」嗎？它的輸入和輸出分別是什麼？
5. 在一個 Triangle Multiplication 步驟裡，哪些是「狀態」、哪些是「中間值」？為什麼中間值會佔很多記憶體？
6. sigmoid 的輸出範圍是多少？它和 gating 一起用時扮演什麼角色？

7. Triangle Multiplication 的運算中，哪一個需要讀取「其他 token」的資料？這為什麼決定了融合的邊界？
 8. 既然 sigmoid 會把數字壓到 $0 \sim 1$ ，為什麼還需要先做 LayerNorm？
 9. a 和 b 分別代表什麼？為什麼融合之後它們仍然必須寫回記憶體，而 z_{in} 不用？
 10. z 是什麼？它和 s 有什麼不同？在預測結束時 z 被拿來做什麼？
-

下一章說明這份 $L \times L \times 128$ 的資料為什麼會造成兩個問題：記憶體放不下、搬資料比計算慢，並介紹分析這類問題的工具（roofline、Amdahl's Law）。

第 2 章 記憶體牆：容量、頻寬與效能分析

本章重點

- 問題一：記憶體放不下。pair representation 隨 L^2 成長，一步 Triangle Multiplication 就同時存在約 7 份；瓶頸在 activation，不在權重。
- 問題二：搬資料比計算慢（memory-bound）。用算術強度判斷。
- 分析工具：記憶體階層、時間公式、Roofline 模型、Amdahl's Law。
- 最後一節給出全書的地圖：所有解法都可以放進「資料表示／資料流／硬體結構」三層。

2.1 問題一：記憶體放不下

算一算 pair representation 有多大

每個數字用 FP32（4 bytes），每格 128 個數字 → 每格 512 bytes。

精度說明：本專題的 kernel 用 FP32（HLS 裡最直接）。LightNobel 的比較基準（ESMFold）是 FP16，大小是下表的一半。和論文比較時要用 FP16 當基準，否則量化省下的比例會被誇大（INT8 相對 FP32 是 1/4，相對 FP16 只有 1/2；見 9.4 節）。

$$\text{大小} = L \times L \times 128 \times 4 \text{ bytes} = L^2 \times 512 \text{ bytes}$$

L	L^2	一份 pair tensor (FP32)
256	65,536	32 MB
500	250,000	128 MB
1,000	1,000,000	512 MB
2,000	4,000,000	2 GB
4,000	16,000,000	8 GB
10,000	100,000,000	51 GB

長度變 2 倍，資料變 4 倍；長度變 10 倍，資料變 100 倍。

實際需要的不只一份

計算一次 Triangle Multiplication 時，同時存在的資料至少有：

資料	形狀	說明
輸入 z	$L \times L \times 128$	pair representation 本身

資料	形狀	說明
LayerNorm 後的 z	$L \times L \times 128$	
a 、 b	各 $L \times L \times 128$	投影後的兩份
gate (a_g 、 b_g 、 g)	各 $L \times L \times 128$	gating 用
輸出	$L \times L \times 128$	

粗估同時有 5~8 份 pair 大小的資料。L = 2000 時，光這一步就可能需要 10~16 GB。再加上 Triangle Attention 的立方級中間值，GPU 記憶體很快就不夠用 (Out of Memory, OOM)。

為什麼問題出在 activation，不是模型權重

類別	是什麼	大小和 L 的關係	ESMFold 的量級
權重 (weights)	模型學到的參數	固定，和 L 無關	ESM-2 3B + folding trunk，FP16 約 7.90 GB (論文 Table 1)
Activation	計算過程中產生的中間資料	隨 L^2 甚至 L^3 成長	L 大時可達數十 GB

L 小的時候，權重佔大頭；L 一大，activation 就遠遠超過權重。論文 Fig. 4：L = 2,034 時 activation 已經是權重的 24.15 倍，需要 144 GB，一張 80 GB 的 GPU 放不下。以 3,364 個胺基酸的 T1169 為例，FP16 的 activation 是 113.49 GB，權重只有 7.90 GB (Table 1)。所以「把模型變小」幫助有限，真正要處理的是 activation。這正是 LightNobel 的切入點。

現在的解法與代價

常見做法是 chunking (分塊)：一次只算一部分，算完就丟。

- 優點：記憶體峰值下降。
- 代價：要重複讀資料、無法完全平行，時間變長。
- LightNobel 的 GPU 比較對象就有「不分塊」和「分塊 (Chunk4)」兩種；分塊後 GPU 能跑更長的序列，但慢很多。

小結：序列長度被記憶體容量卡住，這就是 LightNobel 標題裡的「Sequence Length Limitation」。

2.2 問題二：搬資料比計算慢 (memory-bound)

白話直覺：廚房比喻

- HBM / DRAM (主記憶體) = 遠處的倉庫：很大，但來回一趟很花時間。
- 晶片內記憶體 (cache、BRAM、shared memory) = 手邊的料理台：拿取很快，但很小。
- 運算單元 = 廚師：切菜很快。

如果廚師切一刀，就要等助手跑一趟倉庫拿下一個食材，那廚師大部分時間都在等。這時候瓶頸是「搬運」，不是「廚師的速度」。這就叫 memory-bound。反過來，如果食材都在手邊，只是廚師切不完，就叫 compute-bound。

用數字判斷：算術強度 (Arithmetic Intensity)

算術強度 = 運算次數 (FLOP) ÷ 從主記憶體搬運的資料量 (byte)

單位是 FLOP/byte，代表「每搬 1 byte 可以做幾次運算」。越高越好。

例子：最直白的寫法 (我們的 v0)

每算一次 $a \times b$ 並累加 (2 次運算：一次乘、一次加)，都要從記憶體讀一個 a (4 bytes) 和一個 b (4 bytes)：

算術強度 = 2 FLOP ÷ 8 bytes = 0.25 FLOP/byte

硬體能提供多少？

硬體	記憶體頻寬	峰值運算能力 (FP32)	平衡點 (運算 ÷ 頻寬)
U55C (本專題)	約 460 GB/s (HBM2)	取決於設計	—
一般 GPU (例：A100)	約 1.5~2 TB/s	約 19.5 TFLOP/s	約 10 FLOP/byte

以 GPU 為例，演算法的算術強度要超過約 10 FLOP/byte，才能把運算單元餵飽。v0 只有 0.25，也就是說運算單元大約 97% 的時間都在等資料。

那 Triangle Multiplication 本質上是 memory-bound 嗎？

不一定，取決於怎麼寫：

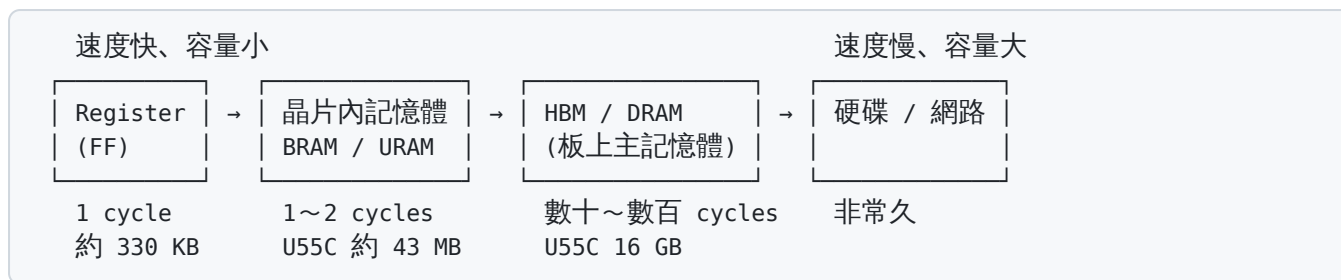
- 理論上：總運算量是 $2 \cdot L^3 \cdot C$ ，但資料只有 $3 \cdot L^2 \cdot C$ 個數字。如果每個數字只搬一次，算術強度可以非常高 (L 越大越高)。
- 實際上：晶片內的空間放不下整個 $L \times L \times 128$ ，只能分批搬，而分批的方式決定要重複搬幾次。

這就是優化的空間所在：同樣的計算，搬運方式不同，速度可以差幾十倍。第 3、4 章會用公式說明每種手法如何減少搬運；第 8 章會看到實際數字 (v2 報告中約 21% 的時間在等記憶體)。

2.3 記憶體階層：為什麼「搬運」有成本

白話直覺

離運算單元越近的記憶體，越快但越小；越遠的記憶體，越大但越慢。



U55C 的具體數字

層級	容量	存取延遲	備註
FF (暫存器)	約 260 萬個 bit (約 330 KB)	1 cycle	直接接在運算單元上
BRAM	4,032 塊 × 18 Kb ≈ 9 MB	1 ~ 2 cycles	可同時存取很多塊
URAM	960 塊 × 288 Kb ≈ 35 MB	1 ~ 2 cycles	比 BRAM 大塊
HBM2	16 GB (32 個 pseudo-channel × 512 MB)	數十~上百 cycles	總頻寬約 460 GB/s

關鍵：一份 $L = 1000$ 的 pair tensor 就有 512 MB，晶片內只有約 43 MB，不可能整份放進來，一定得分批從 HBM 搬。

2.4 一條公式看懂所有優化

一個 kernel 的執行時間，可以粗略拆成「計算」和「搬資料」兩部分：

如果計算和搬運「輪流」做（沒有重疊）：

$$T \approx T_{\text{compute}} + T_{\text{memory}}$$

如果計算和搬運「同時」做（完全重疊）：

$$T \approx \max(T_{\text{compute}}, T_{\text{memory}})$$

其中：

$$T_{\text{compute}} = \text{運算次數} \div (\text{每個 cycle 的運算數} \times \text{時脈})$$

$$T_{\text{memory}} = \text{搬運量} \div \text{頻寬} + \text{每次請求的延遲} \times \text{等待次數}$$

每一招優化，都是在縮小其中一項：

手法	縮小哪一項	對應版本
Tiling/資料重用	搬運量 ↓	v1
寬位元、burst	頻寬 ↑、等待次數 ↓	v2
平行運算單元	每 cycle 運算數 ↑	v2 (16 路)、v4 (64 路)
Double buffering	把 + 變成 max	v3
多通道 HBM	頻寬 ↑	cfg 對照實驗
量化	搬運量 ↓ (每筆資料變小)	v4

接下來每一節都會回到這條公式。

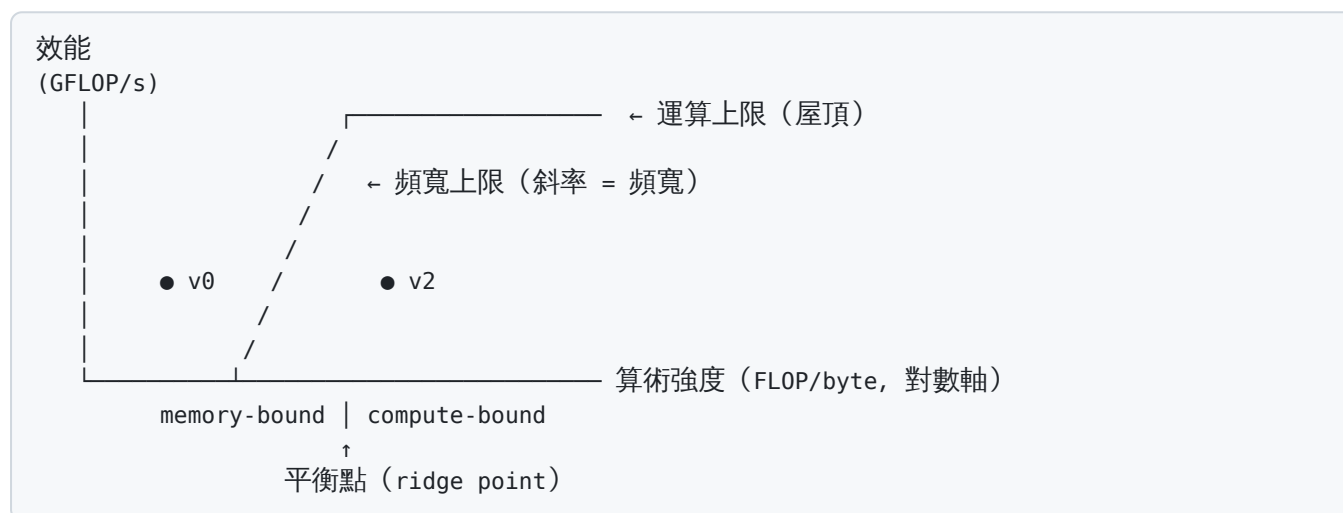
2.5 算術強度與 Roofline 模型

算術強度 (AI) 在 2.2 節已經介紹過：每搬 1 byte，能做幾次運算。Roofline 模型用它來判斷瓶頸在哪裡。

Roofline 模型

硬體有兩個上限：

1. 運算上限（每秒最多做幾次運算）：一條水平線。
2. 頻寬上限（每秒最多搬多少 byte）：AI 越高，同樣頻寬能支撐的運算越多，是一條斜線。



- 在平衡點左邊：被頻寬限制 (memory-bound) → 要提高 AI 或頻寬。
- 在平衡點右邊：被運算能力限制 (compute-bound) → 要增加運算單元。

可達效能的公式

$$\text{可達效能} = \min(\text{運算上限}, \text{AI} \times \text{頻寬})$$

本專題的例子

版本	AI (FLOP/byte)	說明
v0	0.25	每 2 次運算讀 8 bytes
v1、v2、v3 (8×8 tile)	2	見 3.1 節推導
v4 (INT8, 8×8 tile)	約 7.8	見 4.6 節推導

畫 roofline 圖是報告中最重要圖之一：可以一眼看出每一招讓 kernel 往哪個方向移動。

2.6 Amdahl's Law 與瓶頸轉移

Amdahl's Law

$$\text{整體加速} = 1 \div [(1 - p) + p \div s]$$

p = 被改善的部分佔原本時間的比例

s = 那部分被加速的倍數

例：v3 (double buffering, 3.3 節) 只改善「等記憶體」那 21% (p = 0.21)，即使讓它完全消失 (s → ∞)：

$$\text{整體加速} = 1 \div (1 - 0.21) \approx 1.27 \text{ 倍}$$

瓶頸轉移

每解決一個瓶頸，下一個瓶頸就會浮現。以本專題的各版本為例（各版本的細節在第 3、4 章，這裡先看整體脈絡）：

v0：受限於記憶體 (AI = 0.25)

→ v1 tiling：搬運量減少 8 倍，但每 cycle 只做 1 次乘加 → 變成受限於運算

→ v2 寬位元 + 16 路平行：運算變 16 倍 → 記憶體延遲佔 21%

→ v3 double buffering：延遲被藏起來 → 又變成受限於運算 (16 路)

→ 下一步：增加平行度（例如展開 jj 迴圈）→ 又會回到受限於記憶體 ...

找出每一步的瓶頸在哪、為什麼轉移，本身就是有價值的分析。這也是為什麼報告中要畫 roofline：可以看到 kernel 在圖上一步步移動。

2.7 解法的地圖：資料表示、資料流、硬體結構

學到這裡，很容易搞不清楚「哪些是軟體優化、哪些是硬體優化」。比較好用的分法是三層：

層	問的問題	例子	通常由誰決定
① 資料表示	每個數存成什麼格式？	FP32、FP16、INT8、INT4、per-token 的尺、outlier 另外存	演算法／軟體
② 資料流 (dataflow)	資料怎麼搬、放在哪、什麼順序算？哪些中間值要存？	tiling、double buffering、融合、重算、chunking、token-wise attention	軟體，或做成硬體
③ 硬體結構	誰來算、有多少資源？	運算單元的數量與精度、晶片內記憶體大小、介面寬度、HBM 通道數	硬體

在 memory-bound 的問題裡，三層的目標都是減少資料存取：① 讓每筆資料變小、② 讓搬的次數與存的份數變少、③ 讓搬運更快、晶片內放得更多。

為什麼在 FPGA 上特別分不清？在 GPU 上，硬體是固定的，資料流由程式（軟體）決定。在 FPGA/ASIC 上，HLS 的 C++ 迴圈順序會變成電路的資料路徑、陣列大小會變成晶片內記憶體，資料流的決定直接變成硬體結構，兩層合在一起。

為什麼需要軟硬體協同設計？只改①不一定會變快：LightNobel 指出 GPU 上的 INT4 activation 量化反而比 FP16 慢，因為 GPU 的運算單元不是為這種格式設計的。所以 LightNobel 同時設計了③（能直接算多種精度的 RMPU、能執行時量化的 VVPU，第 6 章）。

手法	① 資料表示	② 資料流	③ 硬體結構
v1 tiling		✓ 一次搬一批、重複使用	需要晶片內 buffer
v2 寬位元			✓ 512-bit 介面、16 個運算單元
v3 double buffering		✓ 搬運和計算同時做	需要兩組 buffer
多通道 HBM			✓ 資料分散到不同通道
v4 INT8/v5 INT4	✓ 每個數變小、per-token 尺		需要整數乘法器與解包邏輯
融合、重算 g (3.5、3.6 節)		✓ 中間值不寫回	需要把運算串成管線
chunking (GPU)		✓ 一次只算一部分	
LightNobel AAQ	✓ 三組、INT4/INT8、outlier		
LightNobel RMPU/VVPU (第 6 章)		✓ 管線、執行時量化	✓ 多精度運算單元、排序硬體
LightNobel token-wise attention		✓ 不存 L^3 的 score matrix	需要 scratchpad 與線上最大值追蹤

一句話：目標是減少資料存取；軟體決定資料的樣子和搬運順序，硬體決定這些事做起來多快。在 FPGA 上，後兩者是用同一份程式寫出來的。

本章小結

問題	具體表現	分析工具
記憶體容量	$L = 2000$ 時一份 pair tensor (FP32) 就 2 GB，同時存在 5~8 份	容量公式 (第 9 章 9.4 節會延伸成容量模型)
記憶體頻寬	最直白的寫法算術強度只有 0.25 FLOP/byte	算術強度、roofline

問題	具體表現	分析工具
優化的上限	只改善一部分，整體加速有上限	時間公式、Amdahl's Law

- 記住時間公式： $T \approx T_{\text{compute}} + T_{\text{memory}}$ （沒重疊）或 $\max(\dots)$ （有重疊）。
- 接下來兩章的每一招，都能對應到三層框架中的一層、時間公式中的一項。

自我檢測

讀完這一章，試著回答下面的問題（參考答案在附錄 C）：

1. L 從 1,000 變成 2,000，一份 pair tensor (FP32) 從多大變成多大？為什麼是這個倍數？
2. 為什麼說長序列的瓶頸在 activation，而不是模型權重？
3. v0 的算術強度是多少 FLOP/byte？這個數字代表什麼？
4. 寫出「沒有重疊」與「完全重疊」時的執行時間公式。Double buffering 改變的是哪一點？
5. 把 tiling、INT8 量化、寬位元介面分別放進「資料表示／資料流／硬體結構」三層中的哪一層？

下一章開始講解法：先看不改變資料格式、只改變「怎麼搬、放在哪、存不存」的資料流優化。

第 3 章 資料流優化：少搬、搬得快、不存中間值

本章重點

- 資料流優化不改變資料格式，只改變資料「怎麼搬、放在哪、什麼順序算、存不存」。
- Tiling 減少搬運總量；寬位元與 burst 提高每次搬運的效率；double buffering 把等待藏起來；多通道 HBM 多條路同時搬。
- 融合讓中間值不寫回記憶體；重算用計算換記憶體；分塊與 token-wise attention 也是同一類想法。
- 每一招都對應第 2 章時間公式裡的一項，也都有代價。

3.1 Tiling／資料重用：減少搬運總量 ($v0 \rightarrow v1$)

白話直覺

$v0$ 每用一次食材就跑一趟倉庫。 $v1$ 一次搬一批到料理台，把這批能做的菜都做完，再搬下一批。

為什麼能重用？

看算式 $z[i][j][c] = \sum_k a[i][k][c] \times b[j][k][c]$ ：

- $a[i][k][c]$ 會被所有的 j 用到 (L 次)。
- $b[j][k][c]$ 會被所有的 i 用到 (L 次)。

如果一次處理 T_I 個 i 和 T_J 個 j (一個 tile)，那搬進來的每個 a 可以用 T_J 次，每個 b 可以用 T_I 次。



對每個 k ：

搬 $a[i0..i0+TI-1][k][:]$ ($T_I \times C$ 個數)

搬 $b[j0..j0+TJ-1][k][:]$ ($T_J \times C$ 個數)

做 $T_I \times T_J \times C$ 次乘加 ← 每個 a 用 T_J 次、每個 b 用 T_I 次

實際的行為：用 $L = 4$ 走一遍

為了看清楚，用最小的例子： $L = 4$ 、只看 1 個 channel ($C = 1$)、tile 大小 $T_I = T_J = 2$ 。要算的是 $4 \times 4 = 16$ 個輸出 $z[i][j]$ ，每個都要把 $k = 0 \sim 3$ 的 4 個乘積加起來，總共 64 次乘加。

$v0$ ：每次乘加都去 HBM 拿兩個數

```

for i = 0..3:
  for j = 0..3:
    acc = 0
    for k = 0..3:
      acc += a[i][k] × b[j][k]
    z[i][j] = acc

```

← 一個暫存器
← 每次都從 HBM 讀 a[i][k]、b[j][k]
← 寫回 HBM

實際發生的讀取（前兩個輸出）：

```

算 z[0][0] : 讀 a00 b00 → 讀 a01 b01 → 讀 a02 b02 → 讀 a03 b03
算 z[0][1] : 讀 a00 b10 → 讀 a01 b11 → 讀 a02 b12 → 讀 a03 b13
算 z[0][2] : 讀 a00 b20 → ...
算 z[0][3] : 讀 a00 b30 → ...

```

← a00~a03 又讀了一次！
← 再讀一次
← 再讀一次

- 同一個 a00 被讀了 4 次（每個 j 一次），同一個 b00 也被讀了 4 次（每個 i 一次）。
- 總共讀 128 次（64 次乘加 × 2）。晶片內只放一個累加器，什麼都沒有留下來重用。

v1：一次搬一小批進晶片，用到不能再才丟

```

for 每個 2×2 的 tile (i0, j0) :
  acc[2][2] = 0
  for k = 0..3:
    la[0..1] ← 從 HBM 讀 a[i0][k]、a[i0+1][k]
    lb[0..1] ← 從 HBM 讀 b[j0][k]、b[j0+1][k]
    4 次乘加：acc[ii][jj] += la[ii] × lb[jj]
  把 4 個 acc 寫回 z

```

← 共 4 個 tile
← 4 個累加器放在晶片內（BRAM）
← 讀 2 個
← 讀 2 個
← 每個 la 用 2 次、每個 lb 用 2 次
← 這個 tile 做完

第一個 tile（負責 z00、z01、z10、z11）實際發生的事：

```

k=0 : 讀 a00 a10 b00 b10 → acc00+=a00·b00  acc01+=a00·b10  acc10+=a10·b00  acc11+=a10·b10
k=1 : 讀 a01 a11 b01 b11 → acc00+=a01·b01  acc01+=a01·b11  acc10+=a11·b01  acc11+=a11·b11
k=2 : 讀 a02 a12 b02 b12 → ...
k=3 : 讀 a03 a13 b03 b13 → ...
寫回 z00 z01 z10 z11

```

- 每讀一次 a00，就在晶片內用 2 次（給 z00 和 z01）；v0 則是每用一次就要讀一次。
- 一個 tile 讀 $4 \times 4 = 16$ 次，4 個 tile 共 64 次，是 v0 的一半。
- a00 總共只被讀 2 次（第一列的兩個 tile 各一次），v0 是 4 次。

	v0	v1 (2×2 tile)
晶片內放什麼	1 個累加器	la (2 個)、lb (2 個)、acc (4 個)
每讀一個 a，用幾次	1	TJ = 2
同一個 a 總共被讀幾次	L = 4	$L \div TJ = 2$
總讀取次數	128	64

把例子放大到真實情況：實際的 L = 256、C = 128、tile = 8×8。

- 每個 a 在晶片內重用 8 次，讀取量變成 v0 的 1/8。

- 「讀 $a[i0][k]$ 」一次讀的是一整個 token 的 128 個連續的數，所以可以用 burst 一口氣讀完（3.2 節）。
- $v0$ 的最內層迴圈是 k ，每次跳 128 個數（ $a[(i \cdot L + k) \cdot C + c]$ ， k 加 1 位址就跳 C ），沒辦法 burst，這是它慢的第二個原因。

下面把這個行為寫成一般的公式。

公式推導：總搬運量

$v0$ （沒有重用）：每次乘加讀一個 a 和一個 b 。

$$\text{讀取次數} = 2 \times L^3 \times C \quad (\text{每個 } z[i][j][c] \text{ 要 } L \text{ 次乘加})$$

$v1$ (tiling)：

$$\begin{aligned} \text{tile 的數量} &= (L/TI) \times (L/TJ) \\ \text{每個 tile、每個 } k \text{ 的讀取} &= (TI + TJ) \times C \\ k \text{ 的數量} &= L \\ \text{讀取次數} &= (L/TI) \times (L/TJ) \times L \times (TI + TJ) \times C \\ &= L^3 \times C \times (1/TI + 1/TJ) \end{aligned}$$

比值：

$$\begin{aligned} v0 / v1 &= 2 \div (1/TI + 1/TJ) \\ TI = TJ = 8 &\rightarrow 2 \div (1/8 + 1/8) = 8 \text{ 倍} \\ TI = TJ = 16 &\rightarrow 2 \div (1/16 + 1/16) = 16 \text{ 倍} \\ TI = TJ = 32 &\rightarrow 32 \text{ 倍} \end{aligned}$$

算術強度：

$$\begin{aligned} AI &= 2 \cdot L^3 \cdot C \div (4 \text{ bytes} \times L^3 \cdot C \cdot (1/TI + 1/TJ)) \\ &= TI \cdot TJ \div (2 \cdot (TI + TJ)) \\ 8 \times 8 &\rightarrow 64 \div 32 = 2 \text{ FLOP/byte} \\ 16 \times 16 &\rightarrow 256 \div 64 = 4 \text{ FLOP/byte} \\ 32 \times 32 &\rightarrow 1024 \div 128 = 8 \text{ FLOP/byte} \end{aligned}$$

代價：晶片內記憶體

累加器要放下整個 tile 的輸出： $TI \times TJ \times C$ 個 float。

Tile	累加器大小	佔晶片內記憶體
8×8	8,192 個 float = 32 KB	很少
16×16	32,768 個 = 128 KB	少
32×32	131,072 個 = 512 KB	開始明顯

Tile 越大 → 搬運越少，但晶片內記憶體用越多。這就是 tile size sweep 實驗要量的 trade-off。

這是「解法」的原因

Tiling 不改變計算結果，只改變計算順序，卻讓搬運總量直接除以 4~16 倍。這是所有矩陣運算加速器（GPU 的 cuBLAS、TPU、LightNobel 的運算單元）都會用的基本手法。

3.2 寬位元與 burst：提高每次搬運的效率 (v1 → v2)

白話直覺

v1 雖然減少了趟數，但推車很小，一次只能載 1 個 float (32 bit)。v2 換成大推車，一次載 16 個 float (512 bit)，並且 16 個廚師同時切。

兩個不同的概念：頻寬 vs 延遲

概念	意義	比喻
頻寬 (bandwidth)	每秒能搬多少 byte	高速公路有幾線道
延遲 (latency)	從發出請求到拿到第一筆資料要多久	從家裡開到倉庫要多久

一條 AXI 連接埠 (FPGA kernel 連到 HBM 的介面)：

頻寬上限 = 位元寬度 × 時脈
32-bit × 300 MHz = 1.2 GB/s (v1)
512-bit × 300 MHz = 19.2 GB/s (v2) ← 16 倍

Burst (連續搬運)

每次向 HBM 發出請求都要付一次延遲。如果要的資料在記憶體中是連續的，可以一次請求、連續拿很多筆，這叫 burst。

非 burst：請求 → 等 → 拿 1 筆 → 請求 → 等 → 拿 1 筆 → ... (每筆都付一次延遲)
Burst：請求 → 等 → 拿 1 筆、1 筆、1 筆、1 筆 ... (只付一次延遲)

這就是為什麼資料排列 (memory layout) 很重要：我們把資料排成 [row][col][channel]，同一個 token 的 128 個 channel 是連續的，所以一次可以 burst 讀完。v0 在最內層迴圈改變 k，每次跳過 128 個數字 (stride = C)，無法形成有效的 burst，這也是它慢的原因之一。

Little's Law：要多少資料「在路上」才能用滿頻寬

在途資料量 = 頻寬 × 延遲

例：頻寬 19.2 GB/s、延遲約 64 cycles (300 MHz 下約 213 ns) → 需要約 4 KB 的資料同時在路上，才能讓頻寬不空轉。一次只要一小筆、等到了才要下一筆，永遠用不滿頻寬。這也說明了 burst 和下一節 double buffering 的重要性。

實際的行為：同一個 tile 的一輪 k，v1 和 v2 各做了什麼

接著 3.1 節的真實大小：tile = 8×8、C = 128、每個數是 32-bit float。看其中一輪 k（載入 8 個 a token、8 個 b token，再做乘加）。

一個 token 在記憶體裡的樣子（128 個 float，連續存放）：

```
位址 → c0 c1 c2 ... c15 | c16 ... c31 | ... | c112 ... c127
v1 看到的：128 個 32-bit 的數，一次拿 1 個          → 要 128 拍
v2 看到的：word0      | word1      | ... | word7      → 每個 word 512 bit = 16 個數，8 拍拿完
```

v1：一個 cycle 讀 1 個數、做 1 次乘加

```
load_a：8 個 token × 128 個數 = 1,024 拍
load_b：8 個 token × 128 個數 = 1,024 拍
mac    ：8 × 8 個輸出 × 128 個 channel = 8,192 次乘加 → 8,192 拍（一次一個）
這一輪 k 約 1 萬拍
```

v2：一個 cycle 讀 1 個 word（16 個數）、做 16 次乘加

```
load   ：8 個 token × 8 個 word = 64 拍，加上一個延遲與啟動 77 拍      = 141 拍（a、b 兩個 port 同時讀）
mac    ：8 × 8 × 8 個 word = 512 拍，加上 pipeline 深度 14 拍          = 526 拍（16 個乘法器一起算）
這一輪 k ≈ 141 + 526 + 4 = 671 拍                                     ← 和 HLS 報告一致
```

Burst 在時間軸上的樣子（v2 載入 a 的 64 個 word）：

```
cycle:  0 ————— 約 77 — 78 — 79 — 80 — ... — 141
AXI:    發出一個讀取請求      word0 word1 word2 ... word63
        └── 延遲（等 HBM） ─┘ └── 每拍到一個 word，不用再等 ─┘
```

如果不是 burst（每個 word 都各發一次請求、各等一次），同樣 64 個 word 要付 64 次延遲，時間會變成好幾千拍。

	v1	v2	倍數
每拍搬幾個數	1	16	16×
每拍做幾次乘加	1	16	16×
一輪 k 的 cycle	約 10,000	671	約 15×

倍數沒有剛好 16，是因為 v2 多了固定的延遲（77）和 pipeline 深度（14），這些不會因為變寬而縮短，下一節的 double buffering 就是要處理它們。

這是「解法」的原因

寬位元讓每個 cycle 能搬的量變 16 倍；burst 讓延遲只付一次。兩者結合，同樣的搬運量所需的時間大幅下降。同時，因為一次拿到 16 個數，也能讓 16 個運算單元同時工作，運算能力也跟著提高 16 倍（v2 理論上

限 = $16 \times 2 \times 300 \text{ MHz} = 9.6 \text{ GFLOP/s}$) 。

3.3 Double buffering：把等待時間藏起來 (v2 → v3)

白話直覺

v2 的廚師：搬一批 → 煮一批 → 搬下一批 → 煮下一批，搬的時候廚師在等。v3 多準備一個料理台：廚師煮第 k 批的同時，助手把第 k+1 批搬到另一個料理台。

v2 (沒有重疊)：

時間 $\xrightarrow{\hspace{1.5cm}}$

[搬 k=0][算 k=0][搬 k=1][算 k=1][搬 k=2][算 k=2]

v3 (double buffering, 兩組 buffer 輪流用)：

[搬 k=0][算 k=0][算 k=1][算 k=2][算 k=3]

 [搬 k=1][搬 k=2][搬 k=3]

 ↑ 搬運藏在計算底下

用實際數字看：v2 的 HLS 報告

每一輪 loop_k：

步驟	cycles
載入 a、b (兩個並行)	141
計算 (mac)	526
合計	約 671

套用 2.4 節的公式：

v2 (輪流)： $T \approx T_{\text{compute}} + T_{\text{memory}} = 526 + 141 \approx 671 \text{ cycles}$

v3 (重疊)： $T \approx \max(T_{\text{compute}}, T_{\text{memory}}) = \max(526, 141) \approx 530 \text{ cycles}$

時間減少 $\approx (671 - 530) \div 671 \approx 21\%$ → 速度約 1.27 倍

實際的行為：用 L = 4 看 v3 的兩組 buffer 怎麼輪流

v3 的程式裡有兩組 buffer：buf0 (la0、lb0) 和 buf1 (la1、lb1)。規則是：

迴圈開始前：把 k=0 載入 buf0

k 是偶數：mac 用 buf0，同時把 k+1 載入 buf1

k 是奇數：mac 用 buf1，同時把 k+1 載入 buf0

用 L = 4 (一個 tile 只有 4 輪 k)、真實的 cycle 數畫出時間軸：

時段 (cycle)	計算單元 (mac)	載入單元 (load)
0 ~ 141	閒置 (還沒有資料)	k=0 → buf0

時段 (cycle)	計算單元 (mac)	載入單元 (load)
141 ~ 671	算 k=0 (讀 buf0)	k=1 → buf1 (141 拍就完成，之後閒置)
671 ~ 1201	算 k=1 (讀 buf1)	k=2 → buf0
1201 ~ 1731	算 k=2 (讀 buf0)	k=3 → buf1
1731 ~ 2261	算 k=3 (讀 buf1)	閒置 (沒有下一輪了)

- 每一輪只要 $\max(526, 141) + 4 = 530$ 拍，load 完全藏在 mac 底下。
- 為什麼第 3 列可以放心把 k=2 寫進 buf0？因為 k=0 的 mac 在上一列就已經讀完 buf0 了。如果只有一組 buffer，載入 k+1 會蓋掉 mac 還在讀的 k 的資料——這就是需要兩組的原因（硬體上叫 WAR hazard，先讀後寫的衝突）。
- 代價：la、lb 的 BRAM 用量變 2 倍。

比較同一個 tile：

v2：4 輪 $\times 671 = 2,684$ 拍
v3：第一次載入 141 + 4 $\times 530 = 2,261$ 拍 → 快 1.19 倍

L 越大，開頭那一次 141 拍的比例越小： $(141 + L \times 530) \div (L \times 671)$ ，L = 256 時就接近 $671 \div 530 \approx 1.27$ 倍。

為什麼上限只有約 21%？

因為搬運只佔 21% 的時間。即使把搬運完全藏起來，最多也只能省下這 21%。這是 Amdahl's Law 的例子（見 2.6 節）。

延伸觀察：141 cycles 裡大部分是「延遲」

v2 每輪只搬 $2 \times 64 = 128$ 個 512-bit word（a、b 各 64 個，兩個 port 並行），理想上只要約 64 cycles，但報告顯示 141 cycles。多出的時間大多是 HBM 的延遲（HLS 預設假設 64 cycles）和迴圈的啟動成本。Double buffering 對付的正是這種「延遲型」的等待：延遲再長，只要能與計算重疊，就不會影響總時間。

這是「解法」的原因

Double buffering 不減少搬運量，也不提高頻寬，而是改變時間的結構：從「相加」變成「取最大值」。當計算和搬運的時間接近時，效果最好（最多快 2 倍）。

3.4 多通道 HBM：多條路同時搬

U55C 的 HBM 結構

16 GB HBM2 = 32 個 pseudo-channel (PC)，每個 512 MB、約 14 GB/s

PC0	PC1	PC2	...	PC31
-----	-----	-----	-----	------

總頻寬約 460 GB/s (全部同時用的情況下)

單一 PC 的頻寬和容量都不大，要同時使用多個 PC，才能發揮 HBM 的優勢。

本專題的兩種設定

設定檔	a	b	z	預期效果
trimul_v2.cfg	PC 0~7	PC 8~15	PC 16~23	三份資料走不同的路，互不干擾
trimul_v2_onebank.cfg	PC 0	PC 0	PC 0	三份資料搶同一條路

實際的行為：一輪 k 的載入，分開放和擠在一起差多少 (估算)

以 v2 的一輪 k 為例：a 的 8 個 token = $8 \times 128 \times 4 \text{ bytes} = 4 \text{ KB}$ ，b 也是 4 KB。單一 PC 約 14 GB/s，300 MHz 下一拍約 3.3 ns。

分開放 (a 在 PC0、b 在 PC8)：
PC0：a 的 4 KB $\div$ 14 GB/s $\approx 0.29 \mu\text{s} \approx 88 \text{ 拍}$ ，同時進行
PC8：b 的 4 KB $\div$ 14 GB/s $\approx 0.29 \mu\text{s} \approx 88 \text{ 拍}$ $\rightarrow$ 約 88 拍 (再加延遲)

擠在一起 (a、b 都在 PC0)：
PC0：先服務 a、再服務 b (或交錯)，總共 8 KB $\div$ 14 GB/s $\approx 0.57 \mu\text{s} \approx 176 \text{ 拍}$ (再加延遲)

	分開放	擠在同一個 PC
一輪 k 的載入 (估算)	約 88 拍 + 延遲	約 176 拍 + 延遲
v2 (載入和計算相加)	會變慢	多約 90 拍，變慢約 13%
v3 (載入藏在 526 拍的計算底下)	看不出差別	也可能看不出差別 ($176 < 526$)

這個例子說明兩件事：

1. 通道衝突的影響要看「搬運是不是在關鍵路徑上」。v3 已經把載入藏起來，只要擠在一起後仍然比計算短，就不會變慢。
2. 反過來，當計算變快 (v4 一拍 64 個乘法、tile 變大、或多個 kernel 同時跑)，載入時間不再藏得住，這時分開放的效果才會顯現。

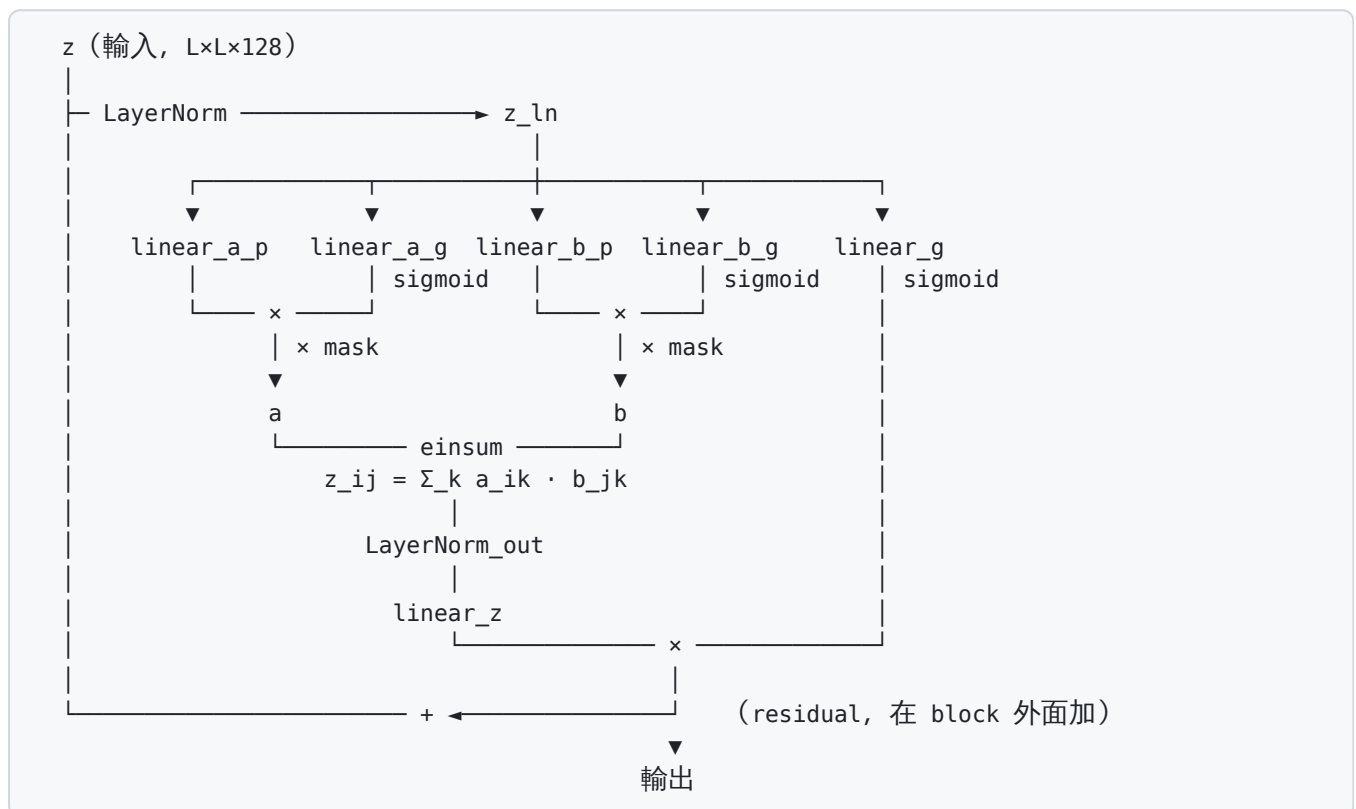
⚠ 以上是用平均頻寬做的估算，沒有考慮 HBM 的 row switch、refresh 和 AXI 的交錯方式；真實數字要上板量 (第二階段)。

這是「解法」的原因

頻寬是「每條路的頻寬 × 同時使用的路數」。讓不同資料走不同的 PC，等於把公式中的 頻寬 乘上倍數；反之，全部擠在同一個 PC 會互相排隊。這個實驗需要上板才看得出效果（HLS 報告無法模擬 HBM 的真實行為）。

3.5 運算融合：讓中間值不寫回記憶體

前面幾節的例子（v0~v3）只處理中間的 einsum。實際上 ESMFold 的 Triangle Multiplication（outgoing）是這樣（每個運算的意義見 1.5 節）：



每一個箭頭的結果都是 $L \times L \times 128$ 的張量。這些 linear 都是「每個 token 各自做 128×128 的矩陣乘法」，不同 token 之間互不相干。

一般實作會「落地」多少資料？

如果每個運算都是獨立的函式（PyTorch 預設的做法），每一步的結果都要寫回記憶體，下一步再讀出來。同一時間存在的 pair 張量，峰值大約是 7 份（z、z_ln、a、b、g、einsum 的結果、各種暫存）。GPU 上的融合實作分析也是這個量級（見 10.6 節的參考資料）。

白話直覺

不融合：廚師切完洋蔥放回倉庫，另一個廚師再從倉庫拿出來炒。融合：切完直接下鍋，洋蔥不回倉庫。

正式說法：producer-consumer

- Producer：產生資料的運算（例如 linear_a_p）。

- Consumer：使用資料的運算（例如 gating 的乘法）。
- 如果 consumer 在 producer 產生資料後馬上用掉，資料就可以留在晶片內（暫存器、BRAM），不必寫回 HBM。
- 這個「產生到使用之間的距離」叫做 reuse distance。距離越短，越容易融合。

實際的行為：一個 token 在兩種做法下走過的路

以一個 token (i, k)、FP32（每個 token $128 \times 4 = 512$ bytes）為例。

不融合（每一步都是獨立的函式，結果寫回 HBM）：

```

1. 讀 z[i][k] (512 B)      → LayerNorm      → 寫 z_ln (512 B)
2. 讀 z_ln                 → linear_a_p      → 寫 a_p (512 B)
3. 讀 z_ln                 → linear_a_g → sigmoid → 寫 a_g (512 B)
4. 讀 a_p、a_g             → 相乘 × mask   → 寫 a (512 B)
5~7. b 同樣走一遍         → 寫 b_p、b_g、b
8. 讀 z_ln                 → linear_g → sigmoid → 寫 g (512 B)

```

這個 token 的中間值在 HBM 上落地 8 次 (z_ln、a_p、a_g、a、b_p、b_g、b、g)，每次都要寫出去再讀回來。

融合（producer kernel，一個 token 全部在晶片內做完）：

```

讀 z[i][k] 的 128 個數 (512 B) → 暫存器 / BRAM
| LayerNorm：算平均、變異數 (2 個暫存器) → 128 個 z_ln，留在晶片內
| 5 個 linear：z_ln × 5 個 128×128 的權重 (權重常駐晶片內)
| sigmoid、gating、mask：逐元素，直接在 linear 的輸出上做
| 量化：找 max|a|、max|b| → 算尺 → 轉成 INT8
▼
寫出 a (128 B + 4 B 的尺)、b (128 B + 4 B) = 264 B      ← 這個 token 唯一落地的東西
(g 不寫出去，交給 consumer 重算，見 3.6 節)

```

每個 token	不融合	融合
從 HBM 讀 (不含 einsum)	約 10 次 × 512 B	1 次 × 512 B
寫回 HBM	8 次 × 512 B = 4 KB	264 B
L = 1000 時，中間值總寫入量	約 4 GB	約 264 MB

（不融合的次數依框架而不同，這裡是逐步落地的粗算；3.5 節開頭說的「約 7 份」指的是同時存在的峰值，不是總流量。）

硬體上要付出的：5 個 128×128 的權重要常駐晶片內 (FP16 約 160 KB，INT16 也一樣大)。這些權重被全部 L^2 個 token 重複使用，只載入一次——這和 3.1 節 tiling 的「重用」是同一個道理，只是重用的對象從 a、b 換成了權重。

為什麼 Triangle Multiplication 適合融合

LayerNorm、5 個 linear、sigmoid、gating、量化，全部都是「每個 token 自己算自己的」：

token (i, k) 的 128 個數
→ LayerNorm (只需要這 128 個數的平均與變異數)
→ 5 個 128×128 linear (只需要這個 token + 權重)
→ gating、mask、量化 (只需要這個 token)

一個 token 的所有處理都可以在晶片內完成，只有最後的 a、b 需要寫出去（因為 einsum 需要跨 token 的資料）。

和 LightNobel 的關係：LightNobel 的硬體就是以 token 為單位流動：RMPU 算完 linear，結果以管線方式直接交給 VVPU 做 LayerNorm、gating、量化，中間值不寫回外部記憶體（Sec. 5）。本節是同一個原則在 Triangle Multiplication 上的具體展開，並額外算出流量與容量各省多少——論文只給整體結果，沒有拆開。

本專題把這個原則落實成兩個 kernel 的設計，並估算流量與容量各省多少，見第 10 章 10.3 節。

3.6 重算：用計算換記憶體

問題

$g = \text{sigmoid}(\text{linear}_g(\text{LayerNorm}(z)))$ 在最後才會用到。如果在 producer 算好存起來，就要多存一份 $L \times L \times 128$ 。

做法：在 consumer 的 epilogue 從 z 重新算

多花的計算：每個輸出位置要多做一次 128×128 的 linear。

重算 g 的運算量 $= 2 \times C^2 = 2 \times 128 \times 128 \approx 32,768 \text{ FLOP}$ / 每個輸出位置
einsum 本身的運算量 $= 2 \times L \times C = 256 \times L \text{ FLOP}$ / 每個輸出位置

比例 $= 32,768 \div (256 \times L) = 128 \div L$
 $L = 256 \rightarrow 50\%$
 $L = 1024 \rightarrow 12.5\%$
 $L = 4096 \rightarrow \text{約 } 3\%$

序列越長，重算的代價越小，而省下的記憶體（每個 L^2 512 bytes）越重要。這正好就是長序列的情境。

實際的行為：consumer 算完一個輸出之後做什麼

Consumer (einsum kernel) 算完 $z_{\text{out}}[i][j]$ 的 128 個累加值之後，進入 epilogue（收尾階段）：

acc[128] (einsum 結果，在晶片內)
→ LayerNorm_out → linear_z → 得到 x[128]
→ 讀 $z[i][j]$ (512 B) → LayerNorm → linear_g → sigmoid → 得到 g[128] ← 重算
→ $x \times g$ → 寫出 (512 B)

和「producer 先把 g 存起來」比較， $L = 1000$ 、FP32：

	存 g	重算 g
額外的記憶體	$L^2 \times 512 \text{ B} = 512 \text{ MB}$	0 (z 本來就在, residual 也要用它)
consumer 每個輸出要讀	$g[i][j]$ (512 B)	$z[i][j]$ (512 B) —— 流量一樣
producer 要多寫	g (總共 512 MB)	不用
額外的計算	0	每個輸出多 32,768 FLOP, 約 einsum 的 $128 \div 1000 \approx 13\%$

重點：讀 z 和讀 g 的流量一樣大，所以重算不增加讀取量，只多花約 13% 的計算，就省下 512 MB 的容量和 512 MB 的寫入。

一般原則：Recomputation

「不存、需要時再算」是記憶體不夠時的常見手法（訓練大型模型時的 activation checkpointing 也是同樣的想法）。判斷值不值得的方法：

值得重算 $\Leftrightarrow$ 重算的時間 < 存取它所需的時間，或 記憶體容量是瓶頸

3.7 分塊、token-wise attention 與其他手法

分塊 (chunking) 可以看成「更上一層的 tiling」：tiling 把資料切小，放進晶片內的記憶體；chunking 把計算切小，讓中間值放得進 GPU/HBM。兩者的取舍相同：切得越小，峰值記憶體越低，但另一邊的資料要重複讀越多。

除了前面幾節，還有幾種常見手法 (v0~v4 沒有用到，但讀論文、做報告時會遇到)：

手法	原理	代價	在 PPM 的例子
Chunking (分塊)	一次只算一部分，算完就丟	時間變長	AlphaFold2/ESMFold 內建 chunk size 設定；LightNobel 的 GPU 比較對象用 Chunk4
Token-wise attention	attention 一次處理一部分 token，不存整個 score matrix	要線上追蹤最大值	LightNobel 的 Triangle Attention (原理見 6.6 節的線上 softmax)
低精度浮點 (FP16/BF16)	每個數用 2 bytes	範圍或精度下降	ESMFold 的基準精度就是 FP16；更低的位元數見第 4 章
稀疏化	跳過接近 0 的值	需要不規則存取	距離很遠的 (i, j) 對

實際的行為一：chunking 省了什麼、多付了什麼 (示意)

以 TriMul outgoing、 $L = 1000$ 、FP32 為例 (每份 $L \times L \times 128$ 張量 = 512 MB)。einsum 是 $z[i][j] = \sum_k a[i][k] \cdot b[j][k]$ 。把 i 切成每塊 64 列：

不分塊：a 整份 (512 MB) + b 整份 (512 MB) + 輸出整份 (512 MB) 同時存在
 分塊：for 每一塊 i (共 $\lceil 1000 \div 64 \rceil = 16$ 塊)：
 只算這 64 列的 a ($64 \times 1000 \times 128 \times 4 \text{ B} \approx 33 \text{ MB}$)
 和整份 b 做 einsum → 得到這 64 列的輸出 (約 33 MB) → 用完就丟

	不分塊	每塊 64 列
a 的峰值	512 MB	約 33 MB
b 要被讀幾次	1 次	16 次 (每塊都要整份 b)

這就是 3.1 節 tiling 的取捨換到記憶體階層上一層：塊越小，峰值越低，另一邊 (b) 重複讀越多次。

實際的行為二：線上 softmax 一批一批算 (token-wise attention 的核心)

假設某個 query 對 4 個 key 的分數是 $s = [2, 5, 1, 4]$ ，一次只能看 2 個。完整做法要先存下 4 個分數：

$$m = 5, \ell = e^{(2-5)} + e^{(5-5)} + e^{(1-5)} + e^{(4-5)} = 0.0498 + 1 + 0.0183 + 0.3679 = 1.4360$$

線上做法只維護 m (目前最大值) 和 ℓ (目前總和)：

第 1 批 [2, 5] : $m = 5, \ell = e^{(-3)} + e^0$	= 1.0498	
第 2 批 [1, 4] : 新最大值仍是 5, 舊 ℓ 不用縮放		
$\ell = 1.0498 + e^{(1-5)} + e^{(4-5)}$	= 1.4360	← 和完整做法一樣

如果順序反過來，最大值會在中途改變：

第 1 批 [1, 4] : $m = 4, \ell = e^{(-3)} + e^0$	= 1.0498	
第 2 批 [2, 5] : 新最大值 $5 > 4 \rightarrow$ 舊 ℓ 要乘 $e^{(4-5)}$ 重新對齊		
$\ell = 1.0498 \times e^{(-1)} + e^{(2-5)} + e^{(5-5)}$	= 1.4360	← 還是一樣

每一批算完就丟，晶片內永遠只有 m 、 ℓ (和輸出累加 o)。對 Triangle Attention 來說，被省掉的就是 $L \times L \times L$ 的 score matrix (公式見 6.6 節)。

3.8 總整理：每一招解決了什麼

問題	手法	從什麼概念上解決
搬運太多	Tiling	利用資料重用，減少重複搬運
搬運沒效率	寬位元、burst	每次搬更多、延遲只付一次
等待延遲	Double buffering	讓搬運和計算重疊
頻寬不夠	多通道	同時使用多條路
資料太大、放不下	量化 (第 4 章)	每筆資料變小，同時降低容量與頻寬需求
中間值太多	融合、重算 (3.5、3.6 節)	中間值不寫回記憶體

本章小結

手法	縮小時間公式的哪一項	代價	本專題
Tiling	搬運量 ↓	晶片內記憶體	v1
寬位元、burst	頻寬 ↑、等待次數 ↓	介面與運算單元變多	v2
Double buffering	+ 變成 max	兩組 buffer	v3
多通道 HBM	頻寬 ↑	要上板才看得到	cfg 對照
融合、重算	中間值的份數 ↓	設計複雜、多花計算	第 10 章的設計

效益不大時，通常是瓶頸轉移了（2.6 節），這本身就是有價值的發現。

自我檢測

讀完這一章，試著回答下面的問題（參考答案在附錄 C）：

1. $T_I = T_J = 16$ 時，tiling 後的讀取量是 v_0 的幾分之一？算術強度是多少？
 2. v_2 每輪 `loop_k` 671 cycles，其中載入 141 cycles。Double buffering 最多能快多少？為什麼不是 2 倍？
 3. 頻寬和延遲有什麼不同？寬位元和 burst 分別改善哪一個？
 4. 為什麼 Triangle Multiplication 裡除了 `einsum` 之外的運算都可以融合？
 5. $L = 512$ 時，在 epilogue 重算 g 大約多花多少比例的計算？
-

下一章看另一個方向：不改搬運方式，而是讓每筆資料變小——量化。

第 4 章 量化：讓每筆資料變小

本章重點

- 先認識 INT（二補數）與 FP（科學記號）在 bit 上的格式。
- 量化是壓縮，不是標準化：尺（scale）跟著資料存下來，用的時候乘回去。
- per-token 為什麼比 per-tensor 準；per-channel 又是什麼。
- 尺的成本與格式：每 128 個數多一個浮點數，只佔 1~3%；完整精度只在晶片內存在一下子。
- 尺在什麼時候乘，取決於加總的方向——這直接影響硬體設計。
- 位元數怎麼選，以及量化的代價、算術強度與容量上的好處。

4.1 量化的原理：壓縮，不是標準化

白話直覺

把食材壓縮成原本的 1/4 再搬，搬到後再還原。會有一點失真，但搬運量少了 4 倍。（1/4 是相對我們的 FP32；LightNobel 的基準是 FP16，INT8 相對 FP16 是 1/2、INT4 是 1/4。）

先認識格式：INT 與 FP 在 bit 上長什麼樣子

整數（INT）用二補數：最高位代表負的 2 的次方，其他位代表正的。

```
INT8 :  b7    b6    b5    b4    b3    b2    b1    b0
權重 : -128   64    32    16     8     4     2     1

5      = 0000 0101                (4 + 1)
-5     = 1111 1011                (-128 + 64 + 32 + 16 + 8 + 2 + 1)
-93    = 1010 0011                (-128 + 32 + 2 + 1)
負數的快速算法：正數的 bit 全部反轉再加 1 (0000 0101 → 1111 1010 → 1111 1011)
```

格式	範圍	例子
INT4	-8~7	5 = 0101、-5 = 1011
INT8	-128~127	-93 = 1010 0011
INT16	-32,768~32,767	1234 = 0000 0100 1101 0010

格子等間距（相鄰的值永遠差 1），硬體直接對 bit 運算，很簡單。（對稱量化只用 -127~127 或 -7~7，不用最負的那個值；而且量化後存的 q 不是真正的值，真正的值是 尺 $\times q$ 。）

浮點數（FP）是科學記號：bit 分成正負號、次方、有效數字三段：值 = $(-1)^{\text{正負號}} \times 1.\text{有效數字} \times 2^{(\text{次方} - \text{偏移量})}$ 。

$$6.5 = 110.1 \text{ (二進位)} = 1.101 \times 2^2$$

FP32 (1 + 8 + 23 bit, 偏移量 127) :

正負號 次方 (8 bit) 有效數字 (23 bit)
 0 10000001 10100000000000000000000 = 0x40D00000
 正數 129 - 127 = 2 「1.» 後面的 101 (開頭的 1 不存, 叫隱藏位)

-6.5 只有正負號不同 : 1 10000001 101000... (0xC0D00000)

0.000787 $\approx 1.612 \times 2^{-11}$: 0 01110100 10011100100111010101101 (116 - 127 = -11)

格式	正負號	次方	有效數字	最大值	大約精度	6.5 的樣子
FP32	1	8	23	約 3.4×10^{38}	約 7 位十進位	0 10000001 101000000000000000000000
FP16	1	5	10	65,504	約 3 位	0 10001 1010000000
BF16	1	8	7	約 3.4×10^{38}	約 2~3 位	0 10000001 1010000

格子不等間距：靠近 0 很密、越大越疏，所以能同時表示很大和很小的數。FP16 最小的正規數約 6×10^{-5} (4.3 節提醒尺太小時 FP16 會不準)；BF16 是把 FP32 的後 16 bit 砍掉，範圍和 FP32 一樣、精度較差。

對硬體的意義：整數乘法直接相乘；浮點乘法要「次方相加 + 有效數字相乘 + 正規化 + 處理 0、無限大、NaN」，成本大很多。所以量化不只讓資料變小，也讓運算單元變簡單：LightNobel 的 RMPU 全是整數乘法器 (6.4 節)，我們的 v4 也是 INT8 相乘、最後才乘 FP32 的尺。

INT8 對稱量化的原理

把一組浮點數 x 轉成 -127~127 的整數 q ：

```
scale s = max(|x|) ÷ 127      ← 讓最大的數剛好對應到 127
q       = round(x ÷ s)        ← 四捨五入成整數, 存 1 byte
還原     $\hat{x} = s \times q$ 
誤差     $x - \hat{x}$  落在  $[-s/2, +s/2]$  之間
```

誤差的大小取決於 s ： s 越大，每一格越粗，誤差越大。（在誤差均勻分布的假設下，均方根誤差約為 $s \div \sqrt{12} \approx 0.29 \times s$ 。）

量化是「壓縮」，不是「標準化」

初學者常見的疑問：「每個 token 用自己的尺，不就變成每個 token 只跟自己比的『標準分數』了嗎？」

不會。關鍵是尺 (scale) 會跟著資料一起存下來，用的時候乘回去 ($\hat{x} = s \times q$)，數值就回到原本的大小，token 之間照樣可以比較、一起運算。量化只是換一個比較省空間的記法；真正會改變數值意義的是 LayerNorm 那種「標準化」，兩者不要搞混。

比喻：量螞蟻用公釐尺、量房子用公尺尺，再在旁邊註明單位。每個東西用自己的尺，換算回來仍然能互相比較，而且都量得很準。如果全部都用公尺尺 (per-tensor)，房子很準，但螞蟻只能記成「0 公尺」。

另外，scale 不是從樣本「估計」出來的，而是直接取這 128 個數的最大絕對值，精確、不會估錯。一把尺只需要涵蓋它要記錄的那些數；涵蓋的範圍越小，刻度越細，誤差越小。這正是 per-token 準確的原因。

數字例子：兩個 token 一大一小

(為了簡化，每個 token 只看 3 個數。)

```
token 1 : [ 0.10, -0.05, 0.08 ]    ← 數值很小
token 2 : [ 30,   -12,   25   ]    ← 數值很大 (outlier token)
```

Per-tensor (兩個 token 共用一把尺) :

```
s = 30 ÷ 127 ≈ 0.236
token 1 : 0.10 ÷ 0.236 = 0.42 → 0
          -0.05 ÷ 0.236 = -0.21 → 0
          0.08 ÷ 0.236 = 0.34 → 0
還原 : [0, 0, 0]                ← token 1 的資訊全部消失
```

Per-token (每個 token 用自己的尺) :

```
token 1 : s1 = 0.10 ÷ 127 ≈ 0.000787
          q  = [127, -64, 102]
          還原 = [0.100, -0.0504, 0.0803] ← 幾乎完全一樣
token 2 : s2 = 30 ÷ 127 ≈ 0.236, 還原誤差也很小
```

代價只是每 128 個數多存一個 scale (4 bytes, 約 3%)。

4.2 Per-token、per-tensor、per-channel

為什麼 outlier 是問題：per-tensor vs per-token

我們的模擬資料：一般的數字大約落在 ± 3 以內，但有 1% 的 token 被放大 30 倍 (outlier, 大約 ± 80)。

Per-tensor (整份資料共用一把尺) :

```
s = 84 ÷ 127 ≈ 0.66
→ 一般 token (大約 ±3) 只能用到約 -4~4 這幾個整數
→ 均方根誤差 ≈ 0.29 × 0.66 ≈ 0.19    ← 相對於數值本身 (約 1) 將近 20% !
```

Per-token (每個 token 各自一把尺) :

```
一般 token : s ≈ 2.8 ÷ 127 ≈ 0.022 → 均方根誤差 ≈ 0.0064 (< 1%)
outlier token : s ≈ 0.66, 但它自己的數值本來就大, 相對誤差一樣很小
```

結論：per-tensor 時，少數 outlier 拖累了其他所有 token；per-token 讓每個 token 用適合自己的精度。我們的 C 模擬實測：per-token 輸出誤差約 1%，per-tensor 約 11%，與上面的推算方向一致。這正是 LightNobel 「token-wise」量化的核心動機。

那 per-channel 呢？

LLM 常用 per-channel（每個 channel 一把尺），因為 LLM 的 activation 是「某幾個 channel 特別大」。LightNobel 觀察到 PPM 相反：channel 之間差異小、token 之間差異大，outlier 集中在特定位置的 token（5.2 節），所以選 per-token。另一個好處是硬體：linear、LayerNorm 都是逐 token 做的，per-token 的 scale 剛好跟著 token 走，per-channel 則要在每個 token 內逐一反量化。第 10 章 10.4 節會用真實資料比較這三種粒度。

4.3 尺的成本與格式

加了尺，資料不是更多嗎？

尺是加上去了，但每個數字變小的幅度遠大於多出來的那把尺。以一個 token（128 個數）為例：

存法	128 個數本身	額外資料	一個 token 總共	相對 FP16
FP32（我們的 v2）	512 B	無	512 B	200%
FP16（論文的基準）	256 B	無	256 B	100%
INT8 + 尺（我們的 v4）	128 B	scale 4 B	132 B	約 52%
INT4 + 尺（論文的 C 組）	64 B	scale 2 ~ 4 B	約 68 B	約 27%
INT8 + 4 個 outlier + 尺（論文的 A 組，估算）	128 B	outlier 8 B + 位置約 4 B + scale 2 ~ 4 B	約 144 B	約 56%

尺只佔整個 token 的 1~3%：128 個數才多一個。比喻：一箱 128 顆蘋果原本每顆用大盒子裝，現在改用小盒子，只在箱子外面多貼一張「換算倍率」的標籤。

尺本身用什麼格式存？

尺是用浮點數存的（FP16 或 FP32），不是用幾個 bit 的整數。我們的 v4 用 FP32（4 bytes），見 trimul_v4.cpp 的 sa、sb；LightNobel 的論文沒有寫明尺的位元數，只說每個 token 的尺和資料存在一起（Fig. 7 的排列：inlier → outlier → 尺 → outlier 位置）。

「0.000787 小數點後這麼多位，不是要很多 bit 嗎？」不會，這正是浮點數的設計。浮點數就像科學記號，只存「有效數字」和「次方」：

$$0.000787 = 7.87 \times 10^{-4}$$
$$= 1.612 \times 2^{-11}$$

（科學記號：只要記 787 和 -4）
（電腦用 2 的次方）

FP16 的 16 個 bit：1 個正負號 + 5 個 bit 存次方（-11）+ 10 個 bit 存有效數字（1.612）

數字多小都一樣：小數點後有幾個 0 只影響「次方」，而次方只佔幾個 bit。
尺存起來的誤差可以忽略：FP16 的相對誤差最多約 $2^{-11} \approx 0.05\%$ ；量化本身每一格的誤差最多是最大值的 $0.5 \div 127 \approx 0.4\%$ ，大了將近 10 倍。（小提醒：FP16 能正常表示的最小值約 6×10^{-5} ，token 的值如果都

非常小，尺會小到 FP16 表示不準，所以很多實作用 FP32 或 BF16 存尺。我們用 FP32，沒有這個問題。)

算尺的時候不是要先有完整精度的資料嗎？

要，但完整精度的資料只在晶片內存在一下子，而且一次只有一個 token：

算出 token (i, j) 的 128 個數 (完整精度, 256~512 B, 放在晶片內)
→ 取最大值、算出尺、轉成 INT8/INT4
→ 只把壓縮後的版本寫進 HBM
→ 換下一個 token

真正佔記憶體體的 LxL 個 token，全部都是壓縮後的版本；完整精度的資料從來不會整份存下來 (10.4 節「量化在源頭完成」)。

4.4 尺在什麼時候乘？

情況一：linear (一個 token × 權重矩陣)。加總沿著同一個 token 的 128 個 channel，這 128 個數共用一把尺，所以可以把尺提出來：整數先乘加完，最後只乘一次尺。

$$y = \sum_c (s_x \times q_x[c]) \times W[c] = s_x \times \sum_c q_x[c] \times W[c]$$

LightNobel 的 linear 就是這樣做 (Sec. 8.3：「最後才乘一次尺，省掉重複的反量化」)。

情況二：Triangle Multiplication 的 einsum (本專題的 kernel)。加總沿著 k，也就是跨越不同的 token (i, k)。每個 k 是不同的 token，尺都不一樣，不能提出來：

$$z[i][j][c] = \sum_k (s_a[i][k] \times q_a[i][k][c]) \times (s_b[j][k] \times q_b[j][k][c]) \\ = \sum_k (s_a[i][k] \times s_b[j][k]) \times (q_a \times q_b) \quad \leftarrow \text{每一項都要乘自己的尺}$$

所以 v4 的 mac 迴圈每個 k 先算一次 $s = s_a \times s_b$ ，再讓 64 個 lane 的整數乘積都乘上 s 才累加。這就是 v4 的代價表中「每次乘加要多乘上 $s_a \times s_b$ 」的原因，也讓 v4 需要較多的 DSP。

數字例子：只看一個 channel，k 只有兩項。每個 token 用 2 個 channel 代表 ([c0, c1])，看 c1 的結果。

$a[i][0] = [0.10, -0.04]$ $a[i][1] = [2.0, 0.9]$ ← 一小一大
 $b[j][0] = [3.0, 1.2]$ $b[j][1] = [-0.4, 0.3]$
正確答案 (c1)： $(-0.04)(1.2) + (0.9)(0.3) = -0.048 + 0.270 = 0.222$

v4 的 mac 迴圈實際做的事：

k	s_a (= max ÷ 127)	q_a	s_b	q_b	整數乘積 (c1)	s = s_a × s_b	乘上 s
0	0.000787	[127, -51]	0.02362	[127, 51]	-51 × 51 = -2,601	1.860e-5	-0.04838
1	0.01575	[127, 57]	0.00315	[-127, 95]	57 × 95 = 5,415	4.960e-5	0.26858

累加： $-0.04838 + 0.26858 = 0.22021$ (誤差 0.8%)

如果錯誤地把尺提出來（整數先加完，最後乘 $k=0$ 的尺）：

$$(-2,601 + 5,415) \times 1.860e-5 = 0.0523 \quad \leftarrow \text{差了 4 倍多}$$

兩個 k 的尺差了將近 3 倍（ $1.86e-5$ vs $4.96e-5$ ），不能共用。這就是 v4 每個 k 都要先乘自己的 s 再累加的原因。

給硬體設計的觀察：如果改用 per-channel 的尺（每個 c 一把），尺在 k 方向上是固定的，就可以提出來、最後乘一次，硬體比較省；但精度可能比較差。這是「精度 vs 硬體成本」的取捨，10.4 節的真實資料誤差表（`rel_l2_grid`）可以幫忙判斷。

4.5 位元數怎麼選

INT4 只有 $-7 \sim 7$ 共 15 個整數，scale 是 INT8 的約 18 倍，誤差也大約大 18 倍。所以 INT4 只適合數值分布平均、outlier 少的資料。LightNobel 就是依這個原則分組：outlier 多的 A 組用 INT8，outlier 少的 C 組用 INT4（5.3 節）。

有 outlier 又想用低位元怎麼辦？常見的做法是混合精度：把每個 token 裡少數幾個特別大的值挑出來，用高精度（例如 INT16）另外存，其餘的值用低位元。這樣尺只需要涵蓋「一般的值」，刻度就細了。代價是要多存 outlier 的值和位置，而且挑出最大的幾個值（top-k）需要排序。LightNobel 的 A、B 組就是這樣做的（5.3 節），它用什麼硬體挑 top-k，見 6.5.2 節。

怎麼決定？用實驗掃過「位元數 $\times$ outlier 數量」，看精度和資料量的取捨，選最有效率的點。LightNobel 就是用這種設計空間探索決定三組的設定（5.3 節）；本專題會用真實資料做同樣的事（10.4 節）。

4.6 代價、算術強度與容量

代價

項目	代價
精度	約 1% 的相對誤差（per-token）
額外資料	每個 token 多存一個 scale（4 bytes / 128 bytes $\approx$ 3%）
額外運算	每次乘加要多乘上 <code>s_a $\times$ s_b</code> （v4 的 mac 迴圈）
量化本身	要先算出每個 token 的最大值（我們在 host 端做，沒有計入 kernel 時間；LightNobel 在硬體裡「執行時」做，見 10.4 節）

算術強度 (v4, 8×8 tile)

$$\begin{aligned} \text{資料量} &= L^3 \cdot C \cdot (1/TI + 1/TJ) \times 1 \text{ byte} && (\text{int8}) \\ &+ L^3 \cdot (1/TI + 1/TJ) \times 4 \text{ bytes} && (\text{scale}) \\ \text{AI} &= 2 \cdot L^3 \cdot C \div [L^3 \cdot (1/TI + 1/TJ) \cdot (C + 4)] \\ &= 2 \cdot 128 \div [(1/4) \cdot 132] \approx 7.8 \text{ FLOP/byte} && \leftarrow \text{約是 v2 的 3.9 倍} \end{aligned}$$

容量上的好處

版本	a、b 各佔	z	總計
FP32 (v2)	$L^2 \times 512 \text{ B}$	$L^2 \times 512 \text{ B}$	$L^2 \times 1,536 \text{ B}$
INT8 (v4)	$L^2 \times 132 \text{ B}$ (含 scale)	$L^2 \times 512 \text{ B}$ (輸出仍是 FP32)	$L^2 \times 776 \text{ B}$

總佔用量約減半 → 相同記憶體下，L 約可增加 $\sqrt{2} \approx 1.4$ 倍。（若輸出也量化，才會接近 2 倍，可以當延伸題目。）

注意：這是和我們自己的 FP32 版本比。LightNobel 的比較基準是 FP16（2.1 節），以 FP16 為基準時 z 只佔 256 B，量化 a、b 的效果會變小；和論文比較時請用 9.4 節的 FP16 數字。

這是「解法」的原因

量化同時縮小兩件事：搬運量（公式中的分子）和記憶體佔用（決定 L 的上限）。在 v0~v4 裡，這是唯一一個直接處理「序列長度受限」的手法，也是 LightNobel 的核心。（另一類處理容量的手法是「不存中間值」：融合、重算、LightNobel 的 token-wise attention，見第 3 章與 5.4 節。）

本章小結

問題	答案
量化在做什麼？	用低位元整數 + 一把尺記錄數字；還原 = 尺 × 整數
為什麼 per-token？	尺只需涵蓋自己的 128 個數，刻度細；outlier token 不會拖累別人
尺的成本？	每 128 個數一個浮點數，約 1~3%
尺什麼時候乘？	沿同一個 token 加總（linear）可以最後乘一次；跨 token 加總（本專題的 einsum）每項都要乘
效益？	資料量變 1/2~1/4；同時降低搬運量與容量需求

自我檢測

讀完這一章，試著回答下面的問題（參考答案在附錄 C）：

1. per-tensor 和 per-token 量化差在哪裡？為什麼有 outlier 時 per-tensor 的誤差特別大？
2. 每個 token 用自己的 scale，會不會讓不同 token 的數值「失去可比性」？為什麼？

3. 尺用什麼格式存？像 0.000787 這種很小的數，為什麼不需要很多 bit？
 4. 「量化在源頭完成」省下了什麼？
 5. 如果記憶體用量減半，可處理的最大序列長度 L 變成幾倍？
 6. 寫出 INT8 的 -5 的 bit 樣式。FP32 的 32 個 bit 分成哪三段？為什麼浮點乘法器比整數乘法器貴？
-

下一章進入案例：LightNobel 怎麼把量化做到極致（三組、混合精度、outlier），以及它的記憶體到底省在哪裡。

第 5 章 案例研究（上）：LightNobel 的觀察、演算法與成果

本章重點

- LightNobel (ISCA 2025) 的速讀卡：問題、觀察、解法、評估、結果。
- 三個關鍵觀察，以及從觀察推出的 AAQ (token-wise 自適應量化)：A、B、C 三組各用什麼精度、為什麼。
- 記憶體到底靠什麼省下來：量化（常數倍）與不存中間值（消掉 L^3 項）；問題是否真的被解決。
- 這個領域的其他做法，以及讀原文時要注意什麼。

5.1 論文速讀卡

項目	內容
標題	LightNobel: Improving Sequence Length Limitation in Protein Structure Prediction Model via Adaptive Activation Quantization
發表	ISCA 2025 (計算機架構領域頂尖會議)；arXiv 2505.05893
類型	硬體與軟體協同設計 (HW/SW co-design) 的加速器論文
對象	PPM (蛋白質結構預測模型)，以 ESMFold 為基準
問題	長序列時 activation 記憶體爆炸 → 可處理的序列長度受限
關鍵觀察	① 序列一長，執行時間集中在 pair representation，其中 Triangle Attention 暴增 (Fig. 3) ② activation 遠大於權重 (Fig. 4) ③ channel 之間差異小、token 之間差異大，而且 outlier 集中在特定 token (Fig. 5，與 distogram 的模式相關)
軟體解法	AAQ (Token-wise Adaptive Activation Quantization)：把 activation 依「在模型中的位置」分成 A、B、C 三組，各用不同的 inlier 精度與 outlier 數量；每個 token 一個 scale；執行時用 top-k 挑出 outlier (見下表)
硬體解法	RMPU (把資料切成 4-bit 小塊、可依精度重組的矩陣單元)、VVP (128 條 SIMD 的向量單元：LayerNorm、Softmax、residual、執行時量化、bitonic top-k 排序)、Token Aligner (把不同格式的 token 重新排齊)、crossbar、scratchpad (token 128 KB × 2 做 double buffering)；32 個 RMPU、每個配 4 個 VVP
評估方法	自寫的 Python cycle-accurate 模擬器 (和 RTL 交叉驗證，平均誤差 3.30%、都在 5% 內) + SystemVerilog RTL、Synopsys DC 合成 (28 nm、1 GHz)、CACTI 7.0 (SRAM)、Ramulator (80 GB HBM2E、2 TB/s)
對象與資料	ESMFold (ESM-2 3B)，Chunk4 設定；CAMEO、CASP14、CASP15、CASP16；和 NVIDIA A100、H100 比較

項目	內容
結果	TM-score 變化 < 0.001；速度最多快 8.44× (vs A100) / 8.41× (vs H100)；能效最多高 37.29× / 43.35×；峰值記憶體最多降 120.05× (和不分塊的 GPU 比；和分塊比是 1.26~5.05×)；80 GB 內最長可處理 9,945 個胺基酸
硬體成本	178.80 mm ² 、67.80 W；crossbar 佔 70% 面積、68% 功耗 (資料搬運與排序比運算本身還貴)

一句話總結論文

找到「token」這個合適的壓縮單位，並設計能有效率地處理這種格式的硬體，讓長序列放得下、也跑得快。

5.2 他們的觀察

四個觀察

1. 序列一長，執行時間幾乎都花在 pair representation (91.9%)，其中 Triangle Attention 最重 (1.4 節)。
2. PPM 的記憶體瓶頸主要來自 activation，而不是權重 (2.1 節)。
3. 同一個 token 的 128 個 channel 數值差不多，但 token 和 token 之間差很多，而且 outlier 集中在特定 token (和 distogram 的模式相關)。(這裡的 token 指 pair representation 中的一格，也就是一個 (i, j) 位置的 128 個數字。) → 所以「每個 token 一把尺」比「每個 channel 一把尺」更合適。
4. 不同位置的 activation 特性也不同：接 residual 的值很大 (平均絕對值約 82)、outlier 多；其他位置的值小 (約 4)、outlier 少。

我們重現了核心直覺

在模擬資料中放入 1% 的 outlier token：per-token INT8 的誤差約 1%，per-tensor INT8 約 11%，相差約 10 倍。這說明了 LightNobel 為什麼要以 token 為單位量化。

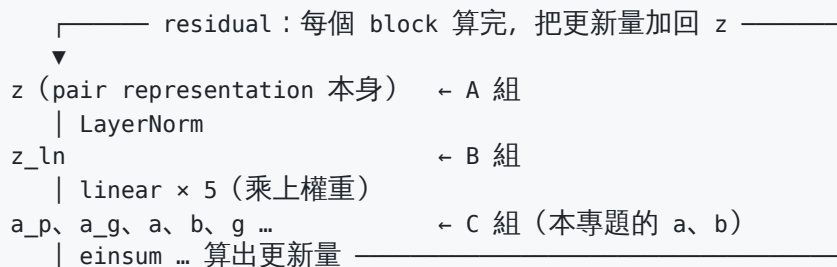
模擬資料和論文的差異：我們的 outlier 是「整個 token 放大 30 倍」，對應論文觀察 3 (token 之間差很多)；論文另外還有「token 內少數幾個值特別大」的 outlier，我們沒有模擬。另外，a、b 是 linear 之後的值，依論文的分法屬於 C 組 (outlier 很少)，所以我們的模擬其實比真實情況更嚴苛。用真實 ESMFold 資料 (8.9 節) 重跑，可以確認這一點。

5.3 AAQ：三組、混合精度與 outlier

AAQ 的三組設定 (Sec. 4.2、Fig. 6、Fig. 11)

組別	是哪些 activation	數值大小 (平均絕對值)	平均 outlier 數	選定的格式
A	LayerNorm 之前、接 residual 的 activation	82.14 (大)	2.31	INT8 inlier + 4 個 INT16 outlier
B	LayerNorm 之後、進 linear 之前	4.05	1.69	INT4 inlier + 4 個 INT16 outlier
C	其餘 (linear 之後的中間值等)	3.85	0.64	INT4, 不處理 outlier

三組在計算流程中的位置 (以 Triangle Multiplication 為例；和胺基酸的種類無關)：



為什麼三組的數值差這麼多 (白話)：

- A 組很大 (平均 82, 大部分落在約 ± 143 內)：pair representation 每經過一個 block 就加上一筆更新量, 48 個 block 一直累加、沒有被縮回來, 所以數值越滾越大, 也容易出現特別大的值。
- B 組變小 (平均 4, 約 ± 3.3)：LayerNorm 把每個 token 標準化到大小約 1, 但最後乘上的可學習係數 γ 會放大某些 channel, 所以仍有一些 outlier。
- C 組最平滑 (平均 3.9, 約 ± 2.3)：乘上權重矩陣時, 128 個數被混合加總, 特別大的值被攤平, outlier 最少, INT4 就夠。
- 量化方式：對稱量化, $\text{scale} = \text{該 token 的最大絕對值} \div (2^{(\text{bits}-1)} - 1)$ 。scale 會和資料一起存下來, 用的時候乘回去, 數值回到原本的大小 (為什麼這不是「標準化」, 見 4.1 節；尺的格式與成本見 4.3 節)。
- 為什麼用對稱量化：和非對稱量化相比, 對稱量化若不處理 outlier, RMSE 多 27.35%；處理 outlier 後只多 9.76% (實際數值差約 0.0004), 所以「對稱 + outlier 處理」就夠了, 硬體更簡單 (Sec. 4.1)。
- 每組的設定是用設計空間探索 (精度 $\times$ outlier 數, 看 TM-score 與資料大小) 選出來的 (Fig. 11)。
- 權重不量化, 用 INT16 (7.90 GB)；比較基準 (baseline) 的 activation 與權重都是 FP16 (Table 1)。

outlier 怎麼存：格式對硬體友善

- 每個 token 裡絕對值最大的 k 個值 (A、B 組 $k = 4$) 當作 outlier, 用 INT16 另外存；其他值 (inlier) 用 INT4 或 INT8。
- 資料依「inlier $\rightarrow$ outlier $\rightarrow$ scale $\rightarrow$ outlier 位置」的順序排好 (Fig. 7), 硬體讀取時不用到處跳。

- top-k 是執行時才算的（VVPUs 用 bitonic 排序），所以每個 token 的 outlier 位置可以不同。
- 白話：大件行李另外托運，其他的壓縮裝箱，箱子依序排好方便拿。

這些優化分別作用在哪裡？

LightNobel 只加速 Protein Folding Block (trunk)；ESM-2 語言模型和 structure module 不在它的硬體上 (Sec. 8.2)。

優化	作用在哪裡	沒有作用的地方
AAQ 量化 (A、B、C 組)	Folding Block 裡 pair representation 資料流的 activation (Sec. 4.1)	權重 (INT16, 不量化)；Fig. 6 裡畫成黑線的 activation
RMPUs	Folding Block 裡的矩陣運算 (linear、einsum、 $Q \times K \cdots$)	ESM-2、structure module
VVPUs	LayerNorm、softmax、residual、執行時量化、top-k	同上
Token-wise attention	Triangle Attention (不存 L^3 的 score matrix)	Triangle Multiplication 本來就沒有 score matrix

本專題的 einsum 對應到其中很小的一片：Triangle Multiplication 的 einsum 在 Folding Block 裡，輸入 a、b 屬於 C 組 (INT4)，在 RMPUs 上計算。einsum 並不是 LightNobel 特別挑出來優化的地方，選它是本專題自己的範圍決定 (10.1 節)。

試過但沒有採用的做法

嘗試	結果	出處
A 組用 INT4	要處理 32 個以上的 outlier 才保得住精度，資料反而變大	Fig. 11(a)
A 組用 INT8 但 outlier 少於 4 個	TM-score 下降	Fig. 11(a)
B 組用 INT8	精度沒問題，但資料變大，不划算	Fig. 11(b)
非對稱量化	對稱量化加上 outlier 處理就夠了，硬體更簡單	Sec. 4.1
channel-wise、tensor-wise 量化	和其他方法比較，token-wise 最好	Table 1、Sec. 3.3
每個 RMPUs 配超過 4 個 VVPUs；超過 32 個 RMPUs	效能不再提升，不值得多花面積	Fig. 12 (6.5.4、6.8 節)

5.4 記憶體是靠什麼省下來的？

LightNobel 的記憶體節省來自兩個不同的機制，論文把它們放在一起報告，所以不容易看出各自的貢獻。

兩個機制

機制	做了什麼	效果	LightNobel 的數字	本專題的對應
① 量化	每個數從 16 bit 變成 4~8 bit	每份資料變小 (常數倍)	資料流程相同時, activation 113.49 → 65.60 GB, 總量約 1/1.65 (Table 1)	v4 (INT8)、v5 (INT4) (第 4 章、10.4 節)
② 不存中間值	中間值算完馬上用掉, 不整份寫回記憶體	份數變少; Triangle Attention 的 L^3 項直接消失	和不分塊的 GPU 比, 峰值最多 1/120	融合設計 (約 7 份 → 約 2.5 份, 10.3 節)

機制 ② 為什麼效果大? GPU 不分塊時, Triangle Attention 會存下整個 $L \times L \times L$ 的 score matrix。以 ESMFold 的 4 個 head、FP16 粗估: $L = 1,410$ 約 22 GB, $L = 2,034$ 約 67 GB, $L = 3,364$ 約 300 GB。LightNobel 的 token-wise attention 算一部分、用掉、丟掉, 從來不存整個矩陣, 立方成長的部分就不見了 (原理見 6.6 節)。

看懂論文的「120 倍」

- 120.05 倍是和「不分塊」的 GPU 比 (GPU 會存下整個立方大小的 score matrix)。
- 和「有分塊」的 GPU 比只有 1.26~5.05 倍。
- 量化本身 (AAQ) 在同樣資料流下是 121.39 GB → 73.50 GB, 約 1.65 倍 (Table 1)。

所以大部分的節省來自機制 ②。本專題的融合 + INT8 讓最大序列長度提升約 1.52 倍 (9.4 節), 和論文中「量化 + 不落地」這部分的量級一致, 並不是做得特別差。

問題真的被解決了嗎? (誠實版)

沒有「根本解決」, 而是「把上限往後推了很多」。

- pair representation 仍然隨 L^2 成長。量化只是乘上一個常數, 曲線的形狀不變; 而 L 只和記憶體的平方根成正比。
- 真正大幅延長序列的, 是把 L^3 那一項消掉 (機制 ②)。
- 論文的成果: 80 GB 內最長 9,945 個胺基酸; GPU 不分塊時約 1,410。GPU 分塊也能跑長序列, 但慢很多 (LightNobel 在分塊的情況下快 3.67~8.44 倍)。

一句話: LightNobel 讓長序列「放得下」又「跑得快」, 但 L^2 的成長仍在。

5.5 領域地圖: 大家用什麼方法處理這個問題

方法類別	代表工作	解決什麼	代價 / 限制
GPU 系統優化	FastFold、ScaleFold	訓練和推論更快 (平行化、kernel 優化)	記憶體仍隨 L^2 成長
分塊 (chunking)	AlphaFold 內建、AutoChunk	峰值記憶體下降	變慢, 重複讀取資料

方法類別	代表工作	解決什麼	代價 / 限制
量化	MEFold (只量化權重)、PTQ4Protein (per-tensor INT8)	資料變小	只量化權重幫助有限；精度降低時 TM-score 下降
通用 LLM 量化	SmoothQuant、LLM.int8()、AWQ	outlier 分離	outlier 以外都用同一精度；GPU 上 activation 量化常常反而變慢 (W4A4 比 FP16 慢)
融合 kernel	OpenFold3 的 GPU 融合實作	中間值不落地，工作空間約 7U → 4U (分塊後約 2.2U)	要手寫 kernel，受 GPU 架構限制
專用加速器	LightNobel	量化 + 專用硬體 + 管線化資料流，記憶體與速度一起改善	以模擬器評估，沒有流片；ASIC 做出來後無法修改
FPGA 加速	本專題	逐步拆解各手法的貢獻；可重組、可實際部署的硬體	只做一個運算；絕對速度不如 GPU；目前以架構模擬評估

搜尋「FPGA + AlphaFold/Evoformer/Triangle Multiplication」沒有找到公開的 FPGA 實作，這是本專題的切入點之一（建議再用 Google Scholar 確認）。參考：[FastFold](#)、[AutoChunk](#)、[MEFold](#)、[OpenFold3 PR #318](#)

5.6 讀原文時的指引

建議閱讀順序

1. 摘要與 Introduction 最後的貢獻列表 → 對照本章 5.1 ~ 5.3 節。
2. 動機章節的圖 (activation 大小、token-wise 分布) → 這是「觀察」，最重要。
3. AAQ 的方法章節 → 找出三組的分法、精度、k 值。
4. 硬體架構圖 → 找出 RMPU、VVPUs、Token Aligner 的位置和資料流。
5. 評估章節 → 比較對象、資料集、主要圖表。

讀完原文後確認的答案

問題	答案 (出處)
三組的分類依據？各用什麼精度？	依 activation 在模型中的位置：A 組 INT8 + 4 個 outlier、B 組 INT4 + 4 個、C 組 INT4 無 outlier (Sec. 4.2、7.1)
top-k 的 k 是多少？	A、B 組 k = 4，C 組不處理；每個 token 在執行時各自挑選 (Sec. 4.2、5.3)
量化套用在哪些 activation？	PPM block 裡的 activation (Fig. 6 標出每個 activation 屬於哪一組)；權重不量化，用 INT16

問題	答案 (出處)
硬體怎麼評估？	Python cycle-accurate 模擬器 (和 RTL 交叉驗證) + Synopsys DC 28 nm 1 GHz + CACTI + Ramulator (Sec. 6)
最長可處理的序列？	80 GB 內 9,945 (CASP16 最長 6,879 的 1.45 倍) ; GPU 不分塊時單卡約 1,410
比較基準的精度？	ESMFold FP16 (activation 113.49 GB、權重 7.90 GB , T1169 3,364 aa ; Table 1)
有哪些限制？	論文指出：crossbar 佔 70% 面積、68% 功耗；GPU 上 activation 量化效率差，所以才需要專用硬體 (Sec. 8.4、9.3) 。我們的觀察：評估以模擬器為主，沒有實體晶片，硬體設計也沒有公開

本章小結

- 問題：長序列時 activation 以 L^2 (attention 以 L^3) 成長，GPU 放不下、也跑不快。
- 觀察：channel 之間差異小、token 之間差異大；不同位置的 activation 特性不同。
- 演算法：每個 token 一把尺；A 組 INT8 + 4 個 outlier、B 組 INT4 + 4 個 outlier、C 組 INT4。
- 記憶體：量化省常數倍 (約 1.65 $\times$) ；不存 L^3 的 score matrix 才讓峰值降到上百倍。
- 下一章：這些演算法要有效率，需要專門的硬體。

自我檢測

讀完這一章，試著回答下面的問題 (參考答案在附錄 C)：

1. LightNobel 的三個創新分別是什麼？各用一句話說明。
2. 為什麼 LightNobel 選擇以「token」為量化單位？
3. Triangle Multiplication 的 a、b 屬於 AAQ 的哪一組？這對我們選 INT8 有什麼意義？
4. LightNobel 的記憶體節省來自哪兩個機制？哪一個讓峰值記憶體降到上百倍？

下一章從硬體設計者的角度，拆開 LightNobel 的每個硬體單元：它們是什麼、原理是什麼、為什麼這樣設計。

第 6 章 案例研究（下）：LightNobel 的硬體架構

本章重點

- 第 5 章講了 LightNobel 的演算法；這一章從硬體設計者的角度，拆開每個單元的結構、原理與設計理由。
- 讀法：先看 6.1「需求 → 架構」，知道每個單元是為了解決哪個問題；再逐一看單元內部。
- 核心技巧有四個：位元拆解乘法（RMPU）、可重組的加法樹（PE Cluster、DAL）、SIMD 向量單元 + 排序網路（VVPU）、線上 softmax（token-wise attention）。
- 最重要的一課：這顆晶片 70% 的面積花在搬資料的 crossbar，不是運算單元。
- 資料來源：論文 Sec. 5（架構）、Sec. 6（評估方法）、Sec. 7.2（硬體設計空間探索）、Table 2（面積與功耗）、Fig. 8~10。論文沒有寫明的細節會標「推測」或「我們的分析」。

6.1 從需求推導架構：設計者面對的四個問題

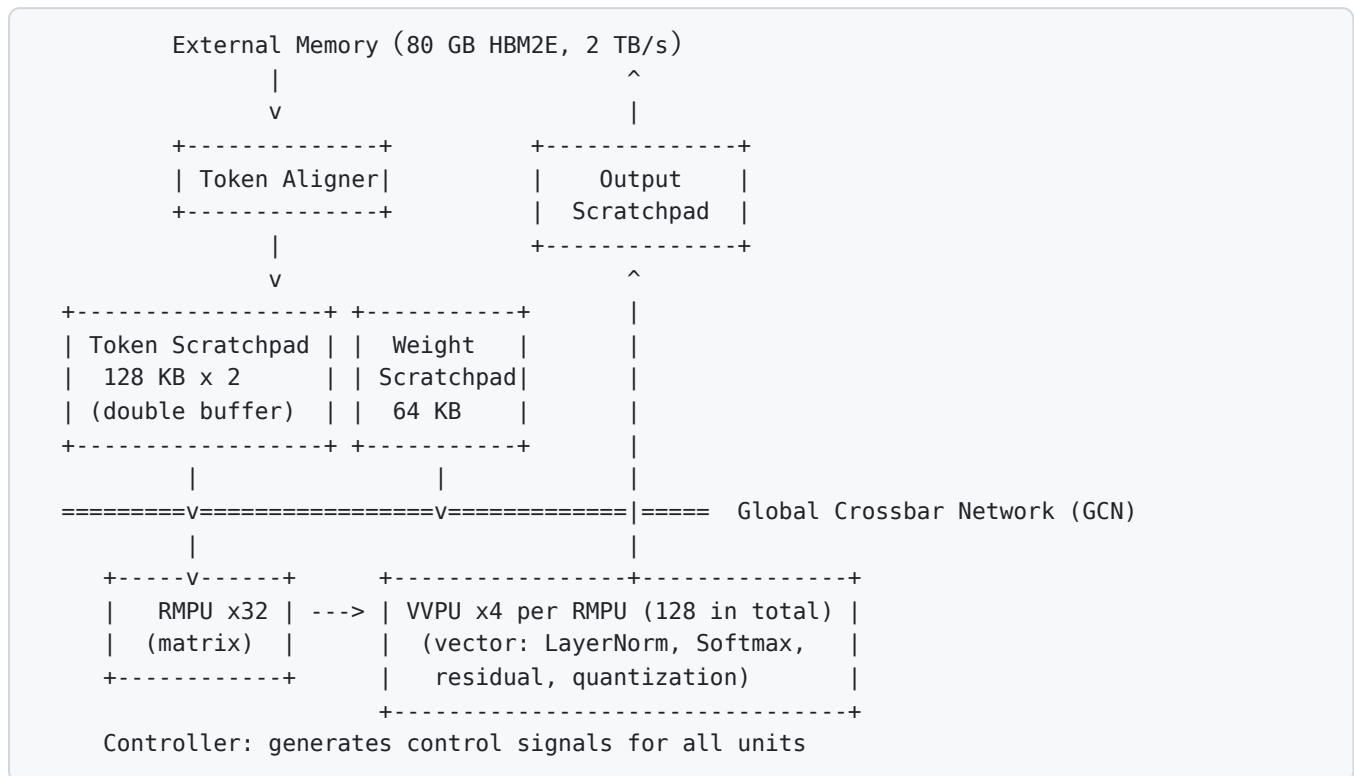
好的加速器不是先想「要放什麼單元」，而是先分析工作負載（workload）的特性，再讓每個單元對應一個需求。LightNobel 面對的 PPM 有四個特性：

#	工作負載的特性	帶來的硬體問題	LightNobel 的對應單元
①	token 很小（128 個數）但數量極多（ L^2 個，動輒數百萬）	要能同時處理很多 token，而且權重要一直重複使用	很多組 RMPU（每組同時算 20 個 token）；weight-stationary 資料流
②	同一份工作裡混著多種精度（INT4、INT8、INT16 outlier）	為每種精度各做一套乘法器會很浪費：用不到的那套就閒置	RMPU 的位元拆解：用 4-bit 小乘法器拼出任何精度
③	activation 要在執行時量化（算尺、挑 top-k）	需要排序、找最大值、重新排列資料的硬體	VVPU（SIMD + bitonic 排序）、Token Aligner
④	向量運算很多（LayerNorm、Softmax、residual，每個 token 都要做）	矩陣單元做這些很沒效率	VVPU 專門做向量運算，和 RMPU 以管線串接

再加上一個全域的限制：記憶體容量與頻寬（第 1、2 章），所以還需要 scratchpad、double buffering，以及不存中間值的資料流（6.6 節）。

6.2 整體架構與資料流

方塊圖（依論文 Fig. 8 簡化）



先釐清：方塊圖是「有哪些單元」，不是「運算的順序」

這顆晶片的硬體是分時共用的，比較像一台依指令執行的機器，而不是把某個運算焊死成固定管線：

- RMPU 負責所有矩陣乘法類的運算：linear、einsum、Q×K。
- VVPU 負責所有逐元素、向量類的運算：LayerNorm、sigmoid、gating、residual、softmax、執行時量化。
- GCN 讓資料在兩者之間任意繞送，順序可以是 VVPU → RMPU → VVPU；論文也寫到「RMPU 或 VVPU 從 scratchpad 取 token」（Sec. 5）。
- Controller 決定每一趟要做哪些運算。

一個模型的運算要分好幾趟跑完；每一趟都是「從記憶體讀 → 在晶片內串接幾個運算 → 寫回記憶體」。

一趟計算怎麼流過這顆晶片

1. Token Aligner 從 HBM 讀進一批（block）壓縮過的 token，拆解、排整齊後放進 Token Scratchpad。Token Scratchpad 有兩份（128 KB × 2），一份給運算單元讀，另一份同時從記憶體裝下一批，也就是 double buffering（3.3 節），藏住記憶體延遲。
2. 這一趟要用的權重事先載入 Weight Scratchpad（64 KB），整趟都不動（weight-stationary）。
3. 需要時，VVPU 先做前處理（例如 linear 之前的 LayerNorm）。
4. RMPU 做矩陣乘法，一次最多 20 個 token。
5. 結果直接以管線送給 VVPU（不回記憶體），做後續的向量運算（例如 sigmoid、gating、residual），最後做執行時量化。

6. 量化好的 token 經 Output Scratchpad 寫回 HBM。

「RMPU → VVPU」這種「矩陣運算後面接向量運算」的組合在模型裡最常見（例如 linear 後面接 sigmoid 和 gating），所以論文特別強調這條管線。

Triangle Multiplication 怎麼對應到這顆晶片（推測）

論文沒有寫出每個運算的排程；下面是依照運算的相依關係（1.5 節）和論文描述的硬體能力推出來的：

```
第 1 趟（逐 token，對每一格 (i, k)）
  讀 z → VVPU：LayerNorm
    → RMPU：4 個 linear (a_p、a_g、b_p、b_g)
    → VVPU：sigmoid、gating、mask、執行時量化 (C 組 → INT4)
    → 寫回 HBM：a、b          ← 必須寫出去 (einsum 要跨 token 讀)

第 2 趟 (einsum, 跨 token)
  讀 a、b (INT4) → RMPU： $x[i][j] = \sum_k a[i][k] \cdot b[j][k]$ 
    → VVPU：反量化、累加

第 3 趟（逐 token，對每一格 (i, j)，可以接在第 2 趟後面做成管線）
  → VVPU：LayerNorm_out
  → RMPU：linear_z、linear_g
  → VVPU：sigmoid、× g、residual 相加、執行時量化 (A 組 → INT8 + outlier)
  → 寫回 HBM：新的 z
```

重點：einsum 是矩陣乘法，在 RMPU 上做；它一定在 LayerNorm、gating 之後；而且中間一定要經過一次 HBM。這就是 1.5 節說的「融合的邊界」，在 LightNobel 的硬體上也一樣存在。

為什麼是 weight-stationary？

「stationary」是指哪一種資料留在運算單元旁邊不動。常見的選擇有三種：

資料流	固定不動的	適合的情況
Weight-stationary	權重	權重小、被很多輸入重複使用
Output-stationary	部分和（累加值）	輸出要累加很多次
Input-stationary	輸入	輸入被很多權重重複使用

PPM 的 linear 權重只有 128×128 個 INT16 = 32 KB，卻要被數百萬個 token 重複使用。把權重固定在晶片內，每個 token 只需要搬一次，權重的搬運成本幾乎是 0。這和 10.3 節「權重小、activation 大」是同一個觀察。

管線（pipeline）與延遲

論文的延遲模型：總延遲由每個管線階段中最慢的那一段決定（RMPU 運算、VVPU 運算、記憶體讀寫）。這和 2.4 節的 $\max(T_{\text{compute}}, T_{\text{memory}})$ 是同一個概念，只是階段更多。

6.3 Token Aligner：記憶體格式和運算的格式不一樣

問題：兩邊想要的排列方式互相衝突

- 記憶體端希望資料大塊、連續，才能用滿 burst (3.2 節)。所以論文把多個 token 打包成一個 block，block 大小依記憶體通道的頻寬決定 (Sec. 4.3)。每個 token 裡的排列是：inlier $\rightarrow$ outlier $\rightarrow$ 尺 $\rightarrow$ outlier 的位置 (Fig. 7)。由於不同組的精度不同，每個 token 壓縮後的長度也不同。
- 運算端希望一行 scratchpad 剛好是一個 token，這樣 RMPU、VVPU 一次讀一行就拿到一個完整的 token。

解法：解碼 + 重新排列

Token Aligner 在資料進 scratchpad 之前，把「打包好的 block」解碼並重新排成一個 token 一行。在硬體上，它的角色類似寬度轉換器 (width converter / gearbox)：輸入是固定寬度的記憶體 word，輸出是長度不一的 token，中間要有 buffer 和移位邏輯，把跨越 word 邊界的 token 拼回來。

學到的設計原則

「儲存格式」和「運算格式」可以不同，中間用一個專門的轉換單元銜接。這樣兩邊都能用最適合自己的格式。代價是轉換單元本身的面積與延遲；和 RMPU、crossbar 比起來，它只是很小的一塊。

6.4 RMPU：用 4-bit 小乘法器拼出任何精度

RMPU (Reconfigurable Matrix Processing Unit) 是做矩陣乘法的單元，要解決 6.1 節的問題 ②：同一份工作裡混著 INT4、INT8、INT16。

6.4.1 為什麼不各做一套乘法器？

假設晶片上放了 INT4、INT8、INT16 三種乘法器：

- 算 INT4 的 token 時，INT8、INT16 的乘法器全部閒置；
- 而且三種的比例要事先固定，但 AAQ 的精度組合在執行時才知道 (依 activation 所屬的組、outlier 的數量)。

結果就是嚴重的利用率不足 (underutilization)。RMPU 的做法是：只放一種最小的乘法器 (處理 4-bit 資料塊)，大的乘法用很多個小的拼起來。(用語說明：本章說的「4-bit 小乘法器」是簡稱，指「處理 4-bit 資料塊的乘法器」；硬體上它的輸入是 5 bit，原因見 6.4.3 節。) 這個想法來自 Bit Fusion (ISCA 2018，論文參考文獻 [57])。

6.4.2 原理：位元拆解乘法

任何整數都可以拆成 4-bit 的小塊 (chunk)：

$$\begin{aligned} A &= a_0 + a_1 \cdot 2^4 + a_2 \cdot 2^8 + a_3 \cdot 2^{12} && \text{(16-bit 的數拆成 4 塊)} \\ A \times B &= \sum_i \sum_j a_i \cdot b_j \cdot 2^{(4(i+j))} && \text{(每一對小塊相乘，再移位相加)} \end{aligned}$$

每個 $a_i \cdot b_j$ 是一對 4-bit 資料塊的小乘法； $\times 2^{(4(i+j))}$ 只是左移，不需要乘法器。

數字例子：INT4 的 activation × INT16 的權重（linear 裡最常見的情況：論文的權重是 INT16）

```
a = -5 (INT4, 只有 1 塊)
w = 1234 = 0x04D2 → 從低到高 4 塊：[2, 13, 4, 0]

部分積      值      左移      結果
-5 × 2 = -10    << 0 = -10
-5 × 13 = -65   << 4 = -1,040
-5 × 4 = -20    << 8 = -5,120
-5 × 0 = 0      << 12 = 0
          合計 = -6,170 ← 正好等於 -5 × 1234
```

所以：

乘法	需要幾個小乘法器 (5-bit 輸入)
INT4 × INT4	1
INT4 × INT16	1 × 4 = 4
INT8 × INT16	2 × 4 = 8
INT16 × INT16	4 × 4 = 16

同一組硬體，精度低就同時算很多個，精度高就合起來算一個，所有乘法器都不會閒著。

6.4.3 有正負號的數怎麼拆？（sign extension）

二補數（two's complement）的整數，只有最高位的那一塊帶正負號，其他塊都是無號數。例如 INT8 的 -93：

```
-93 = 0xA3 → 高 4 位 0xA、低 4 位 0x3
高位塊：0xA 當作有號數 = -6
低位塊：0x3 當作無號數 = 3
驗算：-6 × 16 + 3 = -93 ✓
```

所以 RMPU 的資料對齊器（RDA）在拆塊時：

- 最高位的塊：用它自己的最高位元做符號延伸（sign extension）；
- 其他塊：前面補 0（zero extension）。

有號塊的範圍是 -8~7，無號塊是 0~15，要同時裝得下兩者，每塊要 5 bit（-16~15）。這就是論文 Fig. 9(a) 裡乘法器輸入標成「5bit」的原因。

所以「5-bit」是乘法器的輸入寬度，「4-bit」是資料塊的寬度：

```
4-bit 資料塊 → RDA 擴充成 5-bit 有號數 → 5×5 有號乘法器 → 部分積 → 移位 → 加法樹
例：-93 = 0xA3
  高位塊 1010 → 符號延伸 → 11010 = -6
  低位塊 0011 → 補 0      → 00011 = 3
```

- 為什麼不用 4×4 就好？4-bit 有號只到 7、4-bit 無號沒有負數，都無法同時表示 -8~15。擴充成 5-bit 有號後，同一種乘法器就能處理所有塊，不用區分有號、無號；代價是乘法器從 4×4 變成 5×5，面積大一

些。

- 輸出有幾 bit？兩個 5-bit 有號數相乘最多 10 bit；以實際會出現的值 ($-8 \times 15 = -120$ 到 $15 \times 15 = 225$) 也至少要 9 bit。之後由移位器移 $4(i+j)$ 位，再由加法樹加總。

6.4.4 階層式的結構：PE → PE Lane → PE Cluster → RMPU Engine

PE (Processing Element)

16 個 5-bit 乘法器 + 移位器 + 16-to-1 加法樹

→ 可以算 1 個 $\text{INT}_{16} \times \text{INT}_{16}$ ，或 16 個 $\text{INT}_4 \times \text{INT}_4$ ，或 4 個 $\text{INT}_4 \times \text{INT}_{16}$...

PE Lane = 8 個 PE (共 128 個小乘法器)

兩種模式：(1) 每 2 個 PE 加上 bias → 4 個輸出 (給 head 維度 32 的 attention)

(2) 8 個 PE 全部用加法樹加起來 → 1 個輸出 (內積)

PE Cluster = 20 個 PE Lane + DAL (動態累加邏輯)

RMPU Engine = 4 個 PE Cluster

= 80 個 PE Lane = 640 個 PE = 10,240 個小乘法器 (5-bit 輸入)

→ 一次最多同時處理 20 個 token

6.4.5 為什麼一個 Cluster 是 20 個 Lane？(最小公倍數)

算一個 128 維的內積 (一個 token × 權重的一行)，需要多少個 4-bit 乘法器？

token 的格式	計算	需要的小乘法器	需要的 PE Lane (每個 128 個)
128 個 INT_4 inlier (C 組)	$128 \times 4 = 512$	512	4
124 個 INT_4 inlier + 4 個 INT_{16} outlier (B 組)	$124 \times 4 + 4 \times 16 = 560$	560	$4.375 \rightarrow 5$

PPM 裡大部分的內積需要 4 或 5 個 Lane。一個 Cluster 放 20 個 Lane (4 和 5 的最小公倍數)：

- 需要 4 個 Lane 時： $20 \div 4 = 5$ 個 token 同時算，Lane 全部用滿；
- 需要 5 個 Lane 時： $20 \div 5 = 4$ 個 token 同時算，也全部用滿。

這就是「依工作負載決定硬體數量」的典型例子：先統計工作負載需要的單位 (4 或 5)，再選一個能被兩者整除的數量，讓利用率在兩種情況都是 100%。

6.4.6 DAL：尺要在什麼時候乘？

一個 B 組的 token 有兩種值：

- inlier：量化過，要乘尺才會回到原本的大小；
- outlier：以 INT_{16} 原值存，不用乘尺。

所以不能把 5 個 Lane 的結果直接加起來。DAL (Dynamic Accumulation Logic) 的做法：

需要 4 個 Lane (全是 inlier)：4 個 Lane 的結果相加 → 最後乘一次尺

需要 5 個 Lane (有 outlier)：inlier 的部分先相加 → 乘尺
→ 再加上 outlier 那個 Lane 的結果

4-to-1 和 5-to-1 的加法樹結構不同，DAL 用「關掉其中一個加法樹、把第 5 個 Lane 接到後面」的方式動態切換，前面再放一個 Arbiter 把各 Lane 的輸出排到正確的位置。這就是「可重組的加法樹」。

對照我們的 v4：linear 的內積是沿著同一個 token 的 channel 加總，所以尺可以最後乘一次（4.4 節情況一）。我們的 einsum 是沿著 k 跨 token 加總，每一項的尺都不同（4.4 節情況二），所以 v4 每個 k 都要乘尺。

6.4.7 RMPU 的輸出有很多種「粒度」

加法樹的每一層都有意義，RMPU Engine 把不同層的結果都拉出來用：

取出的位置	意義	用途
2 個 PE 的和	head 維度 32 的內積	multi-head attention
4 或 5 個 Lane 的和	一個量化 token 的內積	量化的 linear（經過 DAL）
8 或 16 個 Lane 的和	沒量化的 token 的內積	高精度的運算
80 個 PE 的和	更大的內積	多個 RMPU 合作

最後一層還有 ReLU。同一組乘法器，靠「從加法樹的哪一層取結果」支援不同的運算。

6.5 VVPU：向量運算、排序與執行時量化

6.5.1 結構

VVPU

- └ 128 條 SIMD Lane：每條有一個 16-bit ALU、自己的小 scratchpad、兩級的指數查表（exp LUT）
- └ SSU（Scalar Support Unit）：處理純量運算，例如求平均、量化資料的格式整理
- └ LCN（Local Crossbar Network）：讓 128 條 Lane 之間可以任意交換資料

- 為什麼是 128 條？一個 token 剛好 128 個數，一條 Lane 負責一個 channel，一個 cycle 就能對整個 token 做同一個運算（SIMD：single instruction, multiple data）。
- 為什麼要指數查表？Softmax 和 sigmoid 都要算 $\exp(x)$ 。用查表加少量運算代替完整的指數電路，面積小很多（參考文獻 [26] 的做法）。
- 為什麼要 SSU？LayerNorm 的平均值、變異數要把 128 個數加起來變成一個數，這種「歸約（reduction）」和格式整理交給純量單元做，SIMD Lane 就能專心做逐元素的運算。
- 為什麼要 LCN？量化後的資料要依記憶體格式重新排列（inlier、outlier 分開放），需要資料在 Lane 之間移動。

6.5.2 用硬體挑 top-k：bitonic 排序網路

執行時量化要找出每個 token 裡絕對值最大的 k 個（A、B 組 $k = 4$ ）。VVPU 用 bitonic sort（雙調排序）：

原理：排序網路（sorting network）是一組固定連線的比較交換器（compare-and-swap）：每個比較器接兩個數，大的往一邊、小的往另一邊。連線是固定的，不管資料是什麼，步驟都一樣。這點對硬體很重要：

- 沒有「if 分支」，每一步所有比較器同時動作，很適合 SIMD Lane 平行做；
- 延遲固定，容易排進管線。

以 8 個數為例，bitonic sort 分 3 個大階段（2、4、8 個一組），共 6 步，每步 4 個比較器同時動作：

階段 1：兩兩比較，做出長度 2 的有序片段
階段 2：合併成長度 4 的有序片段（2 步）
階段 3：合併成長度 8 的有序序列（3 步）
總步數 = $\log_2 8 \times (\log_2 8 + 1) \div 2 = 3 \times 4 \div 2 = 6$

128 個數時： $7 \times 8 \div 2 = 28$ 步，每步 64 個比較器同時動作。只要 top-k，不需要完整排序，還可以提早停止（參考文獻 [56]）。

排序時 VVPU 一路追蹤每個值原本的位置（index），最後就知道 outlier 在哪裡（要存進記憶體的「outlier 位置」）。k = 1 時就是找最大值，所以同一套硬體也用在 softmax（要先找最大值）和算尺（最大絕對值）。

6.5.3 執行時量化的四個步驟

1. top-k：SIMD Lane 找出 outlier 和尺（最大絕對值）
2. 縮放：每個值除以尺、四捨五入成 INT4 / INT8
3. 重排：LCN 依記憶體格式重新排列（inlier、outlier、尺、位置）
4. 對齊：SSU 整理成最終的記憶體排列

這些都在 VVPU 內完成，完整精度的值從來不寫回記憶體（4.3 節「算尺的時候…」、10.4 節「量化在源頭完成」）。

6.5.4 為什麼一個 RMPU 配 4 個 VVPU？（硬體設計空間探索）

論文 Fig. 12(a) 改變「每個 RMPU 配幾個 VVPU」，看延遲怎麼變：

- VVPU 太少：RMPU 算得很快，但 VVPU 做 LayerNorm、量化來不及，整條管線被 VVPU 拖慢；
- 到 4 個時延遲就不再下降（飽和）：VVPU 的工作已經能完全藏在 RMPU 的運算底下，再加也沒用。

這和 2.6 節的 Amdahl's Law、3.3 節的 double buffering 是同一個道理：管線的速度由最慢的那一段決定，把最慢的那段加強到不是瓶頸就夠了，再多只是浪費面積。

6.6 Token-wise multi-head attention：不存 L^3 的 score matrix

問題

Triangle Attention 的 score matrix 是 $L \times L \times L$ （每個 head），不分塊時一定放不下（5.4 節）。

原理：線上 softmax（online softmax）

論文說他們的做法「類似 FlashAttention，並針對 token-wise 運算最佳化」（參考文獻 [16]），但沒有寫出完整的演算法。以下說明 FlashAttention 的核心原理，也就是 LightNobel 能做到「不存 score matrix」的原因。

Softmax 需要先知道所有分數的最大值（數值穩定）和總和：

$$\text{softmax}(s_i) = \exp(s_i - m) \div \sum_j \exp(s_j - m) \quad m = \max(s)$$

看起來必須先算完全部的分數才能開始。線上 softmax 的技巧是邊算邊修正：

維護三個值：目前的最大值 m 、目前的總和 ℓ 、目前的輸出累加 o

每來一批新的分數 s （和對應的 V ）：

$$m_{\text{new}} = \max(m, \max(s))$$

$$\ell = \ell \times \exp(m - m_{\text{new}}) + \sum \exp(s - m_{\text{new}}) \quad \leftarrow \text{舊的總和用新的最大值重新縮放}$$

$$o = o \times \exp(m - m_{\text{new}}) + \sum \exp(s - m_{\text{new}}) \times V$$

$$m = m_{\text{new}}$$

全部算完：輸出 = $o \div \ell$

每一批分數算完就丟，只留下 m 、 ℓ 、 o 三個小東西。 L^3 的 score matrix 從來不需要整份存在記憶體。

在 LightNobel 上怎麼跑 (Sec. 5.4)

1. RMPU 對每個 head 平行算 $Q \times K$ ；
2. 同時，VVPU 做反量化和中間結果的累加；
3. VVPU 做 softmax，和下一批 $Q \times K$ 以管線重疊，減少找最大值的延遲；
4. 最後載入 V ，乘上 softmax 的結果，寫回記憶體。

為什麼在 PPM 上特別划算

一般的語言模型 hidden 維度很大 (LLaMA 7B 是 4,096)，一次放不下很多 token，token-wise 做法的資料重用有限。PPM 的 hidden 維度只有 128，一個 token 很小，晶片內可以同時放很多個，token-wise 的做法就有效率（論文 Sec. 5.4 的論點）。

6.7 互連與記憶體：為什麼 crossbar 佔了 70% 的面積

Crossbar 是什麼

Crossbar 是「任何輸入都能接到任何輸出」的交換網路。 N 個輸入、 N 個輸出、每條 W bit 寬：

$$\text{開關 (crosspoint) 的數量} \propto N \times N \times W$$

面積與功耗隨埠數的平方成長。例如 128 條 Lane、16 bit 的全互連： $128 \times 128 \times 16 \approx 26$ 萬個開關。

LightNobel 用 swizzle switch（參考文獻 [55]）這種面積、功耗較省的 crossbar 實作，但仍然是最大的一塊。

面積與功耗的分配 (Table 2, 28 nm、1 GHz)

論文的 Table 2 在 PDF 中排版錯亂，下表是我們把各列的數字加總對照後推回來的，總數與論文文字（crossbar 70.28% 面積、67.95% 功耗；RMPU Engine 18.20% 面積、22.36% 功耗）一致：

模組	面積 (mm ²)	功耗 (mW)
1 個 RMPU Engine	1.017	473.9
1 個 RDA (資料對齊器)	0.105	112.4
1 個 RMPU 合計	1.127	589.1
32 個 RMPU	36.07	18,853
1 個 VVPU 的 128 條 SIMD Lane	0.115	21.1
1 個 VVPU 的 LCN (crossbar)	0.785	288.0
1 個 VVPU 合計	0.902	309.9
128 個 VVPU	115.43	39,668
GCN (全域 crossbar)	25.13	9,216
Scratchpad (Token 128 KB × 2、Weight 64 KB、Output 128 KB)	約 2.0	—
整顆晶片	178.80	67,805 (67.8 W)

這張表教我們什麼

- 一個 VVPU 裡，crossbar (0.785) 是 128 條運算 Lane (0.115) 的將近 7 倍大。搬資料的硬體比算資料的硬體貴得多。
- 全晶片的 crossbar (128 個 LCN + GCN) 約 125.7 mm²，佔 70%；真正做乘法的 RMPU Engine 只佔 18%。
- 這對應第 2 章的主題：在這類問題裡，瓶頸是資料移動，不是運算。連晶片內部都是如此。
- 對設計者的啟示：想省面積，先檢討是否真的需要全互連。如果資料的移動模式是固定的（例如只需要相鄰交換），用簡單的網路就能大幅縮小面積。

Scratchpad 為什麼叫 scratchpad，不叫 cache？

兩者都是晶片內的小記憶體，差別在誰決定放什麼：

	Cache	Scratchpad
放什麼由誰決定	硬體自動（依位址、替換策略）	設計者／程式明確指定
命中率	不確定	確定（事先規劃好）
需要的額外硬體	tag、比較器、替換邏輯	幾乎沒有

加速器的資料流是事先規劃好的（哪個 token 什麼時候用），不需要 cache 的猜測機制，用 scratchpad 更省面積也更可預測。我們 FPGA kernel 裡的 BRAM 陣列（acc、la、lb）也是 scratchpad。

6.8 決定要放幾個：硬體設計空間探索

論文 Fig. 12(b) 固定每個 RMPU 配 4 個 VVPU，改變 RMPU 的數量：到 32 個時效能就飽和了。

原因 (Sec. 7.2)：32 個 RMPU 已經有足夠的運算能力，處理從記憶體讀進來的所有資料；再加更多 RMPU，資料供應跟不上，只會閒置。

這就是 roofline 的平衡點 (2.5 節)：

運算能力 < 記憶體能供應的量 → 加運算單元有用 (compute-bound)
運算能力 ≥ 記憶體能供應的量 → 加運算單元沒用 (memory-bound)
32 個 RMPU ≈ 剛好在平衡點上

好的硬體設計會把運算資源放到「剛好到平衡點」，再多就是浪費面積和功耗。我們的 9.3 節 DSE (tile 大小 vs BRAM) 是同一種思考：找「膝點」。

6.9 硬體怎麼評估：每個工具在做什麼

論文沒有做出實體晶片，而是用以下工具組合評估 (Sec. 6)。這也是學術界 ASIC 加速器論文的標準流程：

工具	做什麼	產出	對應我們的專題
SystemVerilog RTL	用硬體描述語言寫出每個模組的電路	可模擬、可合成的設計	HLS 從 C++ 自動產生 RTL
Synopsys Design Compiler (28 nm、1 GHz)	邏輯合成：把 RTL 對應到製程的標準元件 (standard cell)	面積、功耗、能不能跑到 1 GHz	Vivado 合成、HLS 的資源與時序報告
記憶體編譯器、CACTI 7.0	估計 SRAM (scratchpad) 的面積與功耗	記憶體的面積、功耗	BRAM/URAM 的用量
Ramulator	cycle 級的 DRAM/HBM 模擬	真實的記憶體延遲與頻寬	(我們沒有；HLS 假設固定延遲)
Python cycle-accurate 模擬器	依每個模組的延遲，模擬整個 PPM 的執行	總延遲	我們的分析模型 (perf_model.py)
RTL 交叉驗證	比較模擬器和 RTL 模擬的結果	誤差平均 3.30%、都在 5% 內	分析模型 vs HLS 報告

CACTI 只支援到 32 nm，論文用常見的縮放係數換算到 28 nm。這種「工具不完全符合時，用文獻中的方法換算」在架構論文很常見，報告時要說明。

6.10 如果要在 FPGA 上實作這些設計 (我們的分析)

這一節是我們自己的分析，不是論文的內容。目的是把 ASIC 的設計對應到 U55C 上的資源，思考哪些能搬、代價是什麼。

LightNobel 的元件	在 FPGA 上的對應	需要注意的地方
4-bit 小乘法器 (RMPU)	用 LUT 做 (小的乘法器用 LUT 比用 DSP 划算)	U55C 有約 130 萬個 LUT，但每個小乘法器要數個到十幾個 LUT，數量有限
INT8 × INT8 乘法	DSP48E2 (27 × 18 的乘法器)	已知的技巧：共用一個運算元時，一個 DSP 可以同時算兩個 INT8 乘法 (AMD 白皮書 WP486)
可重組的加法樹 (DAL)	LUT + 多工器	可重組的彈性要用多工器換，會吃 LUT 和繞線
SIMD Lane	展開的迴圈 (#pragma HLS UNROLL) + 切開的陣列 (ARRAY_PARTITION)	和我們的 v2 16 路、v4 64 路是同一個手法
Crossbar	大量多工器 + 繞線	在 FPGA 上一樣昂貴，而且繞線擁擠會讓時脈跑不高；應盡量用固定的資料移動模式
Scratchpad	BRAM/URAM	U55C 約 43 MB，比 LightNobel 在 Table 2 列出的共用 scratchpad (約 0.45 MB) 大得多，這是 FPGA 的優勢
double buffering	兩組 BRAM + DATAFLOW	就是我們的 v3
exp 查表	BRAM 或 LUT 裡的查表	HLS 的 hls::exp 也可以，但比較耗資源
bitonic 排序	展開的比較交換網路	128 個數要 28 步、每步 64 個比較器，LUT 用量可觀

我們的 v0 ~ v5 對應到其中的哪些部分：

- v1 ~ v3：scratchpad、weight/input 的重用、double buffering (6.2 節的資料流基礎)；
- v4：INT8 乘法 + 浮點尺 (RMPU + DAL 的簡化版，但我們的 einsum 每個 k 都要乘尺，6.4.6 節)；
- v5 (規劃中)：INT4 的解包與乘法，接近 RMPU 的「拆成 4-bit」想法，但只支援單一精度，不需要可重組。

本章小結

單元	解決的問題	核心技巧	設計數量的理由
Token Aligner	記憶體格式 ≠ 運算格式	解碼 + 重新排列 (寬度轉換)	—
RMPU	多種精度混在一起	位元拆解乘法、可重組加法樹、DAL	一個 Cluster 20 個 Lane (4 和 5 的最小公倍數)；32 個 RMPU (到平衡點就飽和)
VVPU	大量向量運算、執行時量化	128 條 SIMD Lane、exp 查表、bitonic 排序	每個 RMPU 配 4 個 (再多就飽和)
Token-wise attention	L ³ 的 score matrix	線上 softmax	—

單元	解決的問題	核心技巧	設計數量的理由
Crossbar、scratchpad	資料要在單元之間移動	swizzle switch、double buffering、weight-stationary	crossbar 佔 70% 面積：資料移動才是最貴的

三個可以帶走的設計原則：

1. 先分析工作負載，再決定硬體：每個單元都對應一個工作負載的特性（6.1 節）。
2. 數量由利用率和平衡點決定：20 個 Lane 讓 4、5 兩種情況都用滿；32 個 RMPU、4 個 VVPU 都是「剛好不是瓶頸」的點。
3. 資料移動比運算貴：不只晶片外（HBM），晶片內（crossbar）也是如此。

自我檢測

讀完這一章，試著回答下面的問題（參考答案在附錄 C）：

1. 用 4-bit 乘法器算 $\text{INT8} \times \text{INT16}$ ，需要幾個乘法器？寫出拆解的公式。
2. 把 INT8 的 -93 拆成兩個 4-bit 塊，高位塊和低位塊各當作有號還是無號？為什麼乘法器的輸入是 5 bit？
3. 為什麼一個 PE Cluster 放 20 個 PE Lane？
4. DAL 為什麼要區分「4 個 Lane」和「5 個 Lane」兩種累加方式？
5. bitonic 排序網路為什麼適合硬體？128 個數要幾步？
6. 線上 softmax 為什麼不需要存整個 score matrix？
7. 為什麼 LightNobel 的 crossbar 佔了 70% 的面積？這對設計者有什麼啟示？
8. 論文的 RMPU 數量為什麼停在 32 個？這和 roofline 有什麼關係？

下一章進入我們自己的實作：先介紹 FPGA 與 AMD 工具鏈的原理，看 C++ 是怎麼變成電路的。

第 7 章 FPGA 與工具原理：為何 C++ 可以變成電路？

本章重點

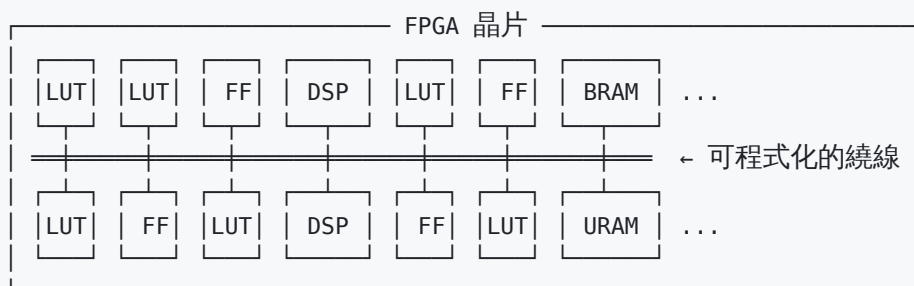
- FPGA 是一塊「可以重新接線的晶片」：裡面有大量可設定的小元件（LUT、FF、DSP、BRAM），靠一份設定檔（bitstream）決定它們怎麼連接。
- 把 C++ 變成電路的是 AMD 的 Vitis HLS，不是 Claude。它把程式拆成運算、排進時脈週期、對應到硬體元件，最後輸出 Verilog。
- C++ 可以先模擬，是因為 C++ 程式本身就是功能規格；HLS 保證產生的電路和程式算出一樣的結果。
- 整條工具鏈：g++（驗證）→ Vitis HLS（C++ → RTL）→ Vivado（RTL → bitstream）→ v++（接上 platform）→ XRT（host 控制板子）。
- 最後一節清楚劃分：哪些是下載／安裝的工具、哪些是 Claude 寫的、Claude 在這個對話中實際執行了什麼。

7.1 FPGA 是什麼？

白話直覺

- CPU 像一位「什麼都會做的通才」：硬體固定，靠執行不同的指令完成不同工作。
- GPU 像「一大群做同樣動作的工人」：硬體固定，擅長大量重複的計算。
- ASIC（例如 LightNobel 設計的加速器）像「為一道菜專門打造的生產線」：最快、最省電，但做好就不能改，而且製造非常貴。
- FPGA 像「一大箱樂高」：可以依需求組成任何生產線，組錯了拆掉重來。速度介於 CPU/GPU 和 ASIC 之間。

FPGA 裡面有什麼



元件	是什麼	白話	U55C 的數量
LUT (Look-Up Table)	一個 6 輸入的小真值表，可以實現任何 6 個輸入的邏輯函數	可以變成任何小邏輯閘組合	約 130 萬
FF (Flip-Flop)	1 bit 的記憶元件，每個時脈更新一次	暫存器，用來「記住上一步的結果」	約 260 萬
DSP (DSP48E2)	一個 27×18 bit 的硬體乘法器 + 累加器	專門做乘法的小計算機	9,024
BRAM	36 Kb 的小記憶體（可拆成兩塊 18 Kb），有 2 個讀寫埠	料理台	2,016 (36 Kb)
URAM	288 Kb 的較大記憶體	大料理台	960
繞線 (routing)	可程式化的連線網路	決定誰接到誰	—

LUT 為什麼能變成「任何邏輯」

LUT 本質上是一塊 64 格的小記憶體 ($2^6 = 64$)。6 個輸入當作「地址」，存在裡面的 0/1 就是輸出。

例：想做「3 個輸入的多數決」（兩個以上是 1 就輸出 1）

輸入 abc : 000 001 010 011 100 101 110 111

輸出 : 0 0 0 1 0 1 1 1 ← 把這 8 個 bit 寫進 LUT 就完成了

改寫 LUT 裡的內容，就改變了電路的功能。這就是「可程式化」的本質。

Bitstream

Bitstream 是一個檔案，裡面記錄了：每個 LUT 要存什麼、每條繞線要不要接通、每個 DSP 和 BRAM 怎麼設定。把 bitstream 寫進 FPGA，它就變成我們設計的電路。在 Vitis 流程中，bitstream 被包在 **.xclbin** 檔案裡。

一個浮點乘法在 FPGA 上長什麼樣子

float x, y → [拆出正負號、指數、尾數 (LUT)]



[尾數相乘 (約 2~3 個 DSP)]



[指數相加、正規化、捨入 (LUT + FF)]



x × y

為了跑到 300 MHz，中間會插入好幾級 FF，所以一個浮點乘法需要好幾個 cycle 才出結果。

v2 的 16 路平行，就是真的放了 16 組這樣的硬體。

7.2 為什麼 FPGA 適合做記憶體優化的實驗？

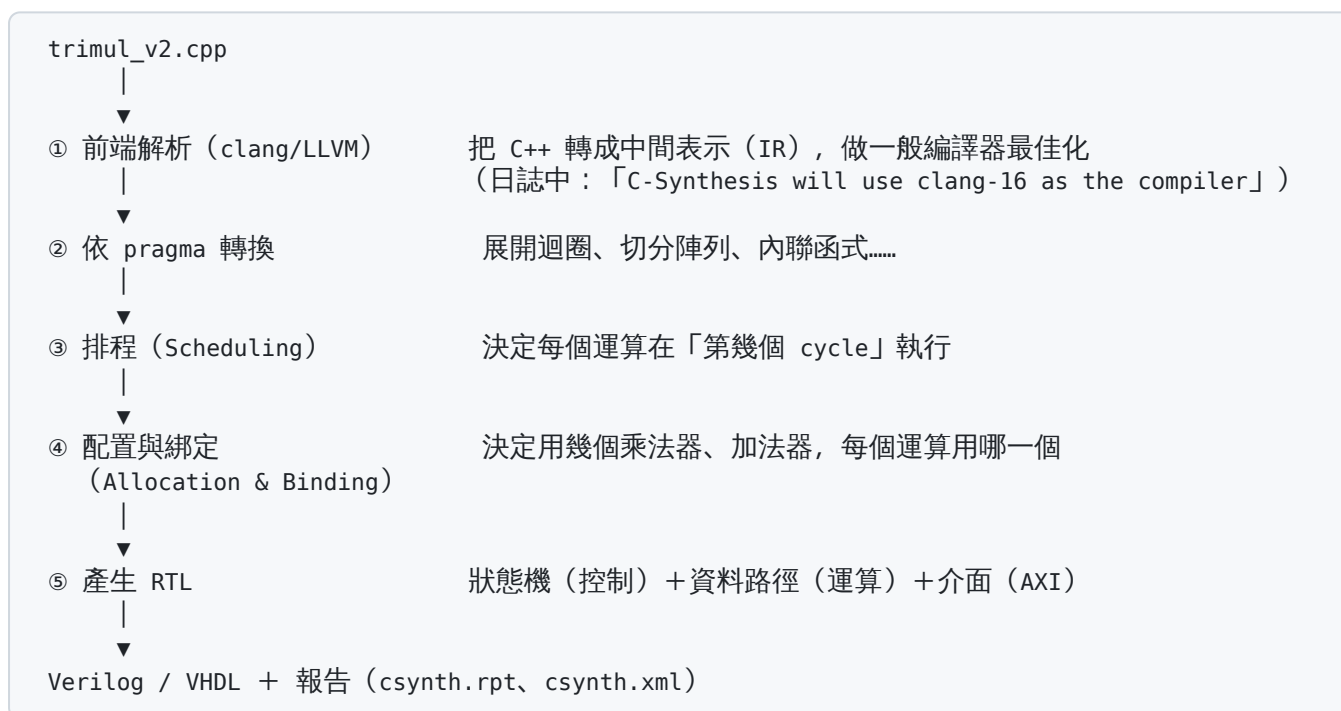
特性	CPU / GPU	FPGA
記憶體階層	固定 (cache 由硬體自動管理)	自己設計 (要多少 buffer、放在哪、怎麼切都可以決定)
資料怎麼搬	由硬體和驅動程式決定	自己決定 (位元寬度、burst 長度、哪個通道)
運算單元數量	固定	自己決定 (16 路、64 路……)
能看到多少細節	很多被隱藏	報告直接列出每個迴圈、每種資源

這讓我們能「一次只改一件事」，清楚量出每一招的效果。

7.3 HLS : C++ 是怎麼變成電路的？

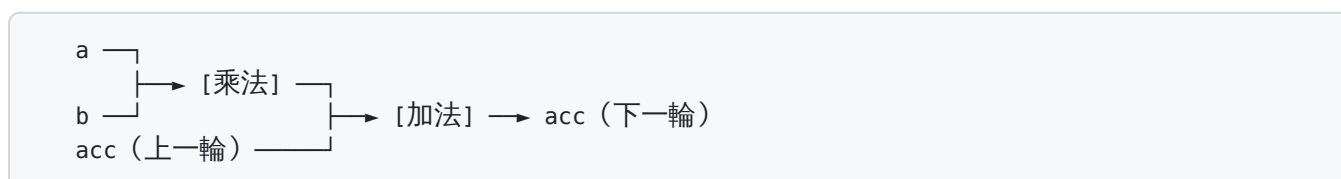
HLS (High-Level Synthesis, 高階合成) : 把 C/C++ 自動翻譯成硬體描述語言 (Verilog/VHDL)。本專題用的是 AMD 的 Vitis HLS (2025.2 以 `vitis-run --mode hls` 執行)。

整體流程



① 前端解析：把程式拆成「運算的圖」

例：`acc += a * b;`



這張圖叫 資料流圖 (data flow graph)：節點是運算，箭頭是資料相依。

③ 排程：把運算放進時脈週期

每種運算在 300 MHz 下需要的 cycle 數不同 (例如浮點乘法、浮點加法都要好幾個 cycle)。HLS 依照資料相依關係，把每個運算排到某個 cycle：

cycle:	1	2	3	4	5	6	7	8	9	10
讀 a、b	█									
乘法		█	█	█	█					
加法					█	█	█	█		
寫 acc									█	

一次迭代從開始到結束需要的 cycle 數叫 iteration latency。

Pipeline 與 II (Initiation Interval)

沒有 pipeline：一次迭代做完才開始下一次。

迭代 0：	[讀][乘乘乘][加加加加][寫]
迭代 1：	[讀][乘乘乘][加加加加][寫]

有 pipeline (II = 1)：每個 cycle 都開始一個新的迭代，像工廠的流水線。

迭代 0：	[讀][乘乘乘][加加加加][寫]
迭代 1：	[讀][乘乘乘][加加加加][寫]
迭代 2：	[讀][乘乘乘][加加加加][寫]
迭代 3：	[讀][乘乘乘][加加加加][寫]

II = 連續兩次迭代開始的間隔。II = 1 代表每個 cycle 都有一個結果產出，是最理想的狀態。

例：v2 的 mac 迴圈 跑 512 次，報告顯示 526 cycles：

$526 \approx 512$ (每 cycle 開始一次, II = 1) + 14 (pipeline 的深度, 最後一次迭代要走完)

為什麼 v0 無法達到 II = 1

v0 的 `acc += a * b`：下一次的加法需要這一次加完的結果。

迭代 0：	[加加加加]
迭代 1：	[加加加加] ← 要等迭代 0 加完才能開始

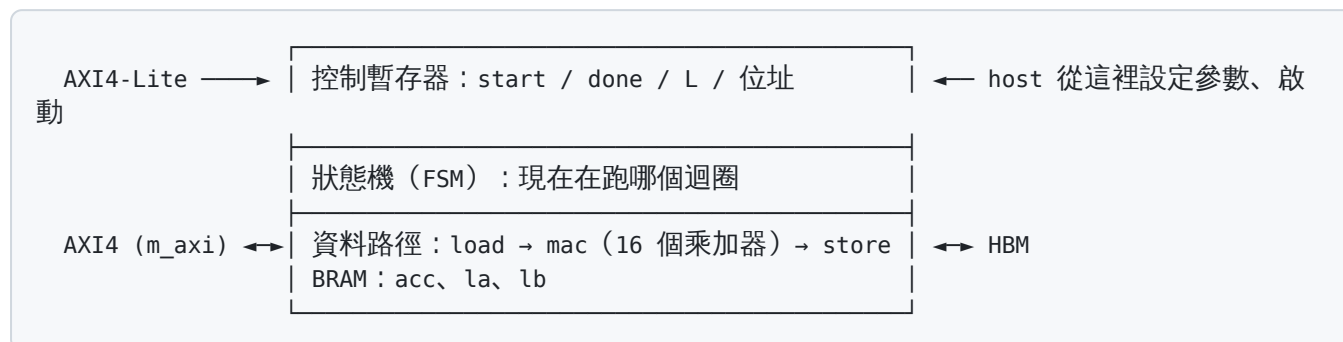
假設浮點加法需要 4 個 cycle，II 至少是 4。這叫 loop-carried dependency (跨迭代的相依)。v1~v4 把迴圈順序換掉，讓相鄰的迭代更新不同的 acc 位置，就沒有這個問題 (再用 DEPENDENCE pragma 告訴 HLS)。

④ 配置與綁定：用幾個硬體？

- 一個 pipeline 迴圈裡如果每個 cycle 都要做 16 次乘法，就需要 16 個乘法器。
- 如果迴圈沒有 pipeline，HLS 可能讓多個運算共用同一個乘法器 (省資源，但變慢)。

- 記憶體的限制：一塊 BRAM 只有 2 個讀寫埠，一個 cycle 最多讀寫 2 次。v2 每個 cycle 要讀 16 個 la，所以必須用 ARRAY_PARTITION 把陣列切成 16 塊，每塊各自讀。

⑤ 產生 RTL：電路長什麼樣子



- #pragma HLS INTERFACE m_axi → 產生 AXI4 master 介面，kernel 可以主動去 HBM 讀寫。
- 在 Vitis 流程中，純量參數（例如 L）和指標的位址會自動經由 AXI4-Lite 控制暫存器傳入。

本專題用到的 pragma 一覽

Pragma	作用	在哪裡用	不加會怎樣
PIPELINE II=1	迴圈做成流水線，每 cycle 開始一次迭代	幾乎所有內層迴圈	迭代一個接一個做，慢很多倍
ARRAY_PARTITION dim=2 complete	把陣列的某一維拆成獨立的記憶體	v2 ~ v4 的 acc、la、lb	一個 cycle 只能讀 2 個數，16 路平行做不到
DEPENDENCE variable=acc inter false	告訴 HLS 跨迭代沒有相依	v1 ~ v4 的 mac	HLS 保守地拉高 II
INTERFACE m_axi port=a bundle=gmem0	a 走獨立的 AXI 連接埠	所有 kernel	多個指標可能共用同一個埠，互相搶頻寬
INLINE off	函式保留成獨立模組	v3 的 load_ab、mac	被併進主函式，可能無法平行執行
LOOP_TRIPCOUNT	告訴 HLS 迴圈大概跑幾次	以 L 為上限的迴圈	報告中的 latency 顯示為「？」

可合成的 C++ 有什麼限制

HLS 只能處理「能對應到固定硬體」的 C++：

不能用	原因
new / malloc (動態記憶體)	硬體的記憶體大小必須在合成時決定
遞迴	無法事先知道要幾層硬體
系統呼叫 (檔案、printf 等)	硬體沒有作業系統
大小不固定的陣列	同動態記憶體

所以 kernel 裡的陣列大小都寫成常數（TI、TJ、C），而讀檔、印結果都放在 testbench 或 host。

7.4 為什麼 C++ 可以直接模擬？

核心觀念：C++ 程式就是「功能規格」

HLS 的承諾是：產生的電路，對同樣的輸入，會算出和 C++ 程式一樣的結果（在可合成的範圍內）。所以：

1. 用一般編譯器（g++）執行 C++ → 得到「應該要算出來的答案」。
2. 如果這個答案是對的，那 HLS 產生的電路在功能上也是對的。

#pragma HLS 只影響硬體怎麼實作（多快、用多少資源），不影響算出什麼。g++ 不認識這些 pragma，直接忽略，所以可以照常編譯執行。

四種模擬，一層比一層接近真實

模擬	在跑什麼	驗證什麼	速度	本專題
C 模擬 (csim)	C++ 程式本身 (g++ 或 HLS 內建的 clang)	演算法對不對	秒	✅ make csim
C/RTL 協同模擬 (cosim)	HLS 產生的 Verilog，用 RTL 模擬器 (Vivado 的 xsim) 執行，testbench 仍是 C++	產生的電路和 C++ 結果一致嗎？時序正確嗎？	分鐘 ~小時	可選 (COSIM=1)
軟體模擬 (sw_emu)	kernel 編成一般程式，搭配 XRT 模擬	host 和 kernel 的介面對不對	快	需要 XRT
硬體模擬 (hw_emu)	kernel 的 RTL + platform 的模型 + XRT	整個系統（含記憶體介面）的行為	慢	需要 XRT

為什麼 C 模擬通過不代表一定沒問題

- C++ 沒有「時間」的概念，無法抓到時序相關的錯誤（例如 DEPENDENCE 宣告錯誤導致讀到舊值）。
- 所以正式流程中，cosim 或 hw_emu 是重要的第二道防線。

7.5 Vivado：從 RTL 到 bitstream

HLS 產生的是 Verilog（描述電路的程式碼），還不是 FPGA 能直接用的檔案。接下來由 Vivado 完成（在 Vitis 流程中由 v++ -l 自動呼叫）：

步驟	做什麼	比喻
Synthesis（邏輯合成）	把 Verilog 轉成 LUT、FF、DSP、BRAM 組成的網表 (netlist)	把設計圖拆成樂高零件清單
Optimization	刪除多餘的邏輯、合併重複的部分	省掉不需要的零件
Placement（擺放）	決定每個零件放在晶片的哪個位置	決定每塊樂高放哪

步驟	做什麼	比喻
Routing (繞線)	決定零件之間的連線走哪條路	接電線
Timing analysis	檢查最慢的路徑能不能在一個 cycle 內跑完	確認流水線每一站都來得及
Bitstream generation	把以上結果寫成設定檔	產出組裝說明書

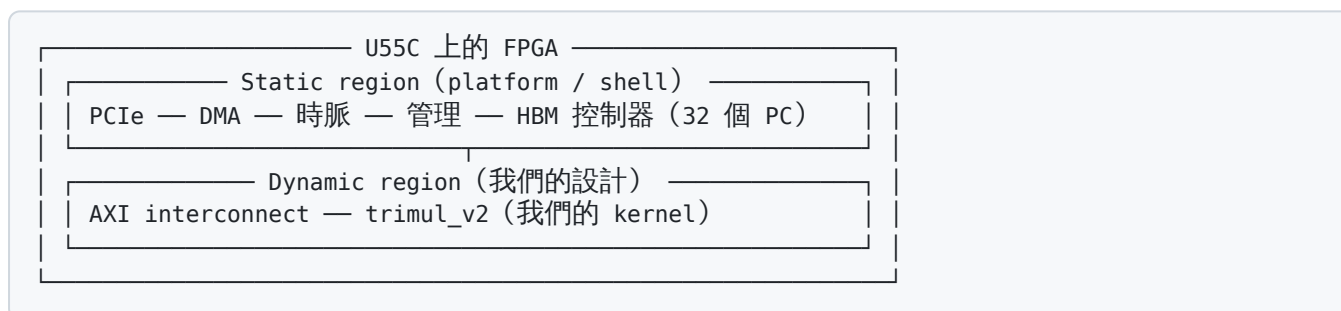
為什麼要跑 1~數小時？擺放和繞線要在數十萬到數百萬個元件中找出好的配置，這是非常難的最佳化問題，工具只能用啟發式方法反覆嘗試。

Timing 沒過會怎樣？電路在目標時脈下可能算錯，工具會自動降低時脈，或需要修改設計。HLS 報告中的 `Estimated 2.431 ns` 只是估計，實際值要等 Vivado 繞線完才知道。

7.6 Vitis 與 `v++`：把 kernel 接到板子上

Platform (平台) 是什麼

FPGA 卡上除了我們的 kernel，還需要很多「基礎設施」：PCIe 介面（和主機溝通）、DMA（搬資料）、HBM 控制器、時脈產生、卡片管理。這些由 AMD 預先做好，叫做 platform 或 shell。



比喻：platform 像電腦的主機板 + 作業系統，kernel 像我們安裝的應用程式。編譯時指定的 platform 必須和卡上實際燒錄的 shell 一致，這就是為什麼 `PLATFORM=` 很重要。

`v++` 的兩個階段

```
v++ -c (compile) : trimul_v2.cpp —(內部呼叫 HLS)—> trimul_v2.xo
                  .xo = RTL + kernel 的描述 (有哪些參數、哪些 AXI 埠)

v++ -l (link) :   trimul_v2.xo + platform + cfg —(內部呼叫 Vivado)—> trimul_v2.xclbin
                  把 kernel 放進 dynamic region、依 cfg 接到 HBM、擺放繞線、產生 bitstream
```

`.xclbin` 裡有什麼

- Bitstream (dynamic region 的電路)
- Metadata：kernel 名稱、每個參數的位置、每個 AXI 埠接到哪幾個 HBM PC (memory topology) → host 程式中的 `krnl.group_id(0)` 就是從這裡查到「參數 0 接在哪塊記憶體」。

7.7 XRT : host 程式怎麼控制 FPGA

XRT (Xilinx Runtime) 是 AMD 提供的軟體：Linux 驅動程式 + 使用者層的函式庫 + 管理工具 (xrt) 。

host 程式的動作	XRT 實際做的事
<code>xrt::device dev(0)</code>	透過驅動程式打開第 0 張卡
<code>dev.load_xclbin(...)</code>	把 bitstream 寫進 FPGA 的 dynamic region，讀取 metadata
<code>xrt::bo bo(dev, size, group)</code>	在指定的 HBM 區域配置一塊記憶體 (buffer object)
<code>bo.sync(TO_DEVICE)</code>	透過 PCIe DMA 把資料從主機記憶體複製到 HBM
<code>run.set_arg(...)</code>	把參數 (HBM 位址、L) 寫進 kernel 的 AXI4-Lite 控制暫存器
<code>run.start()</code>	寫入 start 位元，kernel 開始執行
<code>run.wait()</code>	等待 kernel 的 done 訊號
<code>bo.sync(FROM_DEVICE)</code>	把結果從 HBM 複製回主機

本專題的狀況：你們的 server63 沒有安裝 XRT、也沒有插卡，所以這部分要在有 U55C 的機器上執行。

7.8 Python 端：PyTorch、HuggingFace、ESMFold

元件	誰做的	做什麼	怎麼取得
PyTorch	Meta 主導的開源專案	張量運算、神經網路框架	<code>pip install torch</code>
transformers	HuggingFace 開源專案	各種預訓練模型的程式碼，包含 EsmForProteinFolding	<code>pip install transformers</code>
ESMFold 權重 (<code>facebook/esmfold_v1</code>)	Meta AI 訓練、發布在 HuggingFace Hub	模型學到的參數 (約 8 GB)	第一次執行時自動下載
蛋白質序列	UniProt、RCSB PDB (公開資料庫)	輸入資料	腳本自動從網路下載

`dump_trimul_inputs.py` 的做法是「攔截」：把 ESMFold 裡某個函式換成我們自己的版本，當模型執行到那裡時把資料存下來。模型本身完全沒有修改。

7.9 誰提供了什麼能力？

這是你特別問的問題，這裡清楚劃分。

工具鏈：都是現成的軟體，不是 Claude 做的

能力	提供者	在哪裡	怎麼來的
C++ → 電路 (HLS)	AMD Vitis HLS	你們的 server (/tools/Xilinx/2025.2/)	實驗室／TSRI 安裝的商用軟體（需要授權， /tools/Xilinx/flexlm 是授權管理）
RTL → bitstream	AMD Vivado	同上	同上
把 kernel 接上 platform	AMD Vitis (v++)	同上	同上
RTL 模擬 (cosim)	AMD Vivado 模擬器 (xsim)	同上	同上
控制板子	AMD XRT	需要安裝在有卡的機器上	AMD 開源 (GitHub 上可取得)
U55C platform	AMD	需要安裝在有卡的機器上	AMD 官網下載
C 模擬	GCC (g++)	任何 Linux	開源
流程自動化	GNU Make	任何 Linux	開源
蛋白質模型	Meta ESMFold + HuggingFace transformers + PyTorch	需要時用 pip 安裝、自動下載權重	開源／公開

Claude 做的事

Claude 做了	Claude 沒有做
寫所有的原始碼：5 個 kernel、testbench、host、Makefile、HLS 腳本、cfg、Python 腳本、整理報告的腳本	沒有把 C++ 轉成電路（那是 Vitis HLS 做的，在你們的 server 上執行）
寫所有的文件（包含這份手冊）	沒有執行過 Vitis／Vivado（Claude 的環境裡沒有安裝）
設計實驗（5 個版本、要比較什麼）	沒有連到你們的 server（所有 server 上的指令都是你執行、再截圖給 Claude）
在自己的雲端環境中驗證：g++ C 模擬、Python 小模型測試、假 XRT header 編譯檢查	沒有使用什麼特別下載的模型或外掛來產生程式碼
根據你的截圖推理錯誤原因並修正（例如 2025.2 路徑 bug）	

Claude 的知識從哪裡來

Claude 是一個大型語言模型，在訓練時讀過大量公開資料，包括 AMD 的官方文件、HLS 的教科書與範例、開源程式碼、學術論文等。寫程式時使用的是這些學到的知識，不是即時查詢某個資料庫。

這也意味著 Claude 可能出錯：例如一開始不知道 2025.2 已經移除 `vitis_hls`、也不知道路徑 bug，是看到你的截圖後才修正。所以每一步都需要在真實工具上驗證，這也是為什麼流程要「從快到慢」一步步確認。

7.10 Claude 在這個對話中呼叫的工具

Claude 在一個與你的 server 完全隔離的雲端 Linux 容器裡工作。以下是實際用過的工具：

工具	做了什麼	例子
Bash（在雲端容器中執行指令）	編譯、執行、測試、git 操作	<code>g++ ... tb.cpp</code> 跑 C 模擬； <code>git commit</code> 、 <code>git push</code> ； <code>pip install torch transformers</code>
Write / Edit / Read（檔案操作）	建立與修改檔案	寫 <code>trimul_v2.cpp</code> 、修改 <code>run_hls.tcl</code>
GitHub 工具	讀取 PR 的狀態	檢查 PR #1 有沒有 CI 失敗或審查留言
排程提醒（ <code>send_later</code> ）	定時提醒自己回來檢查	每小時檢查 PR、每 10 分鐘檢查手冊進度
傳送檔案（ <code>SendUserFile</code> ）	把檔案傳給你	傳送 <code>ForClaude.zip</code>

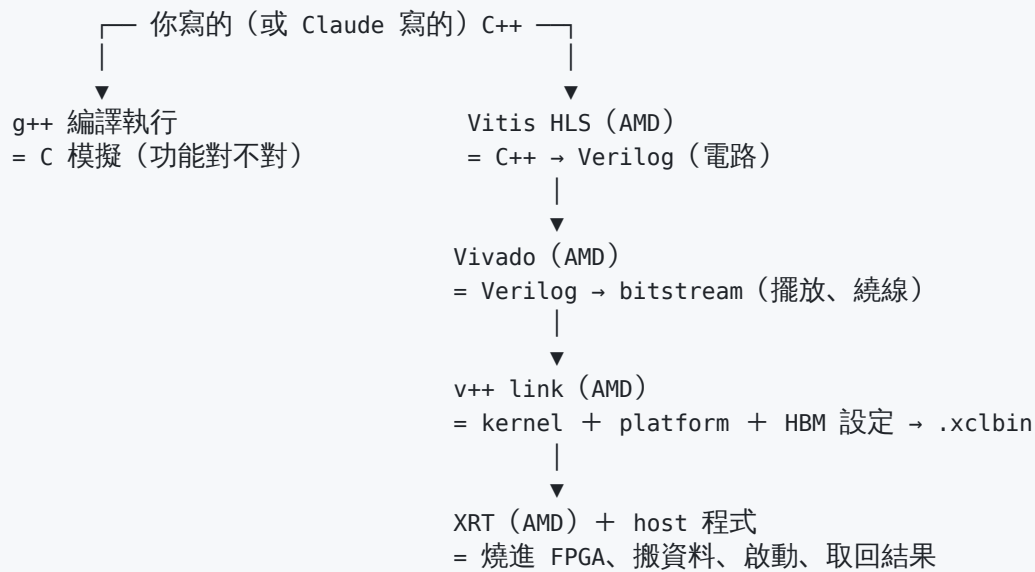
容器裡驗證了什麼、沒驗證什麼

驗證項目	方法	結果
5 個 kernel 的功能	<code>g++</code> C 模擬（ <code>L = 16、32、48、64</code> ；多種 tile 大小）	✅ 全部通過
host 程式能否編譯	自己寫一組「假的」XRT header，只檢查語法	✅ 可編譯（未實際執行）
Python 擷取腳本	隨機初始化的小型 ESMFold（無法下載正式權重，容器的網路政策擋掉了 HuggingFace）	✅ 流程正確、算式差值 0
報告整理腳本	自己產生格式相同的假報告	✅ 可解析
HLS 合成	無法（容器沒有 Vitis）	由你在 server63 上執行
上板	無法	尚待找到有 U55C 的機器

安全性

- Claude 的容器無法存取你的 server，你的 server 也不會連到 Claude 的容器。
- 雙方唯一的交集是 GitHub repo：Claude 推上去，你用 `git clone` / `git pull` 下載。
- 你的 server 只有「下載」程式碼，沒有上傳任何東西。

本章小結



- 把 C++ 變成電路的能力來自 AMD 的 Vitis HLS；Claude 負責寫出「適合被轉成好電路」的 C++，以及整個實驗的設計與文件。
- C++ 能先模擬，是因為 HLS 保證電路與 C++ 的功能一致；pragma 只影響實作方式。
- 每一層工具都有對應的驗證方式，越後面的步驟越慢，所以要先把前面的驗證做好。

自我檢測

讀完這一章，試著回答下面的問題（參考答案在附錄 C）：

1. LUT 為什麼可以實現「任何」小型邏輯函數？
2. 什麼是 II (Initiation Interval)？v0 的累加迴圈為什麼做不到 II = 1？
3. v2 為什麼一定要對 la、lb、acc 做 ARRAY_PARTITION？
4. C 模擬 (csim) 通過，代表什麼？不代表什麼？
5. .xo、.xclbin、platform 分別是什麼？

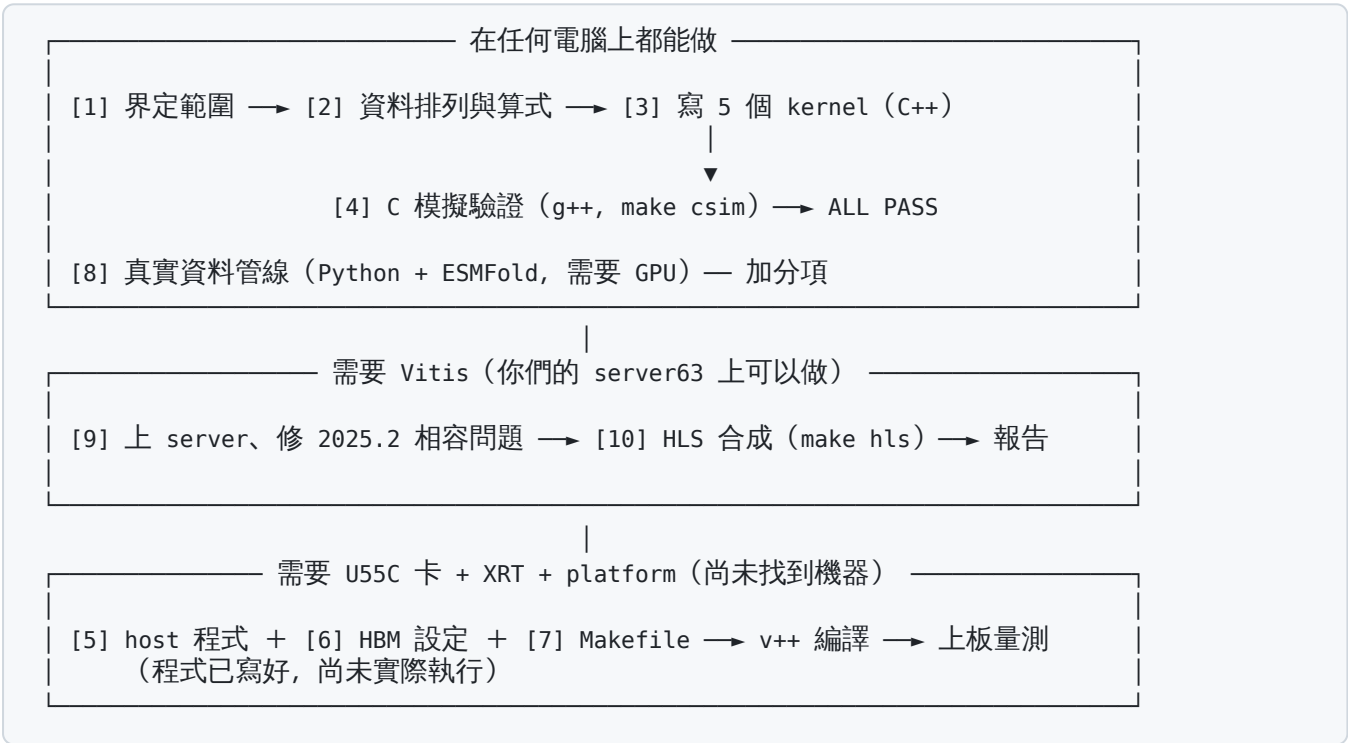
下一章一步一步回顧我們實際做了什麼：每一步為什麼做、程式碼怎麼寫、產生了哪些檔案。

第 8 章 實作：v0~v4 從程式碼到報告

本章重點

- 我們的工作分成十個步驟：界定範圍 → 資料排列 → 寫 kernel → 驗證 → host → HBM 設定 → Makefile → 真實資料 → 上 server 除錯 → 讀報告。
- 每一步都說明：為什麼做、怎麼做、有什麼效果、產生了什麼檔案。
- 最後有一張所有檔案的總表。

8.1 全景圖



8.2 步驟一：界定範圍

為什麼做

LightNobel 處理的是整個 PPM，還設計了專用硬體。專題只有幾週，所以要從中挑出一個能代表問題、又做得完的小範圍。

怎麼決定

考量	Triangle Multiplication 的情況
能代表問題嗎？	✅ 直接處理 LxLx128 的 pair representation

考量	Triangle Multiplication 的情況
算式簡單、容易驗證嗎？	✓ 本質是 128 個矩陣乘法
能展示多種記憶體手法嗎？	✓ tiling、寬位元、重疊、多通道、量化都適用
能呼應 LightNobel 嗎？	✓ 可以比較 per-token vs per-tensor 量化；a、b 屬於論文的 C 組，位元數有依據
是最花時間的運算嗎？	✗ 長序列時是 Triangle Attention (1.4 節)。但它多了 softmax 與 L^3 的 score matrix，不適當第一個目標

產出

一句話的題目定義（見第 10 章 10.1 節），以及「只做 einsum 核心，不含 LayerNorm/gating」的範圍界定。

8.3 步驟二：資料排列（memory layout）與算式

為什麼做

資料在記憶體中怎麼排，決定了能不能做 burst（連續讀取，見第 3 章 3.2 節）。

我們的選擇

a、b、z 都是 $L \times L \times C$ ，排成 [row][col][channel]：
 $\text{index} = (\text{row} \times L + \text{col}) \times C + c$

記憶體中的樣子（ $C = 128$ ）：

(0,0) 的 128 個數	(0,1) 的 128 個數	...	(0,L-1) 的 128 個數	(1,0) ...
----------------	----------------	-----	------------------	-----------

↑ 同一個 token 的 128 個 channel 是連續的 → 可以一次 burst 讀完

效果

- 每個 token 的 128 個 float = 512 bytes = 剛好 8 個 512-bit word，完美對齊寬位元傳輸。
- 這也是 PyTorch 中 ESMFold 的預設排列（最後一維是 channel），所以從 Python 擷取的資料可以直接使用，不用轉換。

產出

fpga/trimul/kernels/trimul.h：定義 $C = 128$ 、tile 大小、f16（16 個 float 的 struct）、q64（64 個 int8 的 struct）等共用設定。

8.4 步驟三：寫 5 個 kernel

以下擷取每個版本最關鍵的部分，完整程式碼在 `fpga/trimul/kernels/`。

v0：最直白的寫法（`trimul_v0.cpp`）

```
for (int i = 0; i < L; i++)
  for (int j = 0; j < L; j++)
    for (int c = 0; c < C; c++) {
      float acc = 0.0f;
      for (int k = 0; k < L; k++) {
#pragma HLS PIPELINE
        acc += a[(i * L + k) * C + c] * b[(j * L + k) * C + c]; // 每次都讀 HBM
      }
      z[(i * L + j) * C + c] = acc;
    }
```

問題在哪：

1. 每次乘加都從 HBM 讀兩個數，完全沒有重用。
2. 最內層的 k 每次跳 C 個位置（stride = 128），無法 burst。
3. `acc += ...` 下一次要等上一次加完（浮點加法需要好幾個 cycle），pipeline 無法每 cycle 開始一次。

v1：Tiling（`trimul_v1.cpp`）

```
float acc[TI][TJ][C]; // 累加器留在晶片內（BRAM）
float la[TI][C], lb[TJ][C]; // 本輪用到的 a、b

for (i0 ...; i0 += TI) // 一次處理 TI 列
  for (j0 ...; j0 += TJ) { // 和 TJ 行
    acc = 0
    for (int k = 0; k < L; k++) {
      load_a: la[ii][c] = a[((i0 + ii) * L + k) * C + c]; // 搬 TI×C 個（連續）
      load_b: lb[jj][c] = b[((j0 + jj) * L + k) * C + c]; // 搬 TJ×C 個（連續）
      mac: acc[ii][jj][c] += la[ii][c] * lb[jj][c]; // TI×TJ×C 次乘加
    }
    store: z[...] = acc[ii][jj][c];
  }
```

改善：每個 `la` 用 TJ 次、每個 `lb` 用 TI 次（搬運量減 8 倍）；每次載入 C 個連續的數（可以 burst）。

一個細節：`mac` 迴圈裡加了

```
#pragma HLS DEPENDENCE variable = acc inter false
```

意思是告訴 HLS：「同一個 `acc` 位置要隔 $TI \times TJ \times C = 8192$ 次迭代才會再被更新，不用擔心前後相依」，這樣 HLS 才敢讓迴圈每個 cycle 開始一次新的迭代（II = 1）。

v2 : 寬位元 + 16 路平行 (`trimul_v2.cpp`)

```
struct f16 { float v[16]; };      // 一個 512-bit word

float acc[TI * TJ * CW][LANES]; // LANES = 16, CW = C/16 = 8
#pragma HLS ARRAY_PARTITION variable = acc dim = 2 complete // 16 個 lane 拆開成獨立的記憶體體

mac:
for (int r = 0; r < TI * TJ * CW; r++) {      // 512 次
#pragma HLS PIPELINE II = 1
    for (int l = 0; l < LANES; l++)            // 這層會被完全展開 → 16 個乘加器同時做
        acc[r][l] += la[...][l] * lb[...][l];
}
```

改善：

- 介面從 `float*` 改成 `f16*` → 每次讀 512 bit (16 個 float)。
- `ARRAY_PARTITION` 把 16 個 lane 拆到不同的記憶體區塊，才能同一個 cycle 讀寫 16 個數。
- 內層 `for l` 在 PIPELINE 裡會被自動展開成 16 份硬體。

v3 : Double buffering (`trimul_v3.cpp`)

```
float la0[...], lb0[...]; // 第一組 buffer
float la1[...], lb1[...]; // 第二組 buffer

load_ab(k = 0 → la0, lb0);
for (int k = 0; k < L; k++) {
    if (k % 2 == 0) {
        if (k + 1 < L) load_ab(k + 1 → la1, lb1); // 搬下一批到 buffer 1
        mac(la0, lb0, acc);                        // 同時用 buffer 0 計算
    } else {
        if (k + 1 < L) load_ab(k + 1 → la0, lb0); // 搬下一批到 buffer 0
        mac(la1, lb1, acc);                        // 同時用 buffer 1 計算
    }
}
```

關鍵：`load_ab` 和 `mac` 是兩個獨立的函式 (`#pragma HLS INLINE off`)，而且使用不同的 buffer、彼此沒有資料相依，所以 HLS 可以讓它們同時執行。

v4 : INT8 per-token 量化 (`trimul_v4.cpp`)

```
struct q64 { int8_t v[64]; };    // 一個 512-bit word = 64 個 int8

// 每個 token 有自己的 scale : sa[i*L + k]、sb[j*L + k]
mac:
for (int r = 0; r < TI * TJ * QW; r++) {
#pragma HLS PIPELINE II = 1
    float s = lsa[ii] * lsb[jj];           // 兩個 token 的 scale 相乘
    for (int l = 0; l < 64; l++) {         // 64 路平行
        int16_t p = (int16_t)la[..][l] * (int16_t)lb[..][l]; // int8 × int8 → int16
        acc[r][l] += (float)p * s;        // 還原尺度後累加
    }
}
```

改善：每次搬 64 個數 (FP32 時只有 16 個)，搬運量變 1/4。為什麼 scale 不能提到迴圈外：每個 k 的 token 不同，scale 也不同，所以每次都要乘。（如果改成 per-channel 量化，scale 就能提到外面，用純整數累加，但就不是 token-wise 了。）

產出

`kernels/trimul_v0.cpp ~ trimul_v4.cpp`，以及共用的 `trimul.h`。

8.5 步驟四：正確性驗證 (C 模擬)

為什麼做

上板編譯一次要 1~數小時。先用幾秒鐘確認程式算得對，才不會浪費時間。

怎麼做 (`tb/tb.cpp`)

- 產生隨機資料 `a`、`b` (`host/common.h` 的 `make_activation`) :
 - 一般 token : 常態分布 $N(0, 1)$
 - 1% 的 token 放大 30 倍：模擬真實 activation 中的 outlier
- 依序呼叫 `v0~v4`，把結果和 CPU 用 double 精度算出的正確答案比較。
- 印出兩種誤差：`rel_l2 = ||z - z_ref|| ÷ ||z_ref||`（整體相對誤差）`max_abs = max |z - z_ref|`（最大的單點誤差）
- `v0~v3` 要求 `rel_l2 < 1e-5`（只有浮點誤差）；`v4` 允許較大誤差（量化本來就有誤差）。

結果

v0 naive	rel_l2=1.141e-07	PASS	
v1 tiled	rel_l2=1.141e-07	PASS	
v2 tiled+wide	rel_l2=1.141e-07	PASS	
v3 tiled+wide+pingpong	rel_l2=1.141e-07	PASS	
v4 int8 per-token	rel_l2=9.871e-03	PASS	← 約 1%
v4 int8 per-tensor	rel_l2=1.126e-01	PASS	← 約 11%

- v0~v3 的誤差完全一樣 (1.1e-7)，代表改寫沒有改變計算結果。
- per-tensor 誤差約是 per-token 的 11 倍，這是本專題第一個可以放進報告的結論。

為什麼 g++ 能跑 HLS 程式？

因為 kernel 就是普通的 C++。#pragma HLS ... 對 g++ 來說是不認識的指示，會被忽略（我們加了 -Wno-unknown-pragmas 讓它不要警告）。詳見第 7 章 7.4 節。

產出

tb/tb.cpp、host/common.h；指令 make csim。

8.6 步驟五：host 程式

為什麼做

FPGA 不會自己去拿資料。需要一個在 CPU 上執行的程式負責：準備資料 → 送進 FPGA 的 HBM → 啟動 kernel → 等它算完 → 取回結果 → 檢查、計時。

流程（host/host.cpp，使用 XRT 的 C++ API）

```
xrt::device dev(0); // 1. 打開第 0 張 FPGA 卡
auto uuid = dev.load_xclbin("trimul_v2.xclbin"); // 2. 把編譯好的電路燒進去
xrt::kernel krnl(dev, uuid, "trimul_v2"); // 3. 找到 kernel

xrt::bo bo_a(dev, bytes, krnl.group_id(0)); // 4. 在 HBM 上配置記憶體
bo_a.write(a.data()); // (group_id 決定放在哪個 PC)
bo_a.sync(XCL_BO_SYNC_BO_TO_DEVICE); // 5. 從 CPU 記憶體複製到 HBM

run.start(); run.wait(); // 6. 啟動 kernel 並等待完成（計時）

bo_z.sync(XCL_BO_SYNC_BO_FROM_DEVICE); // 7. 把結果複製回 CPU
bo_z.read(z.data());
```

最後印出時間、GFLOP/s、估計的搬運量和頻寬、HBM 佔用量、誤差，以及一行 CSV 方便整理。

狀態

⚠ 已寫好，並用假的 XRT header 確認可以編譯，但尚未在真的板子上執行過。

產出

host/host.cpp；指令 make host、make run。

8.7 步驟六：HBM 連接設定

為什麼做

kernel 有好幾個 AXI 連接埠 (a、b、z……)，每個要接到 HBM 的哪個 pseudo-channel，是在連結 (link) 階段由設定檔決定的。

設定檔 (`cfg/trimul_v2.cfg`)

```
[connectivity]
sp=trimul_v2_1.a:HBM[0:7]      # a 接到 PC 0~7 (共 4 GB)
sp=trimul_v2_1.b:HBM[8:15]    # b 接到 PC 8~15
sp=trimul_v2_1.z:HBM[16:23]   # z 接到 PC 16~23
```

- `trimul_v2_1`：kernel 的第一個實例 (instance)。
- `sp`：system port，指定「kernel 的某個參數要接到哪塊記憶體」。
- 對照組 `trimul_v2_onebank.cfg`：全部接到 `HBM[0]`。

產出

`cfg/trimul_v0.cfg` ~ `trimul_v4.cfg`、`trimul_v2_onebank.cfg`。

8.8 步驟七：Makefile

為什麼做

完整流程有很多步驟、每個指令都很長。Makefile 把它們包成短指令，參數也集中管理。

指令一覽

指令	做什麼	需要什麼
<code>make csim</code>	g++ 編譯並執行 testbench	只要 g++
<code>make hls KERNEL=trimul_v2</code>	執行 <code>hls/run_hls.tcl</code> ，產生合成報告	Vitis
<code>make hls-all</code>	依序合成 v0~v4，最後呼叫 <code>make summary</code>	Vitis
<code>make hls-sweep</code>	tile 大小掃描 (4/8/16/32)，各自存成不同專案	Vitis
<code>make model</code>	分析模型：預測、驗證、設計空間探索 (見 9.2 節)	Python
<code>make summary</code>	執行 <code>scripts/hls_summary.py</code> ，產生比較表與 <code>hls_summary.csv</code>	Python
<code>make xclbin KERNEL=... TARGET=hw</code>	<code>v++ -c</code> (編譯) + <code>v++ -l</code> (連結) → <code>.xclbin</code>	Vitis + U55C platform
<code>make host</code>	編譯 host 程式	XRT

指令	做什麼	需要什麼
<code>make run ...</code>	執行 (emulation 或上板)	XRT (上板還需要卡)

可調參數

`KERNEL`、`TARGET` (`sw_emu` / `hw_emu` / `hw`)、`PLATFORM`、`CFG`、`DEFS` (例如 `tile` 大小)、`TAG`、`RUN_L`、`DATA`。

產出

`Makefile`、`hls/run_hls.tcl`、`scripts/hls_summary.py`、`.gitignore`。

8.9 步驟八：真實資料管線 (加分項)

為什麼做

模擬資料的 outlier 是我們「假設」的。要證明結論在真實蛋白質上也成立，需要從 ESMFold 取出真正的 activation。

怎麼做 (`python/dump_trimul_inputs.py`)

1. 下載序列 (UniProt / PDB) 或讀 FASTA。
2. 載入 HuggingFace 的 ESMFold (`facebook/esmfold_v1`)。
3. 把 `trunk.blocks[N].tri_mul_out._combine_projections` 換成我們的函式：當模型執行到這裡時，把 `a`、`b` 存下來，然後丟出例外讓模型停止 (後面的 block 不用跑，省時間)。
4. 計算統計數字 (輸出成 JSON，也存進 `meta.json`)：
 - outlier 程度、per-token / per-tensor 的 INT8 誤差；
 - `paper_group_C_check`：和 LightNobel C 組對照的統計 (每個 token 的平均絕對值、 3σ outlier 數)；
 - `rel_l2_grid`：INT8 / INT4 $\times$ per-token / per-channel / per-tensor 的 einsum 誤差 (用途見 10.4 節)。
5. 補零到 32 的倍數，存成 `a.bin`、`b.bin` (float32，排列和 kernel 相同)。

驗證：「差值 0」是怎麼來的

在隨機初始化的小型 ESMFold 上，擷取 `a`、`b` 後比較：

```
ref = tm._combine_projections(a, b)      # ESMFold 自己的計算
ours = torch.einsum("ikc,jkc->ijc", a, b) # 我們 kernel 的算式
max diff = 0.0
```

證明我們的 kernel 算的確實是 ESMFold 裡的那個運算。

過程中發現的錯誤

一開始用 tokenizer 產生輸入，結果模型報錯 (index out of range)。原因是 ESMFold 的 forward 需要的是 AlphaFold 的胺基酸編號，不是語言模型 tokenizer 的編號；要用 `model.infer(seq)` 才會正確轉換。

狀態

需要 GPU (或 Colab) 才能跑正式模型；使用者沒有 GPU，所以目前是加分項。C 模擬、host 程式和 Makefile 都已支援讀取這些檔案 (DATA=...)。

產出

`python/dump_trimul_inputs.py`、`python/requirements.txt`、`docs/real_protein_data.md`。

8.10 步驟九：在實際 server 上除錯

環境調查

指令	結果	結論
<code>which vivado vitis v++</code>	都在 <code>/tools/Xilinx/2025.2/</code>	工具齊全，版本很新
<code>which vitis_hls</code>	找不到	2025.2 已移除，改用 <code>vitis-run</code>
<code>lspci -d 10ee:</code>	空的	這台沒有 FPGA 卡 (10ee 是 Xilinx 的廠商代號)
<code>xbutil</code> 、 <code>/opt/xilinx/platforms</code>	沒有	沒有 XRT、沒有 U55C platform

→ 這台可以做 C 模擬和 HLS 合成，不能上板。

除錯過程：一次失敗、三次修正

這是一個很好的除錯範例，展示「從錯誤訊息推理出原因」的過程。

第 1 次失敗

```
ld.lld: error: undefined symbol: trimul_v2
```

- 觀察：編譯清單裡有 `tb`、`v0`、`v1`、`v3`、`v4`，唯獨沒有 `v2`。
- 當時的判斷：2025.2 的 C 模擬流程有問題 → 改成預設跳過 HLS 內建的 C 模擬 (反正 `make csim` 已經驗證過)。
- (後來發現真正原因和第 2、3 次相同。)

第 2 次失敗

```
WARNING: Cannot find source file ../kernels/trimul_v2.cpp; skipping it.  
ERROR: Cannot find any design unit to elaborate.
```

- 觀察：工具找不到來源檔。
- 假設：相對路徑的問題 → 改用絕對路徑。

第 3 次失敗（相同錯誤）

- 關鍵觀察：日誌中 Adding design file `'/home/anthony/.../trimul_v2.cpp'` 已經是絕對路徑，但到了合成階段，工具又自己改寫成 `../kernels/trimul_v2.cpp`。
- 推理：專案位置：`fpga/trimul/hls_prj/trimul_v2/`（在目前資料夾往下 2 層）正確的相對路徑：`../../kernels/trimul_v2.cpp` 工具算出來的：`../kernels/trimul_v2.cpp`（只往上 1 層）→ 工具假設專案只在目前資料夾的下一層。專案名稱含有 `/`（`hls_prj/trimul_v2`）時就會算錯。
- 修正：先 `cd hls_prj`，再建立名為 `trimul_v2` 的專案（只差一層）。

第 4 次：成功 ✓

學到的除錯方法

1. 仔細看完整的錯誤訊息，不只看 ERROR，也看前面的 WARNING。
2. 比較「預期」和「實際」（編譯清單少了一個檔案、路徑少了一層）。
3. 修正後結果不變 → 假設錯了，回頭找新的線索。
4. 用算的驗證假設（數路徑的層數）。

產出

修改過的 `Makefile`（自動選 `vitis_hls` 或 `vitis-run`）與 `hls/run_hls.tcl`。

8.11 步驟十：讀懂 HLS 報告（以 v2 為例）

報告位置：`hls_prj/trimul_v2/sol/syn/report/trimul_v2_csynth.rpt`

① 基本資訊

```
* Version:      2025.2
* Solution:      sol (Vitis Kernel Flow Target)
* Product family: virtexuplusHBM
* Target device: xcu55c-fsvh2892-2L-e    ← U55C 的晶片型號
```

② Timing（時脈）

```
| Clock | Target | Estimated | Uncertainty |
| ap_clk | 3.33 ns | 2.431 ns | 0.90 ns |
```

欄位	意義
Target	我們要求的時脈週期（3.33 ns = 300 MHz）
Estimated	HLS 估計電路最慢的路徑需要多久

欄位	意義
Uncertainty	保留給後續繞線的餘裕

判斷：Estimated + Uncertainty = 3.33 ns，剛好在目標內 → 通過。（最終能否達到 300 MHz，要等 Vivado 實際繞線後才知道。）

③ Latency (總時間)

Latency (cycles)	Latency (absolute)	Interval
177,029,185	0.590 sec	177029186

- Latency：從開始到結束的 cycle 數。
- absolute：cycle 數 × 時脈週期。
- Interval：可以接受下一次呼叫的間隔（這裡和 latency 差不多，因為一次只跑一個）。
- ⚠ 這是在 L = 256 下的估計（由 trimul.h 的 L_TRIP 決定）。因為 L 是執行時才知道的參數，HLS 需要 LOOP_TRIPCOUNT 告訴它迴圈大概跑幾次，才能估算。

④ Instance (子模組)

Module	Latency (cycles)	
trimul_v2_Pipeline_init	514	← 清空累加器 (512 次 + 啟動)
trimul_v2_Pipeline_load_a	141	← 載入 a
trimul_v2_Pipeline_load_b	141	← 載入 b
trimul_v2_Pipeline_store	586	← 寫回結果
trimul_v2_Pipeline_mac	526	← 計算 (512 次 + pipeline 深度 14)

每個 pipeline 化的迴圈會變成一個獨立的硬體模組。

⑤ Loop (迴圈)

Loop Name	Latency	Iteration Latency	Trip Count
tile_i	177,029,184	5,532,162	32
tile_j	5,532,160	172,880	32
loop_k	171,776	671	256

驗算：

```

loop_k: 256 次 × 671 = 171,776 ✓
tile_j 每次: 171,776 (loop_k) + 514 (init) + 586 (store) + 4 ≈ 172,880 ✓
tile_i 每次: 32 × 172,880 + 2 = 5,532,162 ✓
總計: 32 × 5,532,162 ≈ 177,029,184 ✓

```

loop_k 的 671 cycles 就是第 3 章 3.3 節分析 double buffering 的依據：load (141, a、b 並行) + mac (526) + 少量 ≈ 671。

⑥ 換算效能

運算量 = $2 \times 256^3 \times 128 \approx 4.29 \times 10^9$ FLOP
時間 = 0.590 s
效能 ≈ 7.3 GFLOP/s (v2 理論上限 9.6 GFLOP/s 的 76%)

⑦ 資源用量 (報告的下一段)

Utilization Estimates 段落會列出 BRAM_18K、DSP、FF、LUT、URAM 的用量和百分比。 `make summary` 會自動把這些數字整理進比較表。

8.12 所有檔案總表

檔案	用途	狀態
README.md	repo 目錄	✓
docs/PROJECT_LOG.md	專題工作紀錄 (交接用)	✓
docs/conversation_notes.md	白話筆記	✓
docs/getting_started_u55c.md	U55C 環境與開發流程	✓
docs/real_protein_data.md	真實資料流程	✓
docs/manual/	本教學手冊 (含 manual.pdf)	✓
docs/three_week_plan.md	三週計畫與海報大綱	✓
docs/competition_directions.md	競賽方向與文獻紀錄	✓
fpga/trimul/kernels/trimul.h	共用設定	✓
fpga/trimul/kernels/trimul_v0~v4.cpp	5 個 kernel	✓ csim ; v2 已合成
fpga/trimul/tb/tb.cpp	C 模擬 testbench	✓
fpga/trimul/host/common.h	資料產生、量化、參考答案、讀檔	✓
fpga/trimul/host/host.cpp	XRT host 程式	⚠ 未上板
fpga/trimul/cfg/*.cfg	HBM 連接	⚠ 未使用
fpga/trimul/hls/run_hls.tcl	HLS 腳本	✓ (2025.2)
fpga/trimul/scripts/hls_summary.py	報告整理成表	✓ (用假報告測過)
fpga/trimul/scripts/perf_model.py	分析模型與設計空間探索 (<code>make model</code> , 見 9.2 節)	✓ (v2 已校正)
fpga/trimul/Makefile	指令捷徑	✓ csim/hls ; ⚠ xclbin/run
python/dump_trimul_inputs.py	擷取真實 activation	✓ (小模型測過)

執行後會產生（不進 git）：`hls_prj/`（HLS 專案與報告）、`hls_*.log`、`hls_summary.csv`、`build/`（v++ 產物）、`*.exe`。

本章小結

- 流程的核心精神：從快到慢驗證（C 模擬 → HLS → emulation → 上板），前一步沒過就不進下一步。
 - 每個 kernel 版本只改一件事，才能清楚歸因每一招的效果。
 - 除錯時，比較預期與實際、修正無效就換假設。
 - 進度（截至 2026-09-26）：C 模擬全部通過、v2 合成成功，v0 ~ v4 全部合成進行中；上板尚待找到有 U55C 的機器。最新進度見 `docs/PROJECT_LOG.md`。
-

自我檢測

讀完這一章，試著回答下面的問題（參考答案在附錄 C）：

1. 為什麼把資料排成 `[row][col][channel]`？這對 burst 有什麼好處？
2. `rel_l2` 怎麼計算？testbench 為什麼對 v4 用比較寬鬆的門檻？
3. Vitis 2025.2 找不到來源檔的根本原因是什麼？最後怎麼修？
4. 請用 v2 報告的數字驗算：`loop_k` 每輪 671 cycles 是怎麼組成的？
5. host 程式的哪一步把資料從 CPU 記憶體搬到 HBM？

下一章說明怎麼評估一個還沒上板的設計：architectural simulation、分析模型與設計空間探索。（要動手跑實驗，指令都整理在附錄 A。）

第 9 章 評估方法：architectural simulation、分析模型與設計空間探索

本章重點

- Architectural simulation 不一定要上板：HLS 合成報告加上校正過的分析模型，就是正統的架構評估方法（LightNobel 也是用模擬器評估）。
- 分析模型：用公式預測 cycle 數；先用 HLS 報告校正常數，再用沒看過的設定驗證。
- 設計空間探索（DSE）與 Pareto 前緣：在上百種設定中找出最佳的取捨。
- 容量模型：記憶體和 L^2 成正比，所以 L 只和記憶體的平方根成正比；和論文比較時要用 FP16 當基準。

9.1 Architectural simulation 是什麼？

白話直覺

在真正做出晶片之前，先用「模型」預測它會有多快、用多少資源。就像蓋大樓之前先做結構計算和模型，而不是直接蓋。

評估的四個層級

層級	方法	速度	準確度	本專題
分析模型	用公式算（cycle = 迴圈次數 × 每次的 cycle）	毫秒	中（要校正）	make model
Cycle 級模擬	模擬器或 HLS 排程結果	分鐘	高	make hls 的報告
RTL 模擬	模擬實際的電路	小時	很高	COSIM=1（可選）
上板量測	真的在硬體上跑	編譯要數小時	最高（真實值）	第二階段

越上面越快、越能大量探索；越下面越準、但越慢。好的做法是：

1. 用下層（HLS 報告）校正上層（分析模型）；
2. 用校正後的上層大量探索；
3. 挑幾個最好的設定，回到下層驗證。

學術界也這樣做

架構類論文（包括 LightNobel）大多用自己寫的 cycle 級模擬器加上合成工具的面積、功耗估計來評估，而不是真的做出晶片。LightNobel 的做法（Sec. 6）：用 Python 寫 cycle-accurate 模擬器，再拿 RTL 模擬的結果交叉驗證，四個資料集的誤差平均 3.30%、都在 5% 以內；面積與功耗用 Synopsys DC（28 nm）和 CACTI 估計，記憶體用 Ramulator 模擬（各工具的作用見 6.9 節）。我們的對應做法：分析模型 ↔ HLS 報告交叉

驗證（9.2 節），目標同樣是誤差 10% 以內。所以「HLS 報告 + 分析模型」是正當的 architectural simulation，不是偷懶。

9.2 分析模型：用公式預測硬體表現

為什麼要做模型

- 1. 快：一次 HLS 合成要幾分鐘到幾十分鐘，模型只要幾毫秒 → 可以探索上百種設定。
- 2. 能解釋：公式直接告訴你時間花在哪裡。
- 3. 能預測：還沒實作的設計（例如融合架構）也能先估計。

模型的結構（以 v2 為例）

總 cycle

= tile 數 × 每個 tile 的 cycle

tile 數

= (L / TI) × (L / TJ)

每個 tile

= init + L × loop_k + store + 少量額外

loop_k

= load + mac + 4

(v2：輪流做)

= max(load, mac) + 4

(v3：重疊)

load

= max(TI, TJ) × C/16 + 77

(77 = HBM 延遲與啟動成本)

mac

= TI × TJ × C/16 + 14

(14 = pipeline 深度)

校正 (calibration)

公式裡有一些無法事先知道的常數（例如 77、14），要從實際的 HLS 報告反推：

報告中的數字	公式	反推出的常數
load = 141	$8 \times 8 + \text{overhead}$	overhead = 77
mac = 526	$8 \times 8 \times 8 + \text{depth}$	depth = 14
store = 586	$512 + \text{overhead}$	overhead = 74

驗證 (validation)

用校正好的模型預測 v2：177,029,120 cycles；HLS 報告：177,029,185 cycles → 誤差 < 0.01%。

⚠ 注意：用同一份報告校正又驗證，當然會很準。真正的驗證是：

- 用 v2 (8×8) 校正，去預測 v2 的 16×16、32×32 (make hls-sweep 的結果)；
- 以及預測 v3、v4 (make hls-all 的結果)。如果誤差在 10% 以內，模型就可信；誤差大的話，要找出是哪一項公式沒考慮到，這個過程本身也是很好的報告內容。

模型的限制（報告中要誠實寫出）

- HLS 報告本身也是估計值，實際上板可能因為 HBM 的真實延遲、時脈而不同。
- 資源 (DSP、BRAM) 目前是粗估，只能看趨勢。
- 沒有模擬 HBM 的頻寬競爭（多個 port 搶同一個通道的情況）。

9.3 設計空間探索 (DSE) 與 Pareto 前緣

設計空間

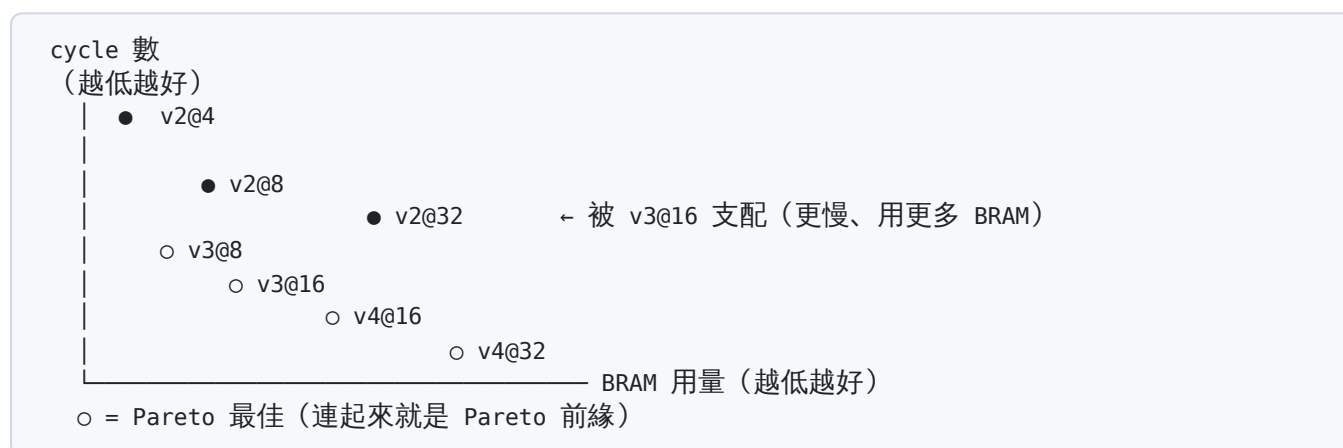
所有「可以調的旋鈕」組成的空間：

旋鈕	可選值
版本	v2、v3、v4、v5、融合
Tile 大小	4、8、16、32
平行度	16、32、64 lane
精度	FP32、INT8、INT4 (v5)

組合起來有上百種，每種都去合成太慢，所以用模型快速評估。

Pareto 前緣

通常沒有「各方面都最好」的設計：快的用比較多資源、省資源的比較慢。Pareto 最佳：找不到另一個設計在所有指標都一樣好、而且至少一項更好。



make model 目前的結果 (模型值, v4 常數待校正) : Pareto 最佳 : v2@8×8、v3@8×8、v3@16×16、v4@16×16、v4@32×32

怎麼選

- 資源很緊 → 選前緣左端 (省資源)。
- 要最快 → 選前緣右端。
- 報告中可以說：「我們選 X，因為它在前緣的『膝點』 (再多花資源，速度提升就很有限)」。

9.4 容量模型：最多能處理多長的序列

公式

$$\begin{aligned}\text{需要的記憶體} &= L^2 \times (\text{每個 } L^2 \text{ 位置的 bytes}) \\ \text{最大 } L &= \sqrt{\text{可用記憶體} \div \text{每個 } L^2 \text{ 位置的 bytes}}\end{aligned}$$

例子 (U55C 的 16 GB HBM)

「典型實作」是每一步都把中間值寫回記憶體（約 7 份，3.5 節）；「融合」是本專題的兩個 kernel 設計（10.3 節）。

配置	bytes / L^2	最大 L
典型實作（約 7 份 FP32）	$7 \times 512 = 3,584$	$\sqrt{16 \times 2^{30} \div 3,584} \approx 2,190$
融合 + INT8 中間值	$2 \times 512 + 2 \times 132 = 1,288$	$\approx 3,650$

上表以 FP32 為基準，會誇大效果。LightNobel 的比較基準（ESMFold）是 FP16，換成 FP16 重算（make model 已加入這幾列）：

配置（FP16 基準）	bytes / L^2	最大 L	相對
典型實作（約 7 份 FP16）	$7 \times 256 = 1,792$	$\approx 3,096$	1.00×
融合 + FP16 z + INT8 a、b	$2 \times 256 + 2 \times 132 = 776$	$\approx 4,705$	1.52×
融合 + FP16 z + INT4 a、b（延伸）	$2 \times 256 + 2 \times 68 = 648$	$\approx 5,148$	1.66×

報告時用 FP16 基準的 1.52 倍比較誠實；FP32 基準的 1.67 倍只能說「相對於我們自己的 FP32 實作」。

為什麼只提升 1.67 倍，不是 7 倍？

因為記憶體和 L^2 成正比，L 只和記憶體的平方根成正比：

$$L \text{ 的提升} = \sqrt{\text{記憶體減少的倍數}} = \sqrt{3,584 \div 1,288} = \sqrt{2.78} \approx 1.67$$

要讓 L 變 2 倍，記憶體要降到 1/4；要變 10 倍，記憶體要降到 1/100。這也是為什麼 LightNobel 要把 peak memory 降低到上百倍，才能大幅延長序列。

LightNobel 的 120 倍是怎麼來的、和我們的 1.5 倍能不能比，見第 5 章 5.4 節。

本章小結

- 評估的四個層級：分析模型 → cycle 級（HLS）→ RTL 模擬 → 上板；用下層校正上層、用上層大量探索。
- 分析模型：v2 預測誤差 < 0.01%，但真正的驗證要用沒看過的設定（tile sweep、v3、v4）。

- DSE：找 Pareto 前緣上的「膝點」。
 - 容量：以 FP16 為基準，融合 + INT8 讓最大 L 提升約 1.52 倍。
-

自我檢測

讀完這一章，試著回答下面的問題（參考答案在附錄 C）：

1. 分析模型用同一份報告校正又驗證，為什麼還不夠？應該怎麼驗證？
 2. Pareto 最佳的定義是什麼？
 3. 每個 L^2 位置的記憶體從 3,584 bytes 降到 1,288 bytes，最大序列長度提升幾倍？
 4. 為什麼和 LightNobel 比較時要用 FP16 當基準？用 FP32 會發生什麼事？
-

下一章把前面所有內容收斂成本專題的策略：定位、設計、要驗證什麼，以及海報和評審問答。

第 10 章 專題策略：定位、設計、海報與評審問答

本章重點

- 定位：以 LightNobel 的原則為基礎，在現成 FPGA 上實作 Triangle Multiplication，拆解每一招的貢獻；融合與執行時量化的原則來自 LightNobel，不是我們的新概念。
- 策略三根支柱：逐步拆解（v0~v5）、以論文為依據的融合與量化設計、經過驗證的分析模型與 DSE。
- 設計：融合成兩個 kernel（流量約 1/7、容量約 7 份 → 2.5 份）；a、b 屬 C 組 → 先用真實資料驗證，再做 INT4 的 v5。
- 呈現：海報對應、30 秒與 3 分鐘講法、評審可能問的問題。

10.1 定位：題目、選題理由、和 LightNobel 的關係

題目一句話

在 FPGA 上實作 PPM 的 pair representation 運算之一（Triangle Multiplication），用逐步優化（v0~v5）與分析模型，量化每一招（包括依論文設定的 token-wise 低位元量化）帶來多少改善、付出多少硬體代價。

和 LightNobel 的關係：繼承它的問題定義、token-wise 量化與管線化（中間值不落地）的原則，簡化成單一運算、單一精度，新增 FPGA 實作、逐步 ablation、真實資料驗證與分析模型。完整對照見本節後面的「繼承、簡化、新增」，策略細節見 10.2 節。

為什麼選 Triangle Multiplication

1. 它是 pair representation 的運算，直接受 L^2 資料量影響；短序列時它佔執行時間最多（36.1%）。
2. 算式簡單（本質是矩陣乘法，沒有 softmax），容易驗證正確性，適合當第一個目標。
3. 可以清楚展示「同樣的計算，不同搬運方式」的差異。
4. 量化的效果（per-token vs per-tensor）可以直接呼應 LightNobel。

einsum 是 LightNobel 的重點嗎？不是。LightNobel 優化的是整個 Folding Block，最大的收益來自 Triangle Attention 不存 score matrix 與全面的量化（5.3、5.4 節）。選 einsum 是本專題的範圍決定：它最單純、是 Triangle Multiplication 裡唯一跨 token 且搬運最多的一步，也是三週內、第一次用 FPGA 做得完的範圍。報告時可以說：「LightNobel 優化整個 Folding Block；我們從其中最基本、最能代表 memory-bound 行為的矩陣運算開始，逐步拆解每一招的效果。」

誠實說明：長序列時最花時間的是 Triangle Attention（75.9%），不是 Triangle Multiplication。Triangle Attention 多了 softmax 和立方大小的 score matrix，需要 FlashAttention 式的 token-wise 做法

(LightNobel 的 Sec. 5.4) ，三週內做不完，列為未來工作。本專題的 tiling、寬位元、double buffering、量化和分析模型都可以直接沿用過去。

為什麼用 FPGA

- FPGA 可以自己決定記憶體怎麼接、資料怎麼搬、運算單元擺幾個，適合展示記憶體優化的每一個細節。
- U55C 有 HBM（多通道高頻寬記憶體），可以做多通道的實驗。
- GPU 上這些細節大多被硬體和驅動程式隱藏，比較難單獨觀察每一招的效果。
- LightNobel 自己也指出：GPU 做 activation 量化效率很差（要在執行時量化和反量化，用不到 Tensor Core；W4A4 甚至比 FP16 慢），所以才需要專用硬體。FPGA 是「不用流片就能做專用資料路徑」的折衷。

繼承、簡化、新增

	內容	說明
繼承自論文	問題定義（長序列的 activation 記憶體）	直接沿用 LightNobel 的動機
	以 ESMFold 為對象	同樣的模型，已驗證本專題的 einsum 與 ESMFold 完全一致
	token-wise 量化的想法	v4 的 per-token INT8
	峰值記憶體、可處理的序列長度作為指標	容量模型（9.4 節）
	管線化、執行時量化（中間值不落地）	LightNobel 的 RMPU → VVPU 管線已經這樣做；我們在 FPGA 上用 HLS 實現同樣的原則（3.5、10.4 節）
	模擬器 + 交叉驗證的評估方法	論文用 cycle-accurate 模擬器，並和 RTL 比對（誤差 < 5%）；我們用分析模型，並和 HLS 報告比對
簡化	三組混合精度 → a、b 只用單一精度（INT8，下一步 INT4）	a、b 屬 C 組，論文對 C 組也只用單一精度
	執行時 top-k outlier → 不另外處理	a、b 屬 C 組，論文本身也不處理 outlier
	完整模型 → 只做 Triangle Multiplication	不是最花時間的運算（長序列時是 Triangle Attention），但它是結構最單純的 pair 運算，適合當第一個目標
	ASIC 模擬 → FPGA 的 HLS 合成	評估對象是現成可燒錄的硬體
	現成 FPGA 平台（U55C，HBM）上的實作	論文沒有公開硬體；我們的設計可以合成、之後可以上板
新增（我們的）	逐步 ablation（v0 ~ v5）	把 tiling、寬位元、double buffering、量化各自的效果分開量；論文只給整體結果
	完整 Triangle Multiplication 的融合設計（含 LayerNorm、gating、重算 g）	把論文的管線化原則具體套到一個運算上，並算出每一步省下多少流量與容量
	分析模型 + 設計空間探索	用 HLS 報告校正，可預測、可找最佳設定

	內容	說明
	真實資料驗證量化假設（規劃中）	用 ESMFold 的真實 a、b 對照論文 C 組的統計，並比較 INT8 / INT4 × 三種粒度的誤差（10.4 節）
	INT4 的 v5 kernel（規劃中）	依論文 C 組的設定，在 FPGA 上實作低位元的 token-wise 資料路徑（10.4 節）

我們的貢獻（誠實版）

1. 在 FPGA 上實作 PPM 的 Triangle Multiplication：搜尋範圍內沒有找到公開的類似實作；LightNobel 本身也只有模擬結果。
2. 把各個記憶體優化手法的貢獻分開量化（ablation，v0 ~ v5）：論文是整套一起評估，看不出每一招各佔多少。
3. 量化設定有論文依據、也有真實資料驗證：不是隨便挑一個 bit 數。
4. 經過驗證的分析模型與設計空間探索：讓結果不只是幾個數據點。

注意：融合（中間值不落地）與執行時量化，LightNobel 都已經做了，不是我們的新概念。我們的角色是「在現成的 FPGA 上實現並拆解這些想法」，不是提出新的原理。

也不要宣稱「比 LightNobel 好」：兩者平台不同（ASIC vs FPGA）、範圍不同（整個模型 vs 一個運算）。比較好的說法是：「以 LightNobel 的原則為基礎，在現成硬體上實作其中一個運算，並量化每一招各自的貢獻。」

10.2 策略：從「比較手法」到「拆解與設計」

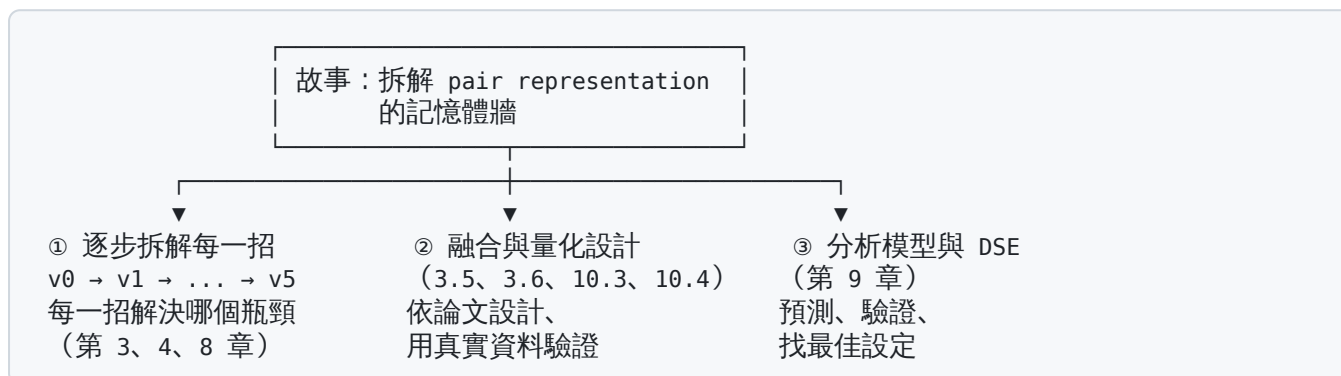
策略的演變

	第一版：比較手法	第二版：提出融合架構	現在：讀完原文後修訂
主軸	比較 5 個常見手法	融合資料流架構，常見手法當 ablation	拆解每一招的貢獻（ablation 本身就是主要貢獻）
量化	INT8 per-token	INT8 per-token	INT8 → INT4（依論文的 C 組設定），並用真實資料驗證
融合	無	主打「我們的架構」	原則來自 LightNobel；我們做設計 + 分析模型評估，kernel 當加分
評估	5 個數據點	分析模型 + DSE	分析模型 + DSE，對應論文「模擬器 + 交叉驗證」的方法
和 LightNobel 的關係	模仿它的量化	「從融合這個互補的角度」	在現成 FPGA 上實作它的原則，並拆解每一招的貢獻

	第一版：比較手法	第二版：提出融合架構	現在：讀完原文後修訂
需要板子？	最後要上板才完整	不需要	不需要，HLS + 模型就是完整的架構評估

第二版會被修訂，是因為讀原文後發現：融合與執行時量化論文已經做了；長序列時最花時間的是 Triangle Attention；論文的比較基準是 FP16。這個過程本身是很好的研究方法教材，見第 11 章 11.11 節。

三根支柱



- ① 是主軸：v0~v4 已完成大半，v5 (INT4) 是第 2 週的工作。LightNobel 沒有拆開每一招，這是我們最直接的貢獻。
- ② 是設計：融合的流量與容量估算、位元數的選擇，都以論文為依據，並用真實 ESMFold 資料驗證。
- ③ 是方法論：讓結果可信、可以推廣。

10.3 融合設計：兩個 kernel 與效益

第 3 章 3.5、3.6 節介紹了融合與重算的原理，這一節是本專題的具體設計。

融合後的兩個 kernel

Kernel 1: Producer (逐 token 處理)
 讀 z (一次)
 → LayerNorm → 4 個 linear → sigmoid gating → mask
 → token-wise 量化 (INT8 或 INT4, per-token scale 當場算出)
 寫出：a、b (低位元 + scale)

Kernel 2: Consumer (einsum + epilogue)
 讀 a、b (tiling 後的 v4/v5 kernel)
 → einsum → LayerNorm_out → linear_z
 → 從 z 重新算 $g = \text{sigmoid}(\text{linear}_g(\text{LayerNorm}(z)))$ → 相乘
 寫出：輸出

用數字看效果：記憶體流量

以「每個 L^2 位置需要搬多少 bytes」計算（FP32、 $C = 128$ 、每份張量 $512 \text{ B}/L^2$ ；einsum 以理想的最少讀取計算）：

步驟	不融合（每步都讀寫 HBM）	融合
LayerNorm	讀 $z \ 512$ + 寫 $z_ln \ 512$	—
5 個 linear	$5 \times (\text{讀 } 512 + \text{寫 } 512) = 5,120$	—
兩個 gating	$2 \times (\text{讀 } 1,024 + \text{寫 } 512) = 3,072$	—
Producer		讀 $z \ 512$ + 寫 $a、b$ 的 INT8 $264 = 776$
einsum	讀 $a、b \ 1,024$ + 寫 $512 = 1,536$	讀 $a、b \ 264$
LayerNorm_out + linear_z	$2 \times 1,024 = 2,048$	—
$\times g$	讀 $1,024$ + 寫 $512 = 1,536$	讀 $z \ 512$ + 寫輸出 512
合計	約 $14,300 \text{ B}/L^2$	約 $2,060 \text{ B}/L^2$ (約 1/7)

這是理想化的估算（einsum 的重複讀取已經在 3.1 節的 tiling 分析過，這裡先不計），目的是說明量級：融合讓搬運量少了將近一個數量級。

用數字看效果：同時存在的資料量（容量）

配置	每個 L^2 的 bytes	U55C 16 GB 可處理的最大 L
典型實作（約 7 份 FP32）	3,584	約 2,190
融合 + INT8 中間值	1,288 (z 輸入、輸出各 512， $a、b$ 各 132)	約 3,650 (約 1.67 倍)

公式見 9.4 節。以 LightNobel 的 FP16 為基準重算，提升約 1.52 倍（9.4 節）。

權重小、activation 大

項目	大小
5 個 128×128 的 linear 權重 (FP32)	$5 \times 64 \text{ KB} = 320 \text{ KB}$
LayerNorm 參數	幾 KB
U55C 晶片內記憶體	約 43 MB
一份 pair 張量 ($L = 2000$)	2 GB

權重可以一直放在晶片內，只有 activation 在流動。這和大型語言模型相反（語言模型是權重大、activation 相對小）。

FPGA 的優勢

- 可以把 LayerNorm $\rightarrow$ linear $\rightarrow$ gating $\rightarrow$ 量化做成一條管線，資料一路流過去，不需要中間的 buffer 寫回。
- 每一級的平行度可以各自調整，讓整條管線速度平衡。
- GPU 做融合要手寫 CUDA，而且受限於 shared memory 的大小與 thread 的組織方式；FPGA 則是直接設計資料路徑。

硬體成本估算 (producer)

每個 token 要做 4 個 128×128 的 linear (a_p 、 a_g 、 b_p 、 b_g) = $4 \times 16,384 = 65,536$ 次乘加。如果要每 T 個 cycle 處理一個 token，需要 $65,536 \div T$ 個乘加器：

每個 token 的 cycle 數 T	需要的乘加器	FP32 約需 DSP (每個約 5)
1,024	64	約 320
256	256	約 1,280
128	512	約 2,560

U55C 有 9,024 個 DSP。這個表就是設計空間探索的一個維度：用多少資源換多少速度。

10.4 硬體細節：LayerNorm、量化在源頭完成、該用幾個 bit

LayerNorm 在硬體上怎麼做

```
mean =  $\sum x_c / 128$ 
var =  $\sum (x_c - \text{mean})^2 / 128$ 
y_c =  $(x_c - \text{mean}) / \sqrt{\text{var} + \epsilon} \times \gamma_c + \beta_c$ 
```

- 一個 token 只有 128 個數，整個放在晶片內，所以可以掃兩次（第一次算平均、第二次算變異數），不會多花 HBM 流量。
- `1 / sqrt(...)` 每個 token 只算一次，成本可以忽略。
- （另一種做法是 Welford 演算法，只掃一次就能同時算出平均與變異數，數值也比較穩定。）

量化在源頭完成

（4.3 節已說明完整精度只需要在晶片內短暫存在，這裡是它在 producer 中的具體做法。）per-token 量化要知道「這個 token 128 個數的最大絕對值」。在 producer 裡，a 的 128 個數剛算完、還在暫存器裡，順便取最大值、算 scale、轉成 INT8，不需要額外再掃一次資料。

對比：如果量化是獨立的步驟，要先把 FP32 的 a 寫出去、再讀回來量化，等於多搬了 2 份資料。「在資料產生的地方就壓縮」是融合設計的另一個好處。這正是 LightNobel 的 VVPU 做的「執行時量化」（runtime quantization, Sec. 5.3）；它還多做了 top-k 挑 outlier。

該用幾個 bit？論文給的線索

選位元數的一般原則見 4.5 節。依 LightNobel 的分組，a、b 是 linear 之後的值，屬於 C 組：平均絕對值約 3.85、平均 outlier 不到 1 個，論文用 INT4、不處理 outlier 就能維持 TM-score。所以：

- 我們的 INT8 是保守的起點，誤差應該比論文的設定小。
- INT4 per-token 是自然的下一步：a、b 再小一半，容量模型見 9.4 節。
- 真正需要 outlier 處理的是 A 組（接 residual 的 z 本身），那是完整模型才需要處理的部分。

先驗證，再實作：兩步驟

第一步：用真實資料確認假設（第 1 週，`python/dump_trimul_inputs.py`）。輸出的 JSON 裡有兩組數字：

欄位	內容	要看什麼
<code>paper_group_C_check</code>	a、b 每個 token 的平均絕對值、超過 3σ 的值平均有幾個	是否接近論文 C 組（3.85、0.64 個）；如果 outlier 明顯更多，a、b 可能需要 outlier 處理
<code>rel_l2_grid</code>	INT8／INT4 × per-token／per-channel ／per-tensor 的 einsum 誤差	INT4 per-token 的誤差是否可接受；per-token 是否真的比 per-channel 好

這張表也能回答兩個取舍：per-token 是否真的比 per-channel 準（4.2 節，論文的觀察）；以及值不值得為了硬體省事改用 per-channel（per-channel 的尺在 einsum 裡可以最後乘一次，見 4.4 節）。

第二步：v5 kernel（第 2 週）。從 v4 改：

- 一個 512-bit word 剛好放 128 個 INT4 = 一個完整的 token（v4 要 2 個 word）。
- 每個 byte 拆成兩個 4-bit 值（要做 sign extension），乘加與 scale 的處理和 v4 相同。
- 預期：讀取量減半、算術強度約加倍；代價是解包的邏輯（LUT）。

如果真實資料顯示 INT4 誤差太大，v5 仍然合成（看硬體代價），結論改成「這個運算需要 INT8」，這同樣是有價值的結果。

10.5 海報、報告講法與評審問答

每個成果對應海報的哪一塊

海報區塊	內容	來源
動機	記憶體 vs L、約 7 份 pair 張量、執行時間分布（引用論文 Fig. 3）	1.4、3.5、9.4
關鍵觀察	memory-bound (roofline)、真實 a、b 和論文 C 組的對照	第 2 章、10.4
架構	逐步優化的 kernel + 融合資料流設計圖	3.5、10.3、10.4
逐步優化	v0 ~ v5 加速倍數與時間拆解	hls-all、v5
DSE	tile sweep、Pareto、模型誤差	9.2、9.3
精度	INT8／INT4 × per-token／channel／tensor 誤差表	10.4（真實資料）、csim

海報區塊	內容	來源
序列長度	最大 L 的比較 (FP16 基準)	9.4

評審可能問的問題

問題	回答方向
為什麼用 FPGA，不用 GPU？	可以自訂資料流與記憶體階層，適合拆解每一招；LightNobel 也指出 GPU 做低位元 activation 量化效率很差 (W4A4 比 FP16 慢)，需要客製硬體，而 FPGA 不用流片就能做到。
HLS 的估計準嗎？	HLS 是 cycle 級的排程結果；我們用它校正分析模型，並用不同設定交叉驗證（附誤差表）。上板是下一步。
量化會不會影響預測結構的準確度？	我們在真實 ESMFold 的 a、b 上量 einsum 輸出的誤差 (10.4)；對結構準確度 (TM-score) 的影響，論文的設定已驗證變化 < 0.001，我們沒有跑完整模型，列為未來工作。
和 LightNobel 差在哪？	LightNobel 是 ASIC (模擬評估) + 三組混合精度量化 + 管線化資料流；我們沿用它的原則，在現成 FPGA 上實作其中一個運算，並把每一招的貢獻拆開量化（對照表見 10.1 節）。不要說融合是我們提出的。
為什麼只做 Triangle Multiplication？	誠實回答：長序列時最花時間的是 Triangle Attention (論文 Fig. 3 : 75.9%)。Triangle Multiplication 結構最單純 (沒有 softmax)，適當第一個目標；tiling、量化、分析模型都能沿用到 Triangle Attention (未來工作)。
1.5 倍看起來不多？	L 和記憶體是平方根關係；以 FP16 為基準，融合 + INT8 讓 L 提升約 1.52 倍。論文的 120 倍主要來自不存 score matrix (和不分塊的 GPU 比)，量化本身約 1.65 倍，量級一致。
每個 token 多存一把尺，不是讓資料更多嗎？	尺只佔 token 的 1~3% (128 個數才一個浮點數)，但每個數從 16 bit 變 8 或 4 bit，整體約變成 1/2~1/4 (4.3 節的表)。
記憶體問題真的解決了嗎？	把上限往後推，沒有消除 L^2 成長。量化省常數倍；不存中間值 (尤其 L^3 的 score matrix) 才是大幅延長序列的關鍵 (5.4 節)。
為什麼 a、b 用 INT4 (或為什麼還是 INT8)？	論文把這類 activation (C 組) 用 INT4；我們先用真實資料確認 a、b 的分布接近 C 組，再做 v5。依結果回答「INT4 可行」或「這個運算需要 INT8」。
你們的貢獻到底是什麼？	① 在現成 FPGA 上實作並拆解每一招的效果與代價 (論文沒有)；② 量化設定有論文依據、有真實資料驗證；③ 經過驗證的分析模型與 DSE。融合與量化的原則來自 LightNobel。

報告時怎麼講

30 秒版

蛋白質結構預測模型裡有一份 $L \times L \times 128$ 的資料，序列越長越放不下。LightNobel 用專用晶片加上 token-wise 量化和管線化資料流解決這個問題，但只有模擬結果。我們在現成的 FPGA 上實作其中的 Triangle Multiplication，逐步拆解每一種記憶體優化手法各自的貢獻，並用分析模型找出最佳設計。

3 分鐘版 (投影片順序)

1. 問題：pair representation 隨 L^2 成長，Triangle Multiplication 一步就有約 7 份；長序列時 Triangle Attention 更花時間 (第 1、2 章)
 2. 為什麼難：memory-bound，roofline 圖 (2.5 節)
 3. LightNobel 的解法與我們的定位 (第 5 章、10.1 節)
 4. 我們的做法：逐步優化的 kernel、依論文選的位元數 (真實資料驗證)、融合資料流設計 (10.3、10.4 節)
 5. 結果：v0~v5 的逐步改善、DSE、可處理的序列長度 (FP16 基準) (第 8、9 章)
 6. 結論與未來工作：Triangle Attention、上板、完整的融合 kernel
-

10.6 參考資料

- LightNobel：[arXiv 2505.05893](https://arxiv.org/abs/2505.05893) (本章引用：Fig. 3 執行時間分布、Table 1 記憶體、Sec. 5 硬體管線、Sec. 6 評估方法、Fig. 11 量化設定)
 - GPU 上的融合 Triangle Multiplication (工作空間約 7U → 約 4U，分塊後約 2.2U)：[openfold-3 PR #318](#)
 - 長序列推論的分塊：[AutoChunk](#)、[FastFold](#)
-

本章小結

- 題目：在 FPGA 上實作 Triangle Multiplication，逐步拆解每一招的效果與代價。
 - 誠實的定位：融合與執行時量化來自 LightNobel；長序列時最花時間的是 Triangle Attention (未來工作)；容量用 FP16 基準報告。
 - 三根支柱：逐步拆解 (v0~v5)、以論文為依據的設計、分析模型與 DSE。
 - 下一步：hls-all 與 tile sweep 校正模型 → 真實資料驗證 a、b 屬 C 組 → v5 (INT4) → 海報。
-

自我檢測

讀完這一章，試著回答下面的問題 (參考答案在附錄 C)：

1. 本專題哪些部分繼承自論文？哪些是新增的？
 2. 為什麼不應該說「我們比 LightNobel 好」？比較好的說法是什麼？
 3. 做 v5 (INT4) 之前，為什麼要先用真實資料檢查 a、b？要看哪兩組數字？
-

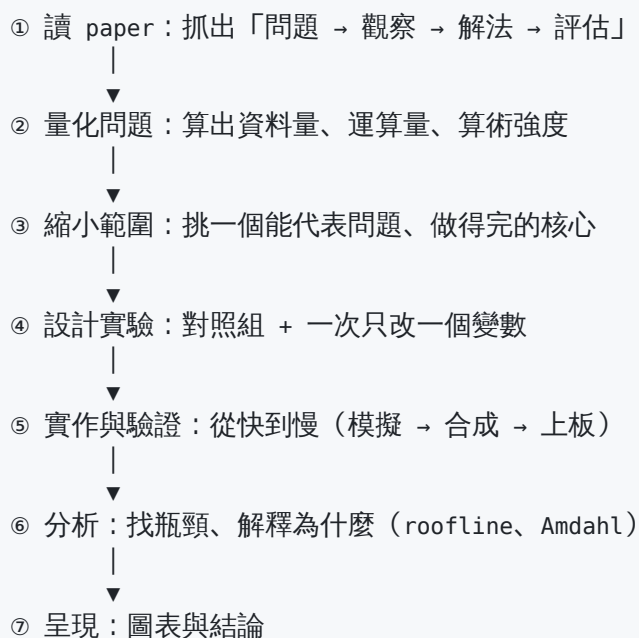
下一章把整個過程整理成可以重複使用的研究方法，之後讀其他 paper 時也能照著做。

第 11 章 研究方法：之後可以重複使用的步驟

本章重點

- 把這次「從一篇 paper 到可執行的實驗」的過程，整理成可以套用到任何架構／加速器 paper 的流程。
- 每一步都附上 checklist，以及本專題的實際做法當範例。
- 你之後的研究會從 LightNobel 延伸，這一章就是每次讀新 paper 時的操作手冊。

11.1 整體流程



11.2 步驟一：讀 paper 的方法

四個問題（每篇都要能回答）

問題	要找的內容	LightNobel 的答案
問題是什麼？	什麼東西不夠好？具體的限制	PPM 無法處理長序列，因為 activation 記憶體隨 L^2 成長
他們觀察到什麼？	讓解法成立的關鍵事實（通常有分析圖）	長序列時時間集中在 pair representation（尤其 Triangle Attention）；瓶頸是 activation 不是權重；channel 之間差異小、token 之間差異大；不同位置的 activation 特性不同
解法是什麼？	演算法 + 硬體	Token-wise 自適應量化 + 專用加速器

問題	要找的內容	LightNobel 的答案
怎麼證明有效？	比較對象、指標、資料集	與 GPU 基準比較記憶體、速度、準確度、可支援的序列長度

閱讀順序（不要從頭讀到尾）

1. 標題、摘要：一句話說出問題與解法。
2. Introduction 的最後一兩段：通常會列出貢獻（contributions）。
3. 動機（Motivation）章節的圖：這裡是「觀察」，最重要。
4. 評估（Evaluation）的主要圖表：看比較對象和提升幅度。
5. 方法章節：前面都懂了，再看細節。
6. Related work：找出這個領域還有哪些相關論文，建立文獻地圖。

讀完要寫下的筆記模板

（第 5 章 5.1 節的速讀卡就是用這個模板整理 LightNobel 的完整範例。）

論文：
 一句話摘要：
 問題（附具體數字）：
 關鍵觀察（附圖號）：
 解法：
 - 演算法層面：
 - 硬體層面：
 評估：
 - 比較對象：
 - 主要指標與結果：
 - 資料集／模型：
 我可以重現／延伸的部分：
 我不確定的地方（要回原文確認）：

11.3 步驟二：量化問題

任何「效能」問題，都先算這三個數字：

數字	怎麼算	本專題的例子
資料量	每個張量的形狀 × 每個元素的 bytes	pair tensor = $L^2 \times 128 \times 4$ bytes ($L = 1000$ 時 512 MB)
運算量	數迴圈：每個輸出要幾次乘加	$2 \times L^3 \times C$ FLOP ($L = 1000$ 時約 2.6×10^{11})
算術強度	運算量 ÷ 實際搬運量	直白寫法 0.25，tiling 後 2~8 FLOP/byte

然後和硬體比：

硬體平衡點 = 峰值運算能力 ÷ 記憶體頻寬

算術強度 < 平衡點 → memory-bound → 優先減少搬運

算術強度 > 平衡點 → compute-bound → 優先增加運算單元

硬體平衡點 = 峰值運算能力 ÷ 記憶體頻寬

算術強度 < 平衡點 → memory-bound → 優先減少搬運

算術強度 > 平衡點 → compute-bound → 優先增加運算單元

Checklist

- ### Checklist
- [] 找出最大的張量是哪一個，它隨什麼參數成長 (L ? L^2 ? L^3 ?)
 - [] 找出運算量最大的運算
 - [] 計算不同規模 ($L = 256, 1000, 4000$) 下的數字，做成表格
 - [] 判斷是記憶體容量、記憶體頻寬還是運算能力的問題

11.4 步驟三：縮小範圍

好的範圍要滿足

條件	為什麼	本專題
能代表 paper 的問題	結論才有意義	Triangle Multiplication 直接處理 L^2 的 pair tensor
算式清楚、容易驗證	能確認做對了	本質是矩陣乘法，可以用 CPU 算正確答案
能和原始模型對上	證明不是自己亂編的	用 ESMFold 驗證算式差值 0
大小做得完	時間有限	只做 einsum，不做 LayerNorm/gating
能展示多種手法	才有比較	五種記憶體手法都適用

常見的縮小方式

- ## 常見的縮小方式
- 只做一個運算（本專題的做法）
 - 只做一種資料精度
 - 固定部分參數（例如固定 $C = 128$ ，只讓 L 變動）
 - 用模擬資料代替真實資料，但要說明模擬資料的特性為什麼合理（例如我們加了 1% outlier）

11.5 步驟四：設計實驗

原則一：一定要有對照組 (baseline)

沒有 baseline，「快了 3 倍」就沒有意義。本專題的 v0 就是 baseline：最直白、沒有任何優化。

原則二：一次只改一個變數

```

v0 —(+ tiling)→ v1 —(+ 寬位元)→ v2 —(+ double buffering)→ v3
                        ↳(+ 量化)→ v4

```

每一版只多一招，才能說「這個改善來自這一招」。v4 是從 v2 分出來的，不是從 v3。這樣 v4 和 v2 的差異就只有量化。

原則三：掃描參數，不要只看一個點

掃描	看什麼
Tile 大小 (4、8、16、32)	搬運量 vs 晶片內記憶體 trade-off
序列長度 L (128 ~ 2048)	時間是否符合 L^3 、記憶體是否符合 L^2
HBM 設定 (多通道 vs 單一)	頻寬的影響

原則四：同時量「好處」和「代價」

好處	代價
時間、GFLOP/s、頻寬	硬體資源 (BRAM、DSP、LUT、FF)
可支援的最大 L	精度損失 (rel_L2)
記憶體佔用	程式複雜度

Checklist

- [] 定義 baseline
- [] 每個版本只改一件事，並寫下「預期會改善什麼、為什麼」
- [] 先預測再量測 (例如：v3 預測快約 21%)
- [] 列出要掃描的參數
- [] 列出要記錄的指標 (好處 + 代價)

11.6 步驟五：實作與驗證

從快到慢

階段	時間	驗證什麼	失敗了怎麼辦
C 模擬	秒	算得對不對	修演算法
HLS 合成	分鐘	能不能變成電路、速度和資源估計	修 pragma、改寫法
C/RTL 協同模擬	分鐘 ~ 小時	電路和 C++ 一致嗎	檢查 DEPENDENCE 等宣告
硬體模擬	小時	整個系統的介面	修 host 或設定
上板	1 ~ 數小時編譯	真實速度	修時序、記憶體設定

前一步沒過，不進下一步。

驗證正確性的技巧

1. 用高精度 (double) 算參考答案。
2. 用相對誤差 (rel_l2) 而不是絕對誤差，因為數值大小不一。
3. 測試多種大小與設定：我們測了 $L = 16, 32, 48, 64$ ，以及 $8 \times 8, 16 \times 4, 32 \times 32$ 的 tile。
4. 刻意加入困難的情況：outlier 就是為了讓量化的差異顯現出來。
5. 和原始模型對照：ESMFold 的「差值 0」驗證。

11.7 步驟六：分析

找瓶頸的方法

1. 看報告中每個迴圈的 cycle 數，找出佔最多時間的部分。
2. 算出各部分的比例（例： $v2$ 中載入佔 $141 / 671 \approx 21\%$ ）。
3. 用 Amdahl's Law 算出改善那一部分的上限。
4. 在 roofline 圖上標出位置，判斷是 memory-bound 還是 compute-bound。

解釋結果時要回答

- 結果和預測一致嗎？不一致的話，為什麼？
- 瓶頸轉移到哪裡了？
- 付出的代價（資源、精度）值得嗎？
- 在真實情況下（更大的 L 、真實資料）結論還成立嗎？

11.8 步驟七：呈現（報告要畫的圖）

圖	x 軸	y 軸	要傳達的訊息
動機圖	L	記憶體 (GB)	記憶體隨 L^2 爆炸 (問題的嚴重性)
各版本比較	$v0 \sim v4$	時間 (對數軸) 或加速倍數	每一招的效果
Roofline	算術強度 (對數)	GFLOP/s (對數)	每一招讓 kernel 往哪裡移動
時間拆解	版本	cycle (堆疊長條：載入／計算／其他)	瓶頸在哪、怎麼轉移
Tile sweep	tile 大小	時間 + BRAM 用量 (雙 y 軸)	trade-off
量化精度	量化方式	rel_l2	per-token vs per-tensor
資源用量	版本	BRAM／DSP／LUT 百分比	代價

11.9 除錯的思路（以 2025.2 路徑 bug 為例）

1. 讀完整的錯誤訊息（包括 WARNING）
↓ 「Cannot find source file ../kernels/trimul_v2.cpp」

2. 比較預期和實際
↓ 預期是 kernels/trimul_v2.cpp，實際變成 ../kernels/...

3. 提出假設並修正
↓ 假設：相對路徑的問題 → 改成絕對路徑

4. 修正無效 → 假設錯了，找新線索
↓ 日誌顯示加入時是絕對路徑，但合成時又被改寫

5. 用算的驗證新假設
↓ 專案在下兩層，但工具只往上一層 → 工具假設專案只在下一層

6. 針對根因修正
↓ 從 hls_prj 裡面建立專案 → 成功

回報問題時的好習慣（對學長姐、助教或 Claude 都適用）

- 附上完整的指令和錯誤前後的 20 ~ 30 行輸出。
- 說明預期會發生什麼。
- 說明已經試過什麼。
- 附上環境資訊（工具版本、作業系統）。

11.10 本專題套用這個流程的完整範例

步驟	本專題的做法	產出
① 讀 paper	LightNobel：長序列受限於 activation 記憶體；token-wise 量化	第 1、2、5 章
② 量化問題	pair tensor $L^2 \times 512 \text{ B}$ ；AI = 0.25	第 1、2 章的表格
③ 縮小範圍	只做 Triangle Multiplication 的 einsum	trimul.h
④ 設計實驗	v0 baseline + 四種手法；tile sweep；HBM 對照；per-token vs per-tensor	5 個 kernel、cfg
⑤ 實作與驗證	C 模擬全部通過 → v2 合成成功 → 其餘進行中（截至 2026-09-26）	make csim、make hls
⑥ 分析	v2 載入佔 21% → 預測 v3 快約 21%	第 3 章 3.3 節
⑦ 呈現	待完成全部數據後製作	hls_summary.csv → 圖表
（回頭）讀原文核對	發現三個誤判，修正定位與策略（11.11 節）	第 5、10 章修訂

11.11 案例：讀原文後發現的三個誤判

本專題一開始無法下載 LightNobel 原文，對論文的理解來自摘要和搜尋結果。拿到 PDF 讀完後，發現三個誤判：

誤判	原本以為	原文（出處）	影響
① 哪個運算最花時間	Triangle Multiplication 是最核心、最吃資源的運算	長序列時 Triangle Attention 佔 75.9%，Triangle Multiplication 只佔 14.5% (Fig. 3)	選題理由改成「結構最單純的第一個目標」，Triangle Attention 列為未來工作
② 我們的新意在哪	融合是和 LightNobel 「互補」的新角度	RMPU → VVPU 的管線與執行時量化已經做到中間值不落地 (Sec. 5)	貢獻改成「FPGA 實作 + 拆解每一招 + 分析模型」
③ 比較基準	以 FP32 計算量化的效果	基準是 FP16 (Table 1)	容量提升從 1.67 倍改成 1.52 倍

另外還有一個新發現：a、b 屬於論文的 C 組（INT4、不處理 outlier），所以 INT4 的 v5 變成有依據的下一步。

學到的事

1. 二手資料只能當線索：摘要、新聞稿、別人的整理都會省略細節。做決策前一定要讀原文，至少讀動機的圖、方法的表格和評估設定。
2. 特別檢查三種東西：
 - 瓶頸的數據（哪個運算、在什麼條件下量的：GPU 型號、序列長度、有沒有分塊）；
 - 比較基準（baseline 用什麼精度、什麼設定）；
 - 論文已經做了什麼（避免把別人做過的事當成自己的新意）。
3. 誤判不可怕，要留下紀錄：把「原本以為 → 原文 → 修正」寫下來（docs/PROJECT_LOG.md 8.1 節），之後接手的人才知道為什麼策略是現在這樣。
4. 修正後要通讀一次：一個前提改了，會影響很多章節的說法。只改有問題的句子，其他段落的邏輯可能就接不上。

本章小結

- 讀 paper：問題 → 觀察 → 解法 → 評估，先看動機的圖。
- 量化：資料量、運算量、算術強度，和硬體平衡點比較。
- 實驗：有 baseline、一次改一件事、先預測再量測、同時記錄好處和代價。
- 驗證：從快到慢，前一步沒過不進下一步。
- 分析：找出比例、用 Amdahl 算上限、在 roofline 上看位置。
- 核對原文：二手資料只當線索；特別檢查瓶頸數據、比較基準、論文已經做了什麼（11.11）。

自我檢測

讀完這一章，試著回答下面的問題（參考答案在附錄 C）：

1. 讀一篇架構 paper 時，一定要能回答哪四個問題？
2. 為什麼 v_4 是從 v_2 改出來，而不是從 v_3 ？
3. 設計實驗時，為什麼要「先預測、再量測」？
4. 修正之後錯誤訊息完全沒變，代表什麼？下一步該怎麼做？
5. 讀一篇論文的原文時，哪三種東西特別容易在二手資料中被誤解？本專題各犯了哪一個？

附錄 A 是指令手冊；附錄 B 是術語表；附錄 C 是各章自我檢測的參考答案。

附錄 B 指令手冊：做過的、可以做的、格式與用途

本章重點

- 先學會看懂指令的格式（指令、選項、參數、管線、導向），之後遇到新指令也能自己拆解。
- 依用途分成九類：Linux 基本操作、環境調查、Git、tmux、本專案的 make 指令、Vitis 工具、看結果、host 程式、Python 腳本。
- 每個指令都標示： 已經做過、 下一步會用到、 之後可以做、 小心使用。
- 本附錄最後有一張「目前為止實際執行過的指令」時間軸，以及常見錯誤的排查方法。
- 圖示的狀態以 2026-09-26 為準，最新進度見 docs/PROJECT_LOG.md。

A.1 先看懂指令的格式

基本結構

指令名稱	[選項]	[參數]
▼	▼	▼
ls	-la	/opt/xilinx
git	clone -b 分支	網址
make	hls	KERNEL=trimul_v2

部分	意思	例子
指令名稱	要執行的程式	ls、git、make
短選項	一個 - 加一個字母，可以合併	-l、-a，合併成 -la
長選項	兩個 - 加單字	--help、--mode hls
參數	指令要處理的對象	檔名、資料夾、網址
子指令	有些指令底下還有功能分類	git clone、git pull 的 clone、pull

組合指令的符號

符號	意思	例子	白話
\	管線：前一個的輸出當成後一個的輸入	ls pci \ grep -i xilinx	列出所有裝置，只留下含 xilinx 的行
>	把輸出寫進檔案（覆蓋）	make summary > out.txt	結果存檔

符號	意思	例子	白話
>>	把輸出附加到檔案結尾	<code>... \ grep ^CSV >> results.csv</code>	累積多次結果
2>/dev/null	把錯誤訊息丟掉	<code>ls /opt/xilinx 2>/dev/null</code>	資料夾不存在也不顯示錯誤
2>&1	錯誤訊息也當成一般輸出	<code>make hls > log.txt 2>&1</code>	錯誤也一起存進檔案
&&	前一個成功才執行下一個	<code>git pull && make csim</code>	更新成功才測試
\	前一個失敗才執行下一個	<code>make hls \ echo 失敗</code>	
;	不管成功與否，依序執行	<code>cd ~; ls</code>	
\	行尾的反斜線：指令太長時接到下一行		
#	後面是註解，不會執行	<code>make csim # 看到 ALL PASS</code>	手冊裡的說明文字
~	你的家目錄	<code>cd ~</code>	<code>/home/anthony</code>
. / ..	目前資料夾 / 上一層	<code>cd ..</code>	
*	萬用字元	<code>ls kernels/*.cpp</code>	所有 .cpp 檔
\$變數	取出變數的值	<code>echo \$XILINX_XRT</code>	

變數的兩種用法

```

KERNEL=trimul_v2 vitis-run ...    # ① 寫在指令前面：只對這一個指令有效（環境變數）
make hls KERNEL=trimul_v2        # ② 寫在 make 後面：傳給 Makefile 的變數

```

常用按鍵

按鍵	用途
Tab	自動補完檔名或指令（按兩下列出所有可能）
↑ / ↓	叫出之前輸入過的指令
Ctrl + C	中斷正在執行的程式（跑錯了、卡住了都用這個）
Ctrl + R	搜尋歷史指令
Ctrl + L	清除畫面
q	離開 less、man 這類翻頁畫面

忘記怎麼用時

```

指令 --help    # 大部分指令都有簡短說明
man 指令       # 完整手冊（按 q 離開）
make help      # 本專案的指令一覽

```

A.2 Linux 基本操作

指令	格式	用途	例子	狀態
ssh	ssh 帳號@主機	登入遠端 server	ssh anthony@scd-lab-server63	✓
pwd	pwd	顯示目前所在的資料夾		☐
ls	ls [-la] [路徑]	列出檔案； -l 詳細、 -a 含隱藏檔	ls /tools/Xilinx	✓
cd	cd 路徑	切換資料夾	cd ForClaude/fpga/trimul	✓
cat	cat 檔案	印出整個檔案	cat cfg/trimul_v2.cfg	☐
head	head -n 行數 檔案	印出前幾行	head -n 60 xxx.rpt	✓
tail	tail -n 行數 檔案	印出最後幾行 (看錯誤訊息最常用)	tail -n 30 hls_trimul_v4.log	🔗
tail -f	tail -f 檔案	持續顯示檔案新增的內容 (看進度)	tail -f hls_trimul_v3.log	☐
less	less 檔案	翻頁閱讀長檔案 (↑↓ 翻、 / 搜尋、 q 離開)	less xxx_csynth.rpt	☐
grep	grep [-i] 關鍵字 檔案	找出含關鍵字的行； -i 不分大小寫； -n 顯示行號	grep -n ERROR hls_trimul_v4.log	✓
find	find 路徑 -name "樣式"	找檔案	find /tools -iname "*u55c*"	✓
which	which 指令	這個指令裝在哪裡 (找不到代表沒裝或沒載入)	which v++	✓
echo	echo 文字或\$變數	印出文字或變數	echo \$PATH	☐
mkdir	mkdir -p 資料夾	建立資料夾	mkdir -p ~/results	☐
cp / mv	cp 來源 目的	複製 / 移動 (或改名)	cp hls_summary.csv ~/results/	☐
rm	rm [-r] 檔案	⚠ 刪除，無法復原； -r 刪資料夾	rm -f host.exe	⚠
df -h	df -h	硬碟剩多少空間		☐
du -sh	du -sh 資料夾	某個資料夾佔多少空間 (hls_prj/ 會越來越大)	du -sh hls_prj	☐
top / htop	top	看 CPU、記憶體用量與正在跑的程式 (q 離開)	確認合成是否還在跑	☐
history	history	列出輸入過的指令	history \ grep make	☐

指令	格式	用途	例子	狀態
<code>source</code>	<code>source</code> 檔案	在目前的終端機執行設定檔（載入環境變數）	見 A.3	

⚠ `rm -rf` 會直接刪除整個資料夾，不會經過資源回收筒。刪除前先用 `ls` 確認內容；想清掉本專案產生的檔案，改用 `make clean` 比較安全。

A.3 環境調查與設定

指令	用途	結果 / 狀態
<code>lspci \ grep -i -E "xilinx\ amd\ altera"</code>	列出 PCIe 裝置，找 FPGA 卡	✅ 空的
<code>lspci -d 10ee:</code>	只列 Xilinx（廠商代號 10ee）的裝置	✅ 空的 → server63 沒有卡
<code>which vivado vitis v++ vitis-run</code>	確認工具有沒有裝	✅ 都在 <code>/tools/Xilinx/2025.2/</code>
<code>which vitis_hls</code>		✅ 找不到（2025.2 已移除）
<code>ls /tools/Xilinx</code>	看裝了哪些版本	✅ 2025.2
<code>xbutil examine</code>	列出 FPGA 卡、shell、狀態（需要 XRT）	✅ command not found
<code>ls /opt/xilinx/platforms</code>	找 U55C platform	✅ 沒有
<code>find /opt /tools -maxdepth 4 -iname "*u55c*"</code>	找 U55C 相關檔案	✅ 沒有
<code>module avail</code>	有些 server 用 module 管理工具版本	✅ 沒有相關結果
<code>nvidia-smi</code>	看有沒有 NVIDIA GPU	✅ 沒有 GPU
<code>find /tools/Xilinx/2025.2 -maxdepth 6 -type d -iname "virtexuplus*"</code>	確認 U55C 晶片型號資料有裝	❌ (HLS 報告顯示 <code>virtexuplusHBM</code> ，代表有裝)

載入環境（找到有 U55C 的機器後會用到）

```
source /tools/Xilinx/2025.2/Vitis/settings64.sh # 載入 Vitis (server63 看起來已自動載入)
source /opt/xilinx/xrt/setup.sh                # 載入 XRT (有卡的機器才有)
```

- 每次登入都要執行，或寫進 `~/.bashrc` 讓它自動執行。
- 載入後 `echo $XILINX_XRT` 會顯示路徑，`make host` 需要這個變數。

在有 U55C 的機器上要跑的指令 ()

```
lspci -d 10ee:           # 應該看到兩行 (U55C 有兩個 PCIe function)
xbutil examine           # 列出卡片與 BDF (例如 0000:3b:00.1)
xbutil examine -d <BDF>  # 看這張卡目前的 shell (platform) 名稱
xbutil validate -d <BDF> # 板子自我檢測 (約數分鐘)
ls /opt/xilinx/platforms # platform 資料夾名稱 = Makefile 的 PLATFORM
```

A.4 Git : 取得與更新程式碼

指令	格式	用途	狀態
git clone	git clone -b 分支 網址	第一次下載整個專案	✓
git pull	git pull	下載 Claude 推上去的最新修改 (最常用)	✓
git status	git status	看有沒有被你改過的檔案	□
git log --oneline	git log --oneline \ head	看最近的修改紀錄	□
git diff	git diff [檔案]	看你改了哪些地方	□
git branch	git branch	看目前在哪個分支	□
git checkout -- 檔案		⚠ 放棄你對某個檔案的修改，還原成最新版	⚠

本專案的 clone 指令：

```
git clone -b claude/lightnobel-memory-optimization-yhokyc
https://github.com/antony3911/ForClaude.git
```

git pull 失敗時

如果出現 Your local changes would be overwritten，代表你在 server 上改過某個檔案：

```
git status      # 看是哪個檔案
git stash       # 先把你的修改暫存起來
git pull        # 再更新
git stash pop    # 需要的話把修改拿回來 (可能要手動合併)
```

⚠ 不要在 server 上執行 **git push**。程式碼的修改由 Claude 推上 GitHub，server 只負責下載與執行。

A.5 tmux : 斷線也不會中斷的終端機

長時間的工作 (make hls-all、上板編譯) 一定要在 tmux 裡跑。

操作	指令 / 按鍵	狀態
開新的 session	<code>tmux</code> 或 <code>tmux new -s hls</code> (取名 hls)	✓
暫時離開 (程式繼續跑)	<code>Ctrl + B</code> , 放開, 再按 <code>D</code>	✓
列出所有 session	<code>tmux ls</code>	
回到 session	<code>tmux attach</code> 或 <code>tmux attach -t hls</code>	
在 tmux 裡往上捲動	<code>Ctrl + B</code> , 放開, 再按 <code>[</code> , 用方向鍵捲; 按 <code>q</code> 離開	
結束 session	在裡面輸入 <code>exit</code>	

A.6 本專案的 make 指令 (最重要)

所有 make 指令都要在 **ForClaude/fpga/trimul** 資料夾裡執行。

格式

```
make 目標 [變數=值 變數=值 ...]
|      |      |
|      |      └─ 調整參數 (不給就用預設值)
|      └─ 要做哪件事 (csim、hls、run ...)
└─ 讀取目前資料夾的 Makefile
```

加上 `-n` 可以只顯示會執行哪些指令、但不真的執行, 用來檢查: `make -n hls KERNEL=trimul_v3`

目標一覽

目標	做什麼	需要	時間	產出	狀態
<code>make help</code>	列出所有指令	—	瞬間	—	
<code>make csim</code>	g++ 編譯 testbench, 5 個版本和正確答案比對	g++	秒	畫面上的 PASS/FAIL	✓
<code>make hls</code> <code>KERNEL=...</code>	合成一個版本, 產生 cycle 數與資源報告	Vitis	數分鐘 ~ 數十分鐘	<code>hls_prj/<kernel>/sol/syn/report/</code>	✓ (v2)

目標	做什麼	需要	時間	產出	狀態
<code>make hls-all</code>	依序合成 v0~v4，最後自動 <code>make summary</code>	Vitis	數十分鐘以上	<code>hls_<kernel>.log</code> 、 <code>hls_summary.csv</code>	● 進行中
<code>make summary</code>	讀取所有報告，印出比較表	Python 3	秒	<code>hls_summary.csv</code>	➡ soon
<code>make hls-sweep</code>	v2 的 tile 大小掃描 (4/8/16/32)，各自存成 <code>hls_prj/trimul_v2_t<N></code>	Vitis	數十分鐘	4 份報告	➡ soon
<code>make model</code>	分析模型：預測 cycle、資源、最大 L，並和 HLS 報告比對誤差	Python 3	秒	<code>model_dse.csv</code>	➡ soon
<code>make xo</code> <code>KERNEL=...</code> <code>TARGET=hw</code>	<code>v++ -c</code> ：把 kernel 編成 .xo	Vitis + platform	數分鐘	<code>build/hw/<tag>.xo</code>	■
<code>make xclbin</code> <code>KERNEL=...</code> <code>TARGET=hw</code>	<code>v++ -l</code> ：接上 platform 產生 bitstream	Vitis + platform	1~數小時	<code>build/hw/<tag>.xclbin</code>	■
<code>make host</code>	編譯 host 程式	XRT	秒	<code>host.exe</code>	■
<code>make run</code> ...	執行 (emulation 或上板)	XRT (上板需要卡)	視 L 而定	畫面上的時間、誤差、CSV 行	■
<code>make clean</code>	⚠ 刪除所有產生的檔案 (<code>build/</code> 、 <code>hls_prj/</code> 、 <code>log</code> 、 <code>csv</code>)	—	瞬間	—	⚠

⚠ `make clean` 會刪掉 `hls_summary.csv` 和所有 HLS 報告。清之前先把要保留的結果複製到別的地方：`mkdir -p ~/results && cp hls_summary.csv ~/results/`

可以調整的變數

變數	預設值	用途	例子
<code>KERNEL</code>	<code>trimul_v2</code>	選版本	<code>KERNEL=trimul_v4</code>

變數	預設值	用途	例子
TARGET	hw_emu	sw_emu (軟體模擬) / hw_emu (硬體模擬) / hw (上板)	TARGET=hw
PLATFORM	xilinx_u55c_gen3x16_xdma_3_202210_1	必須和卡上的 shell 一致	PLATFORM=<ls /opt/xilinx/platforms 看到的名稱 >
CFG	cfg/\$(KERNEL).cfg	HBM 接線設定	CFG=cfg/trimul_v2_onebank.cfg
DEFS	空	編譯時定義，目前用來改 tile 大小	DEFS="-DTILE_I=16 -DTILE_J=16"
TAG	同 KERNEL	輸出檔名，做不同實驗時避免覆蓋	TAG=v2_t16
RUN_L	16	make run 的序列長度	RUN_L=512
RUN_ARGS	空	傳給 host.exe 的額外參數	RUN_ARGS="3 tensor"
DATA	空	真實資料的資料夾 (設定後忽略 RUN_L)	DATA=../../data/spike
KERNELS	v0~v4	hls-all 要跑哪些版本	KERNELS="trimul_v3 trimul_v4"

常用組合（照實驗整理）

```
# — 功能驗證 —
make csim                                # ☒ 隨機資料, L=32
make csim DATA=../../data/hba          # ☐ 真實資料（有 GPU/Colab 產生資料後）

# — 實驗 A：五個版本比較（HLS 報告） —
make hls-all                             # ☒ 進行中
make hls-all KERNELS="trimul_v3 trimul_v4" # ☐ 只重跑失敗或新增的版本
make summary                             # ☒ 重新整理比較表

# — 實驗 B：tile 大小 —
make hls-sweep                           # ☒ 一次跑 4/8/16/32
make hls-sweep SWEEP_KERNEL=trimul_v4 TILES="8 16" # ☐ 換版本或尺寸
make model                               # ☒ 模型 vs 報告、設計空間探索

# — 上板（找到有 U55C 的機器後） —
make xclbin KERNEL=trimul_v2 TARGET=hw PLATFORM=<實際名稱> # ☐ 1~數小時
make run KERNEL=trimul_v2 TARGET=hw RUN_L=512             # ☐

# — 實驗 C：HBM 單一通道對照 —
make run KERNEL=trimul_v2 CFG=config/trimul_v2_onebank.cfg TAG=trimul_v2_onebank TARGET=hw
RUN_L=256 # ☐

# — 實驗 D：量化方式 —
make run KERNEL=trimul_v4 TARGET=hw RUN_L=512 # ☐ per-token（預設）
make run KERNEL=trimul_v4 TARGET=hw RUN_L=512 RUN_ARGS="3 tensor" # ☐ per-tensor

# — 實驗 E：序列長度掃描（收集成一個 CSV） —
for L in 128 256 512 1024; do
  make run KERNEL=trimul_v2 TARGET=hw RUN_L=$L | grep ^CSV >> results.csv
done # ☐
```

A.7 Vitis 工具的原始指令（Makefile 背後在做什麼）

平常用 make 就好。這一節讓你知道 make 實際呼叫了什麼，除錯時會用到。

make 目標	實際執行的指令	說明
make csim	g++ -std=c++17 -O2 ... tb/tb.cpp kernels/*.cpp -o csim.exe && ./csim.exe 32	一般 C++ 編譯
make hls	KERNEL=trimul_v2 DEFS="" vitis-run --mode hls --tcl hls/run_hls.tcl	☒ 2025.2 的 HLS 指令
make xo	v++ -c -t hw --platform <P> -k trimul_v2 -I kernels -o build/hw/trimul_v2.xo kernels/trimul_v2.cpp	編譯 kernel
make xclbin	v++ -l -t hw --platform <P> --config config/trimul_v2.cfg -o build/hw/trimul_v2.xclbin build/hw/trimul_v2.xo	連結 + 擺放繞 線
make host	g++ ... host/host.cpp -I\$XILINX_XRT/include -L\$XILINX_XRT/lib - lxrt_coreutil -pthread -o host.exe	編譯 host

make 目標	實際執行的指令	說明
make run (emulation)	emconfigutil --platform <P> --od . 然後 XCL_EMULATION_MODE=hw_emu ./host.exe ...	產生模擬設定 並執行

v++ 常用選項

選項	意思
-c / -l	compile (產生 .xo) / link (產生 .xclbin)
-t hw\ hw_emu\ sw_emu	目標
--platform	platform 名稱
-k	kernel 函式名稱
--config	設定檔 (HBM 接線等)
--save-temps --temp_dir	保留中間檔案，除錯時可以看 Vivado 的報告

run_hls.tcl 可用的環境變數

```

KERNEL=trimul_v3 CSIM=1 vitis-run --mode hls --tcl hls/run_hls.tcl    # 合成前先在 HLS 裡跑 C
模擬
KERNEL=trimul_v3 COSIM=1 vitis-run --mode hls --tcl hls/run_hls.tcl  # 合成後再跑 C/RTL 協同
模擬 (較慢)

```

也可以透過 make 傳：make hls KERNEL=trimul_v3 COSIM=1 (make 會把命令列變數傳給子程式)。

A.8 看結果

HLS 報告

```

# 報告位置
ls hls_prj/trimul_v2/sol/syn/report/

# 前 60 行: timing 與 latency 摘要 
head -n 60 hls_prj/trimul_v2/sol/syn/report/trimul_v2_csynth.rpt

# 資源用量那一段 
grep -n -A 20 "Utilization Estimates" hls_prj/trimul_v2/sol/syn/report/trimul_v2_csynth.rpt

# 只看各迴圈的 latency 與 II 
grep -n -A 15 "\* Loop:" hls_prj/trimul_v3/sol/syn/report/trimul_v3_csynth.rpt

```

make hls-all 的進度與錯誤

```
ls hls_*.log           # 哪些版本跑過了
tail -n 30 hls_trimul_v4.log # 看某一版最後的輸出（失敗時截圖這個）
grep -n "ERROR" hls_trimul_*.log # 一次找出所有錯誤
tail -f hls_trimul_v3.log  # 即時看正在跑的版本（Ctrl+C 離開）
```

比較表

```
make summary           # 印出表格並存成 hls_summary.csv
column -s, -t < hls_summary.csv # 在終端機把 CSV 排整齊
python3 scripts/hls_summary.py --L 256 # 直接呼叫腳本（--L 要和 L_TRIP 一致）
```

把結果帶回自己的電腦

在你自己的電腦（Windows 的 PowerShell 或命令提示字元）執行：

```
scp anthony@scd-lab-server63:~/ForClaude/fpga/trimul/hls_summary.csv C:\ForClaude\
```

scp 的格式是 scp 來源 目的，遠端路徑寫成 帳號@主機:路徑。

A.9 host 程式與 Python 腳本的參數

host.exe（上板用，）

```
./host.exe <xclbin> <kernel> <L> [reps=3] [token|tensor] [samples=2000] [data_dir]
```

參數	意思
xclbin	編譯好的電路檔
kernel	kernel 名稱（trimul_v2 等）
L	序列長度，必須是 tile 大小的倍數（給了 data_dir 就以檔案為準）
reps	重複執行幾次，取最快的一次
token / tensor	v4 的量化方式
samples	隨機抽幾個點驗證正確性
data_dir	真實資料的資料夾

輸出的最後一行是 CSV：

```
CSV,kernel,L,time_ms,gflops,est_traffic_GB,est_GBps,hbm_footprint_MB,rel_l2,max_abs
```

python/dump_trimul_inputs.py (GPU/Colab, 或記憶體夠大的電腦用 CPU 跑短序列,  第 1 週)

```
python dump_trimul_inputs.py (--uniprot ID | --pdb ID | --fasta 檔案 | --seq 序列) --out 資料夾
                                [--block 0~47] [--max-len N] [--pad 32] [--device cuda|cpu]
                                [--chunk-size 64] [--eval-channels 16]
```

例: `python dump_trimul_inputs.py --uniprot P0DTC2 --out ../data/spike`

沒有 GPU 時可以用 CPU 跑短序列 (較慢, 需要約 16 GB 以上的記憶體): `python dump_trimul_inputs.py --uniprot P69905 --device cpu --out ../data/hba` (P69905 是 142 個胺基酸的血紅素 α 鏈。) 輸出的 JSON 要看 `paper_group_C_check` 和 `rel_l2_grid`, 怎麼解讀見 10.4 節。

docs/manual/build_pdf.py (重新產生這份手冊的 PDF)

```
pip install markdown
python3 docs/manual/build_pdf.py      # 產生 docs/manual/manual.html 與 manual.pdf
```

A.10 目前為止實際執行過的指令 (時間軸)

#	指令	在哪裡	結果
1	<code>lspci \ grep -i -E "xilinx\ amd\ altera\ intel.*fpga"</code>	server63	空的
2	<code>which vivado vitis vitis_hls v++ xbutil</code>	server63	Vivado/Vitis/v++ 有; vitis_hls、xbutil 沒有
3	<code>ls /tools/Xilinx /opt/Xilinx /opt/xilinx</code>	server63	2025.2 DocNav flexlm xic
4	<code>xbutil examine</code>	server63	command not found
5	<code>which quartus aoc aocl、ls /opt/intelFPGA*</code>	server63	沒有 (不是 Intel FPGA)
6	<code>module avail \ grep ...</code>	server63	沒有結果
7	<code>lspci -d 10ee:</code>	server63	空的 → 沒有 FPGA 卡
8	<code>ls /opt/xilinx/platforms、find ... "*u55c"</code>	server63	沒有 platform
9	<code>which vitis-run g++ make git</code>	server63	都有
10	<code>git clone -b ... ForClaude.git</code>	server63	需要 GitHub 帳號 (private repo)
11	<code>cd ForClaude/fpga/trimul</code>	server63	
12	<code>make csim</code>	server63	(待確認是否 ALL PASS)
13	<code>make hls KERNEL=trimul_v2</code>	server63	失敗 3 次 (2025.2 路徑問題) → 修正後成功

#	指令	在哪裡	結果
14	<code>git pull</code>	server63	取得修正版
15	<code>head -n 60 ../trimul_v2_csynth.rpt</code>	server63	第一筆數據：177,029,185 cycles
16	<code>tmux → make hls-all</code>	server63	🟡 進行中

A.11 常見錯誤與排查

看到的訊息	意思	怎麼辦
<code>command not found</code>	沒裝，或環境沒載入	<code>which</code> 指令；試 <code>source ../settings64.sh</code> ；問管理員
No such file or directory	路徑打錯，或不在正確的資料夾	<code>pwd</code> 看自己在哪； <code>ls</code> 看檔案在不在；善用 Tab 補完
Permission denied	沒有權限	不要用 <code>sudo</code> 硬來，先問管理員
make: *** No rule to make target	不在 <code>fpga/trimul</code> 裡，或目標名稱打錯	<code>cd ~/ForClaude/fpga/trimul</code> ； <code>make help</code>
Username for 'https://github.com'	repo 是 private	改成 public，或用 fine-grained token
Your local changes would be overwritten	server 上的檔案被改過	見 A.4 節的 <code>git stash</code>
ERROR: [HLS ...]	HLS 失敗	<code>tail -n 30 hls_<kernel>.log</code> 截圖給 Claude
WARNING: [HLS ...]	通常可以忽略	只有同時出現 ERROR 才需要處理
No space left on device	硬碟滿了	<code>du -sh hls_prj build</code> ；確認結果已保存後 <code>make clean</code>
畫面卡住很久沒動	可能正常（合成很慢），也可能卡住	另開一個 tmux 視窗用 <code>top</code> 看 CPU 有沒有在跑

回報問題時請提供

1. 完整的指令
2. 錯誤前後 20~30 行的輸出（`tail -n 30 xxx.log`）
3. 你預期會發生什麼
4. 已經試過什麼

本章小結

- 指令 = 名稱 + 選項 + 參數；用 `|`、`>`、`&&` 串接；忘了就 `--help`。
 - 本專案的所有操作都可以用 `make 目標 變數=值` 完成，`make -n` 可以預覽不執行。
 - 現在最常用的四個指令：`git pull`、`make hls-all`、`tail -n 30 hls_*.log`、`make summary`。
 - 長時間工作用 `tmux`；刪除前先確認，不要在 server 上 `git push`。
-

自我檢測

讀完這一附錄，試著回答下面的問題（參考答案在附錄 C）：

1. `lspci | grep -i xilinx` 中的 `|` 做了什麼？`-i` 是什麼意思？
2. 想知道 `make hls KERNEL=trimul_v3` 會執行哪些指令、但不要真的執行，要怎麼下？
3. 合成要跑很久，SSH 又可能斷線，該怎麼做？斷線後怎麼回到原本的畫面？
4. 想合成 16×16 tile 的 v4，但不要覆蓋 v4 預設設定的報告，指令怎麼寫？
5. 執行 `make clean` 之前應該先做什麼？

附錄 B 是術語表：看到不懂的詞可以在那裡查。

附錄 C 術語表

依主題分類。每個詞附上：白話解釋、正式說明、在本專題哪裡出現。忘記某個詞的意思時，回來這裡查。

B.1 蛋白質與模型

術語	白話	說明	本專題
胺基酸 (amino acid)	珠鍊上的一顆珠子	蛋白質的基本單位，共 20 種，用一個英文字母表示	序列中的每個字母
序列長度 L	珠子的數量	蛋白質中胺基酸的個數	整份手冊的核心變數
PPM	蛋白質結構預測模型	Protein structure Prediction Model，例如 AlphaFold2、ESMFold	LightNobel 的研究對象
AlphaFold2	DeepMind 的蛋白質結構預測模型	2021 年發表，使用 MSA 與 Evoformer	第 1 章
ESMFold	Meta 的蛋白質結構預測模型	用 ESM-2 語言模型代替 MSA，結構較簡單	我們驗證算式的對象
ESM-2	蛋白質的「語言模型」	像讀文字一樣讀胺基酸序列，產生特徵	ESMFold 的第一部分
MSA	多序列比對	Multiple Sequence Alignment，把相似的蛋白質序列排在一起	AlphaFold2 的輸入，ESMFold 不需要
Evoformer / Folding trunk	模型的主體	由 48 個 block 組成，反覆更新兩種 representation	第 1 章 1.3 節
Structure module	輸出座標的部分	把特徵轉成 3D 原子座標	第 1 章 1.3 節
Sequence representation	每個胺基酸的特徵	形狀 $L \times 1024$ (ESMFold)	第 1 章
Pair representation	每一對胺基酸之間的關係	形狀 $L \times L \times 128$ ，隨 L^2 成長	本專題處理的資料
Triangle Multiplication	用「第三者」推論兩者關係	$z[i][j][c] = \sum_k a[i][k][c] \cdot b[j][k][c]$ (outgoing)	本專題實作的運算
Outgoing / Incoming	兩種三角形方向	outgoing 用 (i,k)、(j,k)；incoming 用 (k,i)、(k,j)	我們做 outgoing
Triangle Attention	三角形版本的 attention	中間值隨 L^3 成長，實作上需分塊；長序列時最花時間 (1,410 aa 佔 75.9%)	第 1 章 1.4 節 (未實作，未來工作)

術語	白話	說明	本專題
Token	一格資料	在 pair representation 中指一個 (i, j) 位置 (一對胺基酸) 的 128 個數字；在 sequence representation 中才是一個胺基酸	1.3 節；per-token 量化的單位
Channel (C)	每格的數字個數	特徵維度，ESMFold 的 pair 為 128	trimul.h 的 C = 128
LayerNorm	把一個 token 拉回標準範圍	減平均、除標準差，再乘 γ 加 β	1.5 節
Linear	重新混合特徵	$y = Wx + b$ ，每個輸出是所有輸入的加權和；128×128 的 W 是學出來的	1.5 節
Sigmoid	把任何數壓到 0 ~ 1	$1 \div (1 + e^{(-x)})$ ；當閘門用，也提供非線性	1.5 節
Gating (閘門)	決定每個值保留多少	內容 $\times$ sigmoid(閘門)，逐元素相乘	1.5 節
Mask	忽略補上去的位置	乘上 0 或 1	1.5 節
Einsum	用索引字符串描述的乘加	"ikc,jkc->ijc"：輸出沒有的索引 (k) 就加總；Triangle Multiplication 裡唯一跨 token 的運算	1.4、1.5 節
Residual	把修正量加回原本的值	$z = z + \text{修正量}$ ；讓深層模型訓練得起來，但會讓 z 越來越大	1.5 節
Recycling	把結果送回開頭再跑一次	ESMFold 的 trunk 預設最多重跑 4 次	1.2 節
Attention	依重要性加權的加總	先用 softmax 算出每個對象的權重 (總和為 1)，再加權加總	1.5 節
狀態 vs 中間值	要傳下去的 vs 只在一步內有用的	z 是狀態； z_{in} 、a、b、g 是中間值	1.2 節
Chunking	分批計算	一次只算一部分以降低記憶體峰值，代價是時間	第 3 章 3.7 節
HuggingFace	公開模型的平台	提供模型程式碼 (transformers) 與權重下載 (Hub)	dump_trimul_inputs.py
UniProt / PDB	公開的蛋白質資料庫	UniProt 存序列、PDB 存實驗測定的結構	下載測試序列
TM-score	預測結構有多像真實結構	0 ~ 1，越高越好；LightNobel 用它證明量化不傷準確度	5.1 節
CAMEO / CASP	蛋白質結構預測的標準測試集	CASP 是兩年一次的預測競賽；CAMEO 是持續更新的評測	5.1 節

術語	白話	說明	本專題
Distogram	胺基酸之間距離的分布	每一對胺基酸落在各個距離區間的機率；ESMFold 由 z 預測 64 個區間；LightNobel 觀察到 activation 有與之相關的模式	1.3、5.2 節

B.2 效能與記憶體概念

術語	白話	說明	本專題
Memory-bound	等搬資料	效能受限於記憶體頻寬或延遲，而非運算能力	v0、v2 的載入部分
Compute-bound	廚師忙不過來	效能受限於運算單元數量	v2 的 mac 部分
Weights (權重)	模型學到的參數	大小固定，與 L 無關	第 2 章 2.1 節
Activation	計算中產生的中間資料	推論時每一步算出來的資料 (pair representation 是最主要的一份)；隨 L 成長，權重則固定	1.3 節；LightNobel 要壓縮的對象
OOM	記憶體不夠	Out of Memory	長序列在 GPU 上的問題
FLOP	一次浮點運算	一次乘法或一次加法	一次乘加 = 2 FLOP
GFLOP/s	每秒十億次運算	效能單位	v2 約 7.3 GFLOP/s
頻寬 (bandwidth)	每秒能搬多少	GB/s	單一 512-bit 埠 19.2 GB/s；U55C HBM 約 460 GB/s
延遲 (latency)	等多久才開始有資料	從請求到第一筆資料的時間	v2 載入的 141 cycles 大多是延遲
算術強度 (AI)	每搬 1 byte 做幾次運算	$\text{FLOP} \div \text{byte}$	v0 0.25、v2 2、v4 約 7.8
Roofline 模型	判斷瓶頸的圖	可達效能 = $\min(\text{運算上限}, \text{AI} \times \text{頻寬})$	第 2 章 2.5 節
Ridge point (平衡點)	roofline 的轉折點	峰值運算 $\div$ 頻寬	判斷 memory/compute-bound
Tiling	分批搬、重複用	把計算切成小塊，讓搬進來的資料被重用多次	v1
資料重用 (data reuse)	同一個食材做多道菜	同一份資料被多次運算使用	tile 中每個 a 用 TJ 次

術語	白話	說明	本專題
Burst	一次請求連續拿很多筆	只付一次延遲	連續的 128 個 channel
Double buffering (ping-pong)	兩個料理台輪流用	一個 buffer 計算時，另一個載入下一批	v3
Little's Law	要多少資料在路上	在途資料量 = 頻寬 × 延遲	第 3 章 3.2 節
Amdahl's Law	改善的上限	加速 = $1 \div [(1-p) + p/s]$	v3 最多約 1.27 倍
Memory layout	資料在記憶體中的排列	決定能否連續讀取	[row][col][channel]
Stride	連續兩次存取的間距	stride 大 → 無法 burst	v0 的 stride = 128
Operator fusion (融合)	運算合併	中間值不寫回記憶體	3.7 節、第 3 章 3.5 節 (原則來自 LightNobel；本專題做設計與分析)
Recomputation (重算)	用計算換記憶體	不存中間值，需要時重算	3.7 節、3.6 節 (重算 g)

B.3 量化

術語	白話	說明	本專題
量化 (quantization)	把數字壓縮成較少的 bit	例如 FP32 (4 bytes) → INT8 (1 byte)	v4
FP32 / FP16 / BF16	浮點格式	32 / 16 / 16 bit 的浮點數	kernel 用 FP32
INT8	8 bit 整數	-128 ~ 127 (對稱量化用 -127 ~ 127)	v4
Scale (s)	量化的尺	$s = \max x \div 127$, $x \approx s \times q$	每個 token 一個
對稱量化	以 0 為中心	正負範圍相同，不需要 zero point	v4 的做法
Per-tensor	整份共用一把尺	一個 scale 給整個張量	誤差約 11%
Per-token	每格一把尺	每個 token 一個 scale	誤差約 1%
Per-channel	每個 channel 一把尺	每個 c 一個 scale；LLM 常用，但 PPM 的 channel 之間差異小，LightNobel 改用 per-token (kernel 未實作，真實資料的誤差表有比較)	4.2、8.4、10.4 節
Outlier	特別大的值	少數數值遠大於其他值	模擬資料中 1% 的 token ×30

術語	白話	說明	本專題
Fake quantization	量化再還原	用浮點數模擬量化誤差	Python 腳本評估誤差
rel_l2	整體相對誤差	$\ z - z_{\text{ref}}\ \div \ z_{\text{ref}}\ $	testbench、host
max_abs	最大單點誤差	$\max z - z_{\text{ref}} $	testbench、host
AAQ	LightNobel 的量化方法	Token-wise Adaptive Activation Quantization：每個 token 一個 scale；activation 依位置分 A、B、C 三組（INT8 + 4 outlier / INT4 + 4 outlier / INT4）；outlier 用 INT16、執行時 top-k 挑出	5.3 節
Inlier	一般大小的值	用低精度量化的部分	4.5、5.3 節
Top-k outlier	最大的 k 個值另外存	以較高精度另外保存，避免拖累其他值（LightNobel：A、B 組 k = 4，用 INT16）	4.5、5.3 節
3 σ 法則	判斷 outlier 的常用規則	和平均值相差超過 3 個標準差的值視為 outlier	10.4 節
INT4	4 bit 整數	-7 ~ 7（對稱量化）；兩個數擠在一個 byte，資料量是 INT8 的一半	10.4 節（v5）
Dequantization（反量化）	把整數還原成實數	$\hat{x} = s \times q$	4.1 節

B.4 FPGA 硬體

術語	白話	說明	本專題
FPGA	可以重新接線的晶片	Field-Programmable Gate Array	U55C
ASIC	專用晶片	做好就不能改，最快最省電	LightNobel 的加速器
HW/SW co-design	軟硬體一起設計	演算法配合硬體、硬體配合演算法	2.7 節、第 5、6 章
位元拆解乘法	大乘法拆成小乘法	把數字切成 4-bit 小塊，每對小塊相乘再移位相加；RMPU 的核心（源自 Bit Fusion）	6.4 節
Sign extension（符號延伸）	補正負號	把有號數擴寬時，用最高位補滿；二補數拆塊時只有最高位塊是有號的	6.4.3 節
加法樹（adder tree）	一層層兩兩相加	N 個數用 $\log_2 N$ 層加法器加總；取不同層的結果可支援不同運算	6.4 節
SIMD	一個指令、多筆資料	很多條 lane 同時做同一個運算；VVPV 有 128 條	6.5 節

術語	白話	說明	本專題
Bitonic sort (雙調排序)	固定連線的排序網路	由比較交換器組成、步驟與資料無關，適合硬體平行；128 個數 28 步	6.5.2 節
線上 softmax	邊算邊修正的 softmax	維護目前最大值與總和，新資料來時重新縮放，不需存整個 score matrix (FlashAttention 的核心)	6.6 節
Crossbar	任意輸入接任意輸出的交換網路	開關數 $\propto$ 埠數 ² ，LightNobel 70% 面積	6.7 節
Scratchpad	由設計者管理的晶片內記憶體	不像 cache 由硬體自動決定內容，可預測、省面積	6.7 節
Weight-stationary	權重固定不動的資料流	權重留在晶片內，輸入串流通過；適合權重小、重用多	6.2 節
RMPU / VVPU / Token Aligner	LightNobel 的硬體單元	RMPU：把數字切成 4-bit 小塊、不用反量化就能做多精度矩陣運算/VVPU：128 條 SIMD 的向量單元，做 LayerNorm、Softmax、執行時量化、top-k/Token Aligner：把格式不同的 token 排齊	第 5、6 章
ISCA	計算機架構頂尖會議	International Symposium on Computer Architecture	LightNobel 發表處
U55C	我們的 FPGA 卡	AMD Alveo U55C，晶片 xcu55c-fsvh2892-2L-e	整個專題
Alveo	AMD 資料中心 FPGA 卡系列	插在伺服器的 PCIe 插槽	U55C 屬於此系列
LUT	小真值表	6 輸入，可實現任意邏輯	報告的 LUT 用量
FF (Flip-Flop)	1 bit 暫存器	每個時脈更新	報告的 FF 用量
DSP	硬體乘法器	DSP48E2，27×18 bit 乘法 + 累加	浮點乘法用
BRAM	小塊晶片內記憶體	36 Kb (可拆成兩個 18 Kb)，2 個讀寫埠	acc、la、lb
URAM	大塊晶片內記憶體	288 Kb	大 tile 時可能用到
HBM	高頻寬記憶體	High Bandwidth Memory，堆疊在晶片旁	U55C 有 16 GB
Pseudo-channel (PC)	HBM 的一條通道	U55C 有 32 個，每個 512 MB	cfg 中的 <code>HBM[0:7]</code>
PCIe	主機和卡之間的連線	資料從 CPU 記憶體搬到 HBM 走這裡	XRT 的 sync
時脈 (clock)	電路的節拍	300 MHz = 每 3.33 ns 一拍	<code>create_clock - period 3.33</code>
Cycle	一拍	一個時脈週期	報告的 latency 單位

術語	白話	說明	本專題
Bitstream	FPGA 的設定檔	決定每個元件和連線的設定	包在 .xclbin 中
廠商代號 10ee	Xilinx 的 PCI ID	<code>lspci -d 10ee</code> : 可列出 Xilinx 裝置	確認 server63 沒有卡

B.5 HLS 與硬體設計

術語	白話	說明	本專題
HLS	C++ 轉電路	High-Level Synthesis	Vitis HLS
RTL	電路的描述	Register Transfer Level, 用 Verilog/VHDL 撰寫	HLS 的輸出
Verilog / VHDL	硬體描述語言	描述電路結構與行為	我們不需要手寫
Kernel	在 FPGA 上跑的函式	加速的核心運算	<code>trimul_v0 ~ v4</code>
Pragma	給 HLS 的指示	<code>#pragma HLS ...</code> , 不影響功能、只影響實作	第 7 章 7.3 節
Pipeline	流水線	多個迭代重疊執行	幾乎所有內層迴圈
II (Initiation Interval)	流水線的節奏	連續兩次迭代開始的間隔, 1 最好	mac 迴圈 II = 1
Iteration latency	一次迭代的長度	一次迭代從頭到尾的 cycle 數	loop_k 為 671
Pipeline depth	流水線的長度	一次迭代需要走過的級數	mac 約 14
Loop-carried dependency	下一輪要等這一輪	跨迭代的資料相依, 會拉高 II	v0 的 acc
Unroll (展開)	把迴圈複製成多份硬體	平行執行	v2 的 16 lane
Array partition	把陣列拆開	讓多個元素能同時存取	<code>ARRAY_PARTITION dim=2 complete</code>
Scheduling (排程)	決定何時做	把運算分配到 cycle	第 7 章 7.3 節
Binding (綁定)	決定用誰做	把運算分配到硬體單元	第 7 章 7.3 節
FSM	狀態機	控制電路目前在做哪一步	HLS 自動產生
AXI4 / m_axi	讀寫記憶體介面	kernel 主動讀寫 HBM 的標準介面	<code>INTERFACE m_axi</code>
AXI4-Lite / s_axilite	設定參數的介面	host 寫入參數和 start 訊號	L、位址
Bundle	AXI 埠的名稱	相同 bundle 共用一個埠	gmem0、gmem1、gmem2

術語	白話	說明	本專題
Trip count	迴圈次數	告訴 HLS 以便估計 latency	<code>L_TRIP = 256</code>
Timing (时序)	電路能否在一拍內完成	Estimated 需小於 Target	2.431 ns < 3.33 ns
Synthesis (合成)	把設計轉成元件	HLS 合成 (C++→RTL) 或邏輯合成 (RTL→netlist)	兩種都有
Place & Route	擺放與繞線	決定元件位置與連線	Vivado，耗時最久
Netlist	元件清單 + 連線	邏輯合成的輸出	Vivado 內部

B.6 AMD 工具鏈

術語	白話	說明	本專題
Vitis	AMD 的加速應用開發環境	包含 HLS、 <code>v++</code> 、 <code>vitis-run</code> 等	2025.2
Vitis HLS	C++ → RTL 工具	2024.2 起以 <code>vitis-run --mode hls</code> 執行	<code>make hls</code>
Vivado	RTL → bitstream 工具	合成、擺放、繞線、產生 bitstream	<code>v++ -l</code> 內部呼叫
<code>v++</code>	Vitis 的編譯器	<code>-c</code> 產生 <code>.xo</code> ； <code>-l</code> 產生 <code>.xclbin</code>	<code>make xclbin</code>
<code>vitis-run</code>	執行 Vitis 工具的指令	<code>--mode hls --tcl</code> 執行 HLS 腳本	<code>make hls</code>
<code>.xo</code>	編譯好的 kernel	RTL + kernel 描述	<code>v++ -c</code> 輸出
<code>.xclbin</code>	可以燒進 FPGA 的檔案	bitstream + metadata	<code>v++ -l</code> 輸出
Platform / Shell	卡上的基礎設施	PCIe、DMA、HBM 控制器等，由 AMD 提供	<code>xilinx_u55c_gen3x16_xdma_3_202210_1</code>
Static / Dynamic region	固定區／可變區	shell 在固定區，kernel 在可變區	第 7 章 7.6 節
XRT	控制卡片的軟體	Xilinx Runtime：驅動程式 + 函式庫	<code>host.cpp</code>
<code>xbutil</code>	XRT 的管理工具	查看卡片狀態、測試	<code>xbutil examine</code>
Buffer object (bo)	卡上的一塊記憶體	XRT 在 HBM 上配置的空間	<code>xrt::bo</code>
Connectivity (<code>.cfg</code>)	接線設定	指定 kernel 參數接到哪些 HBM PC	<code>cfg/*.cfg</code>

術語	白話	說明	本專題
sw_emu / hw_emu / hw	三種執行目標	軟體模擬／硬體模擬／實際上板	TARGET=
csim / cosim	兩種 HLS 模擬	C 模擬／C 與 RTL 協同模擬	第 7 章 7.4 節
xsim	Vivado 的 RTL 模擬器	cosim 與 hw_emu 使用	未使用
csynth.rpt / csynth.xml	HLS 報告	文字版／機器可讀版	hls_summary.py 讀 xml
Tcl	工具的腳本語言	AMD 工具用 Tcl 撰寫自動化流程	run_hls.tcl
flexlm	授權管理	商用軟體的授權伺服器	/tools/Xilinx/flexlm

B.7 軟體與開發工具

術語	白話	說明	本專題
g++	C++ 編譯器	GNU 的開源編譯器	make csim
clang / LLVM	另一套編譯器	Vitis HLS 的前端使用 clang	HLS 日誌
Make / Makefile	指令捷徑	把多個步驟寫成規則	fpga/trimul/Makefile
Testbench	測試程式	產生輸入、呼叫 kernel、檢查輸出	tb/tb.cpp
Host 程式	CPU 端的控制程式	準備資料、控制 FPGA	host/host.cpp
git clone / pull / push	下載／更新／上傳	Git 版本控制指令	取得程式碼
分支 (branch)	同一專案的不同版本	修改先放在分支，再合併進 main	claude/lightnobel-...
PR (Pull Request)	合併請求	請求把分支合併進 main	PR #1
Personal Access Token	GitHub 的鑰匙	代替密碼的存取權杖	clone private repo 時需要
tmux	不會斷線的終端機	SSH 斷線後程式繼續跑	跑 make hls-all
PyTorch	深度學習框架	張量運算	Python 腳本
transformers	預訓練模型函式庫	HuggingFace 的開源函式庫	EsmForProteinFolding
Colab	免費的雲端 GPU	Google 提供的筆記本環境	沒有 GPU 時的替代方案

B.8 架構評估與設計方法

術語	白話	說明	本專題
Architectural simulation	用模型預測硬體表現	在做出硬體前，用分析模型、cycle 級模擬或合成報告評估設計	9.1 節
分析模型 (analytical model)	用公式算 cycle	把迴圈次數、overhead 寫成公式，快速預測	make model 、 9.2 節
校正 (calibration)	用實測數字定常數	從 HLS 報告反推公式裡的 overhead、pipeline 深度	9.2 節 (77、14)
驗證 (validation)	檢查模型準不準	用沒拿來校正的資料比對預測誤差	9.2 節
DSE (設計空間探索)	試遍所有設定	掃過 tile、平行度、精度等組合，找最佳設計	make model 、 9.3 節
Pareto 前緣	無法同時更好的設計	找不到在所有指標都不輸、且至少一項更好的其他設計	9.3 節
Ablation	拆掉一招看差多少	逐一加入或移除手法，量出每一招的貢獻	v0 ~ v4
Producer / Consumer	產生者 / 使用者	產生資料的運算與使用資料的運算	3.5 節
Reuse distance	產生到使用的距離	距離越短，越容易留在晶片內	3.5 節
落地 (materialize)	中間值寫回記憶體	融合的目的就是避免中間值落地	3.5 節
Epilogue	kernel 的收尾階段	主計算之後的處理 (LayerNorm、linear、gating)	10.3 節
Welford 演算法	一次掃完算平均與變異數	數值穩定的線上統計方法	10.4 節
Tile sweep	掃過不同 tile 大小	量測 tile 大小的 trade-off	make hls-sweep
容量模型	最多能處理多長	最大 $L = \sqrt{\text{記憶體} \div \text{每個 } L^2 \text{ 的 bytes}}$	9.4 節

本附錄小結

遇到不懂的詞，先在這裡查白話解釋，再回到「本專題」欄位指出的章節看完整說明。

附錄 C 自我檢測參考答案

先自己想過再看答案。答不出來的題目，回到括號裡標示的章節重讀。

第 1 章 蛋白質結構預測與 pair representation

1. $\mathbf{z} = \mathbf{a} \times \mathbf{b}^T$: $\mathbf{a}$ 的第 i 列和 $\mathbf{b}$ 的第 j 列做內積，就是 $\mathbf{z}$ 的 (i, j) 。 (1.4)
2. Triangle Attention (1,410 aa 時佔 75.9%)，因為它的 score matrix 是 L^3 。先做 Triangle Multiplication 是因為它結構單純（矩陣乘法、沒有 softmax）、容易驗證，而且 tiling、量化、分析模型等方法都能直接沿用到 Triangle Attention；後者列為未來工作。 (1.4、10.1)
3. pair representation 的 token 是一格 (i, j) ，也就是一對胺基酸的 128 個數；sequence representation 的 token 才是一個胺基酸（1024 個數）。pair representation 有 $L \times L$ 個 token。 (1.3)
4. 不是。輸入是一條完整已知的胺基酸序列，輸出是摺疊後每個原子的 3D 座標；每條序列從頭算一次，沒有「預測下一個」的過程。 (1.2)
5. 狀態是 $\mathbf{z}$ （更新後要傳給下一步）；中間值是 $\mathbf{z}_{\text{in}}$ 、 $\mathbf{a}$ 、 $\mathbf{b}$ 、 $\mathbf{g}$ 、 einsum 的結果 $\mathbf{x}$ 、修正量。中間值只在這一步有用，但每一份都是 $L \times L \times 128$ 、和 $\mathbf{z}$ 一樣大，一般寫法會整份寫回記憶體，同時存在約 7 份。
(1.2、2.1)
6. $0 \sim 1$ 。gating 時，sigmoid 把一個 linear 的結果變成「要讓多少通過」的比例，再和另一個 linear 的結果逐元素相乘，決定每個特徵保留多少。 (1.5)
7. einsum 。其他運算（LayerNorm、linear、sigmoid、gating、mask）都只用這個 token 自己的 128 個數，可以一個 token 在晶片內做完； einsum 要讀整列 i 、整列 j 的 token，所以 $\mathbf{a}$ 、 $\mathbf{b}$ 必須先寫出去，它就是融合的邊界。 (1.5、3.5)
8. 兩者功能不同：LayerNorm 一起看整個 token、保留相對關係與正負號，把振幅調到約 1（像 AGC）；sigmoid 逐個數處理、只輸出 $0 \sim 1$ ，而且只用在閘門那條路。更重要的是 sigmoid 會飽和：沒有 LayerNorm 時，大的數（例如 80、85、90）經過 sigmoid 全部變成 1，資訊消失。 (1.5)
9. $\mathbf{a}[i][k]$ 是 $i \rightarrow k$ 這條邊的訊息、 $\mathbf{b}[j][k]$ 是 $j \rightarrow k$ 這條邊的訊息， einsum 把所有第三者 k 的證據配對加總。 $\mathbf{a}[i][k]$ 會被 L 個不同的輸出 $\mathbf{x}[i][\cdot]$ 用到，而且分散在整個計算中，產生當下無法用完，只能先存起來； $\mathbf{z}_{\text{in}}$ 只在算同一個 token 時用一次，可以留在晶片內。 (1.5)
10. $\mathbf{z}$ 是 pair representation： $\mathbf{z}[i][j]$ 是描述胺基酸 i 和 j 之間關係的 128 個特徵值； $\mathbf{s}$ 則描述每個胺基酸自己（ $L \times 1024$ ）。 $\mathbf{z}$ 一開始只有序列位置的資訊，每個 block 從 $\mathbf{s}$ 補資訊、用三角形推理修正；最後被拿去預測每一對胺基酸的距離分布（distogram），structure module 用它排出 3D 形狀。 (1.3)

第 2 章 記憶體牆：容量、頻寬與效能分析

1. $512 \text{ MB} \rightarrow 2 \text{ GB}$ （4 倍）。大小是 $L^2 \times 128 \times 4 \text{ bytes}$ ， L 變 2 倍， L^2 變 4 倍。 (2.1)

2. 權重大小固定 (ESMFold 的 FP16 權重約 7.90 GB) , 和 L 無關; activation 隨 L^2 (pair representation) 甚至 L^3 (Triangle Attention 的中間值) 成長。 L 一大, activation 就遠超過權重: $L = 2,034$ 時已經是權重的 24 倍 (144 GB) 。 (2.1)
 3. 0.25 FLOP/byte : 每做 2 次運算要讀 8 bytes。以 GPU 約 10 FLOP/byte 的平衡點來看, 運算單元大約 97% 的時間都在等資料。 (2.2)
 4. 沒有重疊: $T \approx T_{\text{compute}} + T_{\text{memory}}$; 完全重疊: $T \approx \max(T_{\text{compute}}, T_{\text{memory}})$ 。 Double buffering 把「相加」變成「取最大值」。 (2.4、3.3)
 5. tiling $\rightarrow$ 資料流; INT8 量化 $\rightarrow$ 資料表示; 寬位元介面 $\rightarrow$ 硬體結構。 (2.7)
-

第 3 章 資料流優化

1. $v0$ 讀取量 $2 \cdot L^3 \cdot C$; tiling 後 $L^3 \cdot C \cdot (1/16 + 1/16) = L^3 \cdot C / 8$, 是 $v0$ 的 $1/16$ 。算術強度 $16 \cdot 16 \div (2 \cdot 32) = 4$ FLOP/byte。 (3.1)
 2. 每輪最多從 671 降到約 530 cycles, 約快 1.27 倍。因為載入只佔 21%, 就算完全藏起來也只能省下這 21% (Amdahl's Law) 。 (3.3、2.6)
 3. 頻寬是「每秒能搬多少」, 延遲是「從請求到拿到第一筆要等多久」。寬位元提高頻寬 (每 cycle 搬 16 個 float) ; burst 減少延遲的影響 (一次請求連續拿很多筆, 延遲只付一次) 。 (3.2)
 4. LayerNorm、linear、sigmoid、gating、量化都是「每個 token 只用自己的 128 個數」的運算, 一個 token 可以在晶片內一路處理完; 只有 einsum 需要跨 token 的資料, 才必須把 a、b 寫出去。 (3.5)
 5. 比例是 $128 \div L$, $L = 512$ 時約 25%。 (3.6)
-

第 4 章 量化

1. per-tensor 整份資料共用一個 scale ; per-token 每個 token 各自一個 scale。有 outlier 時, per-tensor 的 scale 被 outlier 撐大, 一般的數值只能用到很少幾個整數, 誤差變大。模擬結果: per-token 約 1%, per-tensor 約 11%。 (4.2)
 2. 不會。scale 會和資料一起存下來, 使用時乘回去 ($\hat{x} = s \times q$) , 數值回到原本的大小; 量化只是換一種較省空間的記法, 不像 LayerNorm 會改變數值的意義。 (4.1)
 3. 用浮點數 (FP16 或 FP32 ; 我們的 v4 用 FP32) 。浮點數像科學記號, 只存有效數字和次方: $0.000787 = 1.612 \times 2^{-11}$, 小數點後有幾個 0 只影響次方, 而次方只佔幾個 bit。每 128 個數才一個尺, 只佔 1~3%。 (4.3)
 4. 省下「把 FP32 的 a、b 寫出去、再讀回來量化」的兩趟搬運: a 的 128 個數剛算完還在暫存器裡, 就順便算 scale、轉成 INT8。這就是 LightNobel VVPU 的執行時量化。 (10.4)
 5. $\sqrt{2} \approx 1.41$ 倍。記憶體和 L^2 成正比, 所以 L 和記憶體的平方根成正比。 (4.1、9.4)
 6. $-5 = 1111\ 1011$ ($5 = 0000\ 0101$ 反轉再加 1) 。FP32 = 1 bit 正負號 + 8 bit 次方 (偏移 127) + 23 bit 有效數字 (開頭的 1 不存) 。浮點乘法要次方相加、有效數字相乘、再正規化並處理特殊值; 整數乘法直接相乘, 所以便宜很多。 (4.1)
-

第 5 章 LightNobel 的觀察、演算法與成果

1. ① token-wise 自適應量化：每個 token 一個 scale，activation 依位置分 A、B、C 三組給不同精度（INT8 / INT4）；② outlier 另外存：執行時用 top-k 挑出、以 INT16 存放，資料格式對硬體友善；③ 專用硬體：RMPU（4-bit 小塊、多精度）、VVPU（執行時量化、top-k）、Token Aligner 有效率地處理不同格式的 token。（5.1、5.3 節，第 6 章）
 2. 他們觀察到同一個 token 的 channel 之間差異小、token 之間差異大，outlier 也集中在特定 token；以 token 為單位，既能讓每個 token 用適合自己的 scale，又不需要像 per-element 那樣存大量 scale。（5.2 節）
 3. a、b 是 linear 之後的值，依定義屬於 C 組（平均 outlier 少於 1 個），論文對 C 組用 INT4、不處理 outlier。所以我們的 INT8 是保守的選擇，INT4（v5）是下一步，但要先用真實資料確認。（5.3、10.4 節）
 4. ① 量化：每個數變成 4~8 bit，資料量約變成 1/1.65（Table 1）；② 不存中間值：中間值算完就用掉，尤其 Triangle Attention 不存 L^3 的 score matrix。讓峰值降到上百倍（和不分塊的 GPU 比）的是 ②。（5.4）
-

第 6 章 LightNobel 的硬體架構

1. 8 個（INT8 拆成 2 塊、INT16 拆成 4 塊， $2 \times 4 = 8$ ）。 $A \times B = \sum_i \sum_j a_i \cdot b_j \cdot 2^{4(i+j)}$ ，每個 $a_i \cdot b_j$ 是一個 4-bit 乘法，乘 2 的次方只是移位。（6.4.2）
 2. $-93 = 0xA3$ ：高位塊 0xA 當有號（-6），低位塊 0x3 當無號（3）， $-6 \times 16 + 3 = -93$ 。有號塊是 -8~7、無號塊是 0~15，要同時裝得下需要 5 bit（-16~15）。（6.4.3）
 3. PPM 的內積大多需要 4 個 Lane（全 INT4）或 5 個 Lane（有 INT16 outlier）。20 是 4 和 5 的最小公倍數，兩種情況都能把 Lane 用滿（5 個或 4 個 token 同時算）。（6.4.5）
 4. inlier 要乘尺、outlier 不用。4 個 Lane 時全是 inlier，加完再乘一次尺；5 個 Lane 時要先把 inlier 加起來乘尺，再加上 outlier 的結果。（6.4.6）
 5. 連線固定、步驟和資料無關，所有比較器可以同時動作，延遲固定、容易排進管線。128 個數要 $7 \times 8 \div 2 = 28$ 步。（6.5.2）
 6. 只維護目前的最大值、總和和輸出累加；每來一批新分數，用新的最大值把舊的總和與輸出重新縮放，分數用完就丟。（6.6）
 7. crossbar 的開關數和埠數的平方成正比，而 LightNobel 需要大量資料在單元之間任意移動。啟示：資料移動比運算貴；如果移動模式固定，應改用較簡單的網路。（6.7）
 8. 32 個 RMPU 時，運算能力已足以處理記憶體供應的所有資料，再加只會閒置，這正是 roofline 的平衡點。（6.8）
-

第 7 章 FPGA 與工具原理

1. LUT 本質上是一塊 64 格的小記憶體，6 個輸入當作地址、存在裡面的 0/1 就是輸出，所以寫入不同的內容就能變成任何 6 輸入的邏輯函數。（7.1）

2. Π 是連續兩次迭代開始的間隔。v0 的 `acc += a*b` 下一次要等上一次的浮點加法做完 (loop-carried dependency)，所以 Π 至少等於加法的延遲。(7.3)
 3. 一塊 BRAM 只有 2 個讀寫埠，一個 cycle 最多讀寫 2 次；v2 每個 cycle 要同時讀寫 16 個值，必須把陣列切成 16 塊，讓每塊各自存取。(7.3)
 4. 代表演算法 (功能) 是對的；不代表時序正確、也不代表 pragma 宣告 (例如 `DEPENDENCE`) 沒有錯。要靠 cosim 或 hw_emu 驗證。(7.4)
 5. `.xo` : kernel 編譯後的結果 (RTL + 參數描述)；`.xclbin` : 可以燒進 FPGA 的檔案 (bitstream + metadata)；platform : 卡上預先做好的基礎設施 (PCIe、DMA、HBM 控制器)，kernel 要接在它上面。(7.6)
-

第 8 章 實作：v0~v4 從程式碼到報告

1. 同一個 token 的 128 個 channel 在記憶體中是連續的，一次 burst 就能讀完，而且剛好是 8 個 512-bit word。這也和 PyTorch 的排列一致，真實資料不用轉換。(8.3)
 2. `rel_l2 = ||z - z_ref|| ÷ ||z_ref||`。v4 使用量化，本來就會有誤差，所以只檢查誤差在合理範圍內 (功能正確)，而不是要求和浮點結果一樣。(8.5)
 3. 專案名稱含有 `/` (`hls_prj/trimul_v2`，在目前資料夾下兩層)，但 2025.2 改寫來源路徑時假設專案只在下一層，算出錯的 `../kernels/...`。修法是先 `cd hls_prj`，再建立名稱不含 `/` 的專案。(8.10)
 4. 載入 a、b 141 (兩個 port 並行，只算一次) + 計算 526 (512 + pipeline 深度 14) + 額外 4 = 671。(8.11)
 5. `bo.sync(XCL_B0_SYNC_B0_T0_DEVICE)` : 透過 PCIe DMA 把資料從主機記憶體複製到 HBM。(8.6、7.7)
-

第 9 章 評估方法

1. 同一份資料校正又驗證，當然會很準，無法證明模型能預測沒看過的情況。應該用 v2 8×8 校正，去預測 16×16、32×32，以及 v3、v4 的報告，看誤差是否在 10% 以內。(9.2)
 2. 找不到另一個設計在所有指標都一樣好、而且至少一項更好。(9.3)
 3. $\sqrt{(3,584 \div 1,288)} \approx \sqrt{2.78} \approx 1.67$ 倍 (約 2,190 → 3,650)。(9.4)
 4. 論文的比較基準 (ESMFold) 是 FP16。用 FP32 當基準，原始資料大了一倍，量化「看起來」省更多，會高估效果：同樣的融合 + INT8，FP32 基準是 1.67 倍，FP16 基準只有 1.52 倍。(9.4、2.1)
-

第 10 章 專題策略

1. 繼承：問題定義、以 ESMFold 為對象、token-wise 量化、管線化與執行時量化 (中間值不落地) 的原則、以峰值記憶體與序列長度為指標、模擬器 + 交叉驗證的評估方法。新增：FPGA 上的實作、v0~v5 逐步

- ablation、真實資料驗證量化假設、完整 Triangle Multiplication 的融合設計（含重算 g）、分析模型與設計空間探索。（10.1 節）
2. 平台（ASIC 模擬 vs FPGA）和範圍（整個模型 vs 一個運算）都不同，直接比較不公平；而且融合與執行時量化 LightNobel 都已經做了。較好的說法：「以 LightNobel 的原則為基礎，在現成硬體上實作其中一個運算，並量化每一招各自的貢獻。」（10.1 節）
 3. INT4 的前提是「a、b 屬於論文的 C 組（outlier 很少）」，這個假設要用真實資料確認，否則 v5 可能白做。要看 `paper_group_C_check`（平均絕對值、每個 token 的 3σ outlier 數，和論文的 3.85、0.64 比）與 `rel_l2_grid`（INT8/INT4 $\times$ token/channel/tensor 的誤差）。（10.4）
-

第 11 章 研究方法

1. 問題是什麼、他們觀察到什麼、解法是什麼、怎麼證明有效。（11.2）
 2. 這樣 v4 和 v2 之間只差「量化」一件事，才能把效果歸因給量化；如果從 v3 改，差異就混進了 double buffering。（11.5）
 3. 預測逼你先想清楚「為什麼會有效」；量測後比對預測，吻合代表理解正確，不吻合就是新的發現（例如 v3 應該快約 1.27 倍）。（11.5、3.3）
 4. 代表假設錯了（例如路徑 bug 改成絕對路徑後仍失敗），要回頭找新的線索，例如比較日誌中「加入時的路徑」和「合成時的路徑」。（11.9）
 5. ① 瓶頸的數據與量測條件：我們以為 Triangle Multiplication 最花時間，其實長序列時是 Triangle Attention（Fig. 3）。② 比較基準：我們用 FP32，論文是 FP16（Table 1）。③ 論文已經做了什麼：我們以為融合是新角度，其實 RMPU $\rightarrow$ VVPU 管線已經做到（Sec. 5）。（11.11）
-

附錄 A 指令手冊

1. | 把 `lspci` 的輸出交給 `grep`，只留下含 `xilinx` 的行；`-i` 表示不分大小寫。（A.1）
2. `make -n hls KERNEL=trimul_v3`。（A.6）
3. 在 `tmux` 裡執行；按 `Ctrl + B` 再按 `D` 可以暫時離開，重新登入後用 `tmux attach` 回來。（A.5）
4. `make hls KERNEL=trimul_v4 DEFS="-DTILE_I=16 -DTILE_J=16" TAG=trimul_v4_t16`，或 `make hls-sweep SWEEP_KERNEL=trimul_v4 TILES="16"`。（A.6）
5. 把要保留的結果（例如 `hls_summary.csv`、報告）複製到別的地方，因為 `make clean` 會刪除 `hls_prj/`、`log` 和 `csv`。（A.6）